

TITAN

THE RISE OF BIG GRAPH DATA



MARKO A. RODRIGUEZ
MATTHIAS BROECHELER



AURELIUS

[HTTP://THINKAURELIUS.COM](http://thinkaurelius.com)

ABSTRACT



A graph is a data structure composed of vertices/dots and edges/lines. A graph database is a software system used to persist and process graphs. The common conception in today's database community is that there is a tradeoff between the scale of data and the complexity/interlinking of data. To challenge this understanding, Aurelius has developed Titan under the liberal Apache 2 license. Titan supports both the size of modern data and the modeling power of graphs to usher in the era of Big Graph Data. Novel techniques in edge compression, data layout, and vertex-centric indices that exploit significant orders are used to facilitate the representation and processing of a single atomic graph structure across a multi-machine cluster. To ensure ease of adoption by the graph community, Titan natively implements the TinkerPop 2 Blueprints API. This presentation will review the graph landscape, Titan's techniques for scale by distribution, and a collection of satellite graph technologies to be released by Aurelius in the coming summer months of 2012.

SPEAKER BIOGRAPHIES



Dr. Marko A. Rodriguez is the founder of the graph consulting firm Aurelius. He has focused his academic and commercial career on the theoretical and applied aspects of graphs. Marko is a cofounder of TinkerPop and the primary developer of the Gremlin graph traversal language.



Dr. Matthias Broecheler has been researching and developing large-scale graph database systems for many years in both academia and in his role as a cofounder of the Aurelius graph consulting firm. He is the primary developer of the distributed graph database Titan. Matthias focuses most of his time and effort on novel OLTP and OLAP graph processing solutions.

SPONSORS

The Pearson logo consists of the word "PEARSON" in white, uppercase, sans-serif font, centered within a solid blue rectangular background.

PEARSON

As the leading education services company, Pearson is serious about evolving how the world learns. We apply our deep education experience and research, invest in innovative technologies, and promote collaboration throughout the education ecosystem. Real change is our commitment and its results are delivered through connecting capabilities to create actionable, scalable solutions that improve access, affordability, and achievement.



AURELIUS

Aurelius is a team of software engineers and scientists committed to applying graph theory and network science to problems in numerous domains. Aurelius develops the theory and technology whereby graphs can be used to model, understand, predict, and influence the behavior of complex, interrelated social, economic, and physical networks.

The Jive logo features the word "jive" in a bold, lowercase, sans-serif font. The letters are black, and the "j" has a distinctive hook that extends downwards.

Jive is the pioneer and world's leading provider of social business solutions. Our products apply powerful technology that helps people connect, communicate and collaborate to get more work done and solve their biggest business challenges. Millions of users and many of the world's most successful companies rely on Jive day in and day out to get work done, serve their customers and stay ahead of their competitors.

OUTLINE

1. THE GRAPH LANDSCAPE

- An introduction to graph computing.
Graph technologies on the market today.



2. INTRODUCTION TO TITAN

- Getting up and running with Titan.
Titan's techniques for scalability.



3. THE FUTURE OF AURELIUS

- Satellite technologies and the OLAP story.
The graph landscape reprise.



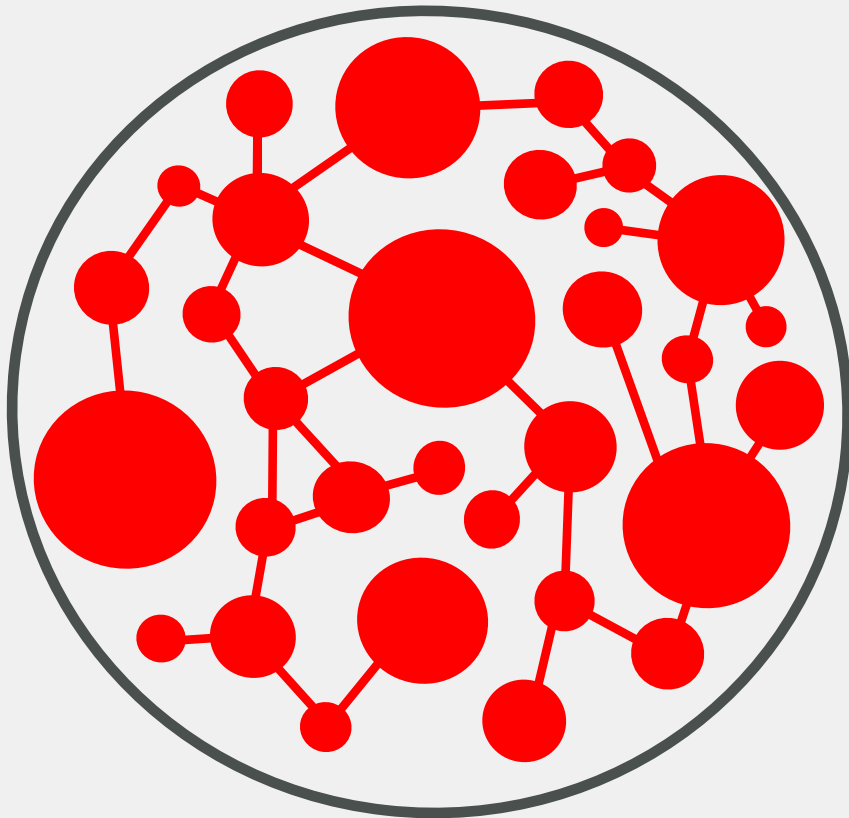
PART 1: THE GRAPH LANDSCAPE



MARKO A. RODRIGUEZ



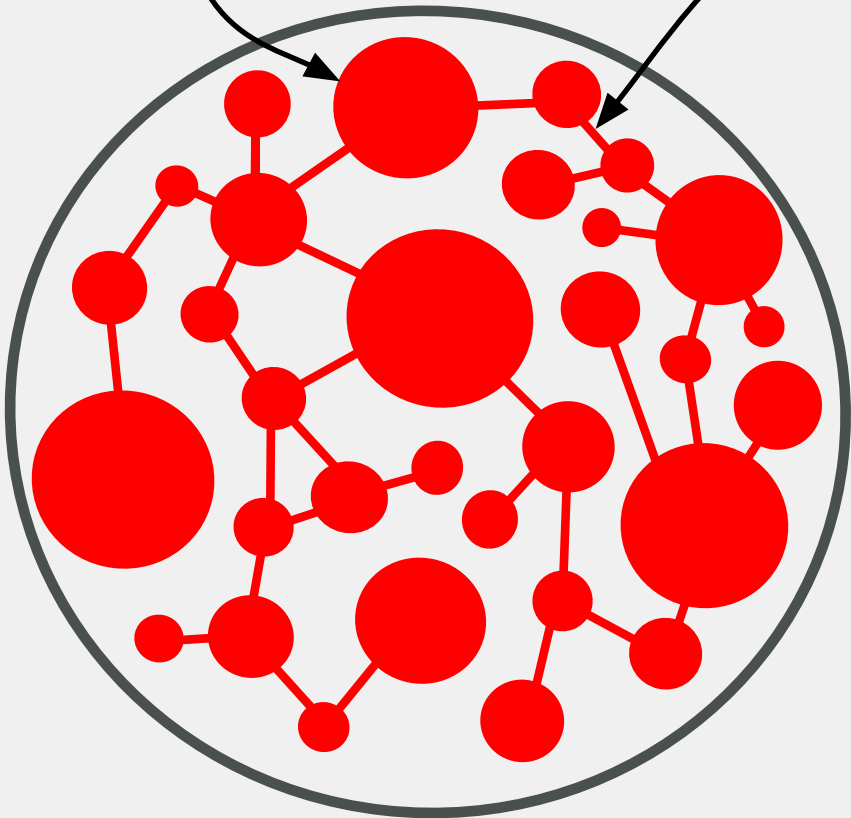
GRAPH



VERTEX

EDGE

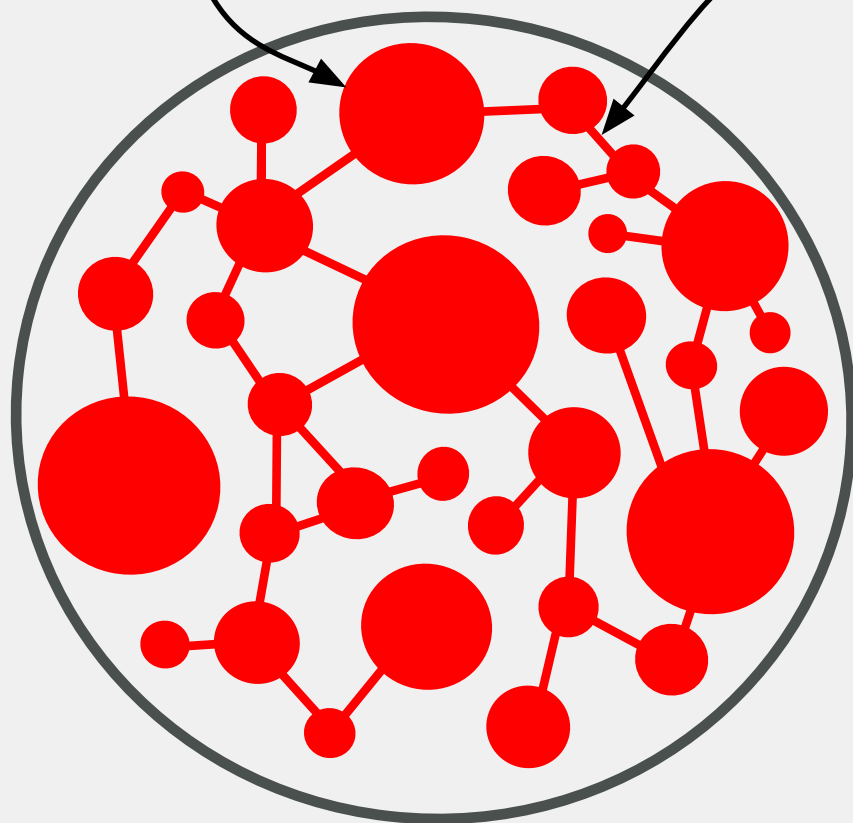
GRAPH



VERTEX

EDGE

GRAPH

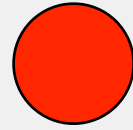


$$G = (V, E)$$

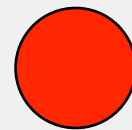
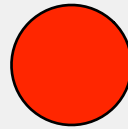
Graph Vertices Edges

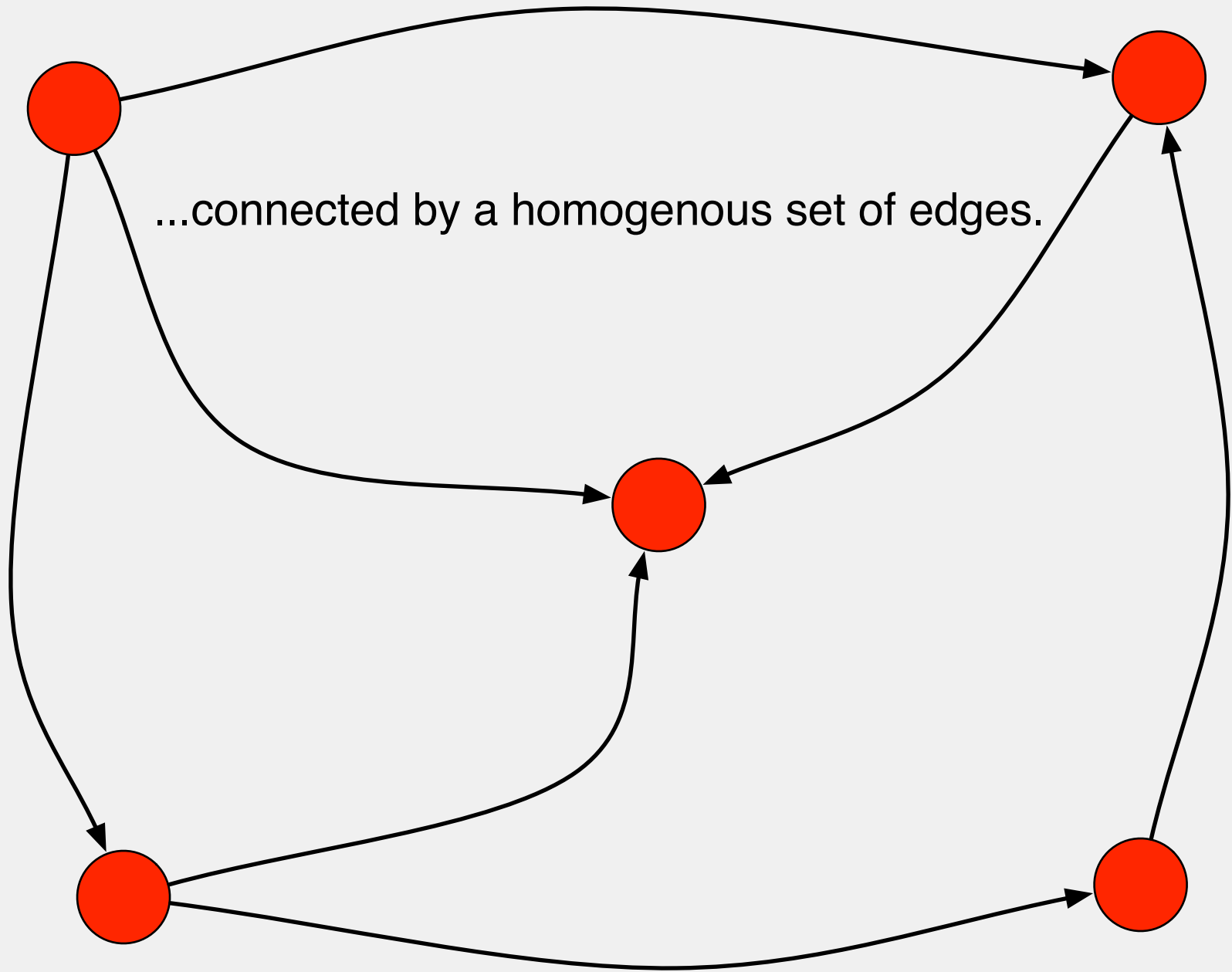
$$G = (V, E)$$

Classic Textbook Graph Structure

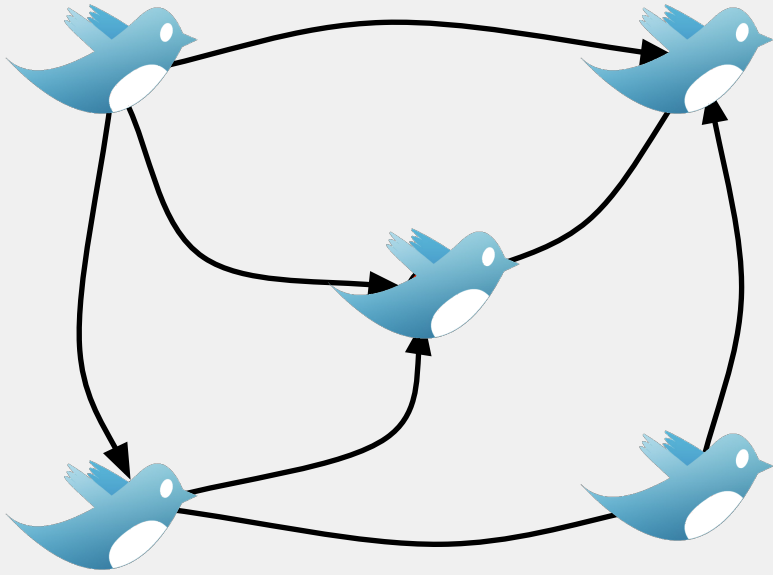


A homogenous set of vertices...



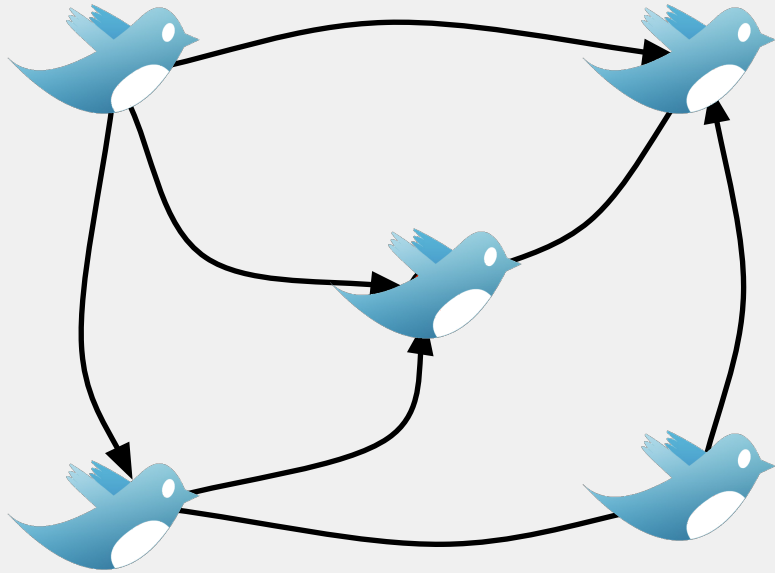


RESTRICTED MODELING



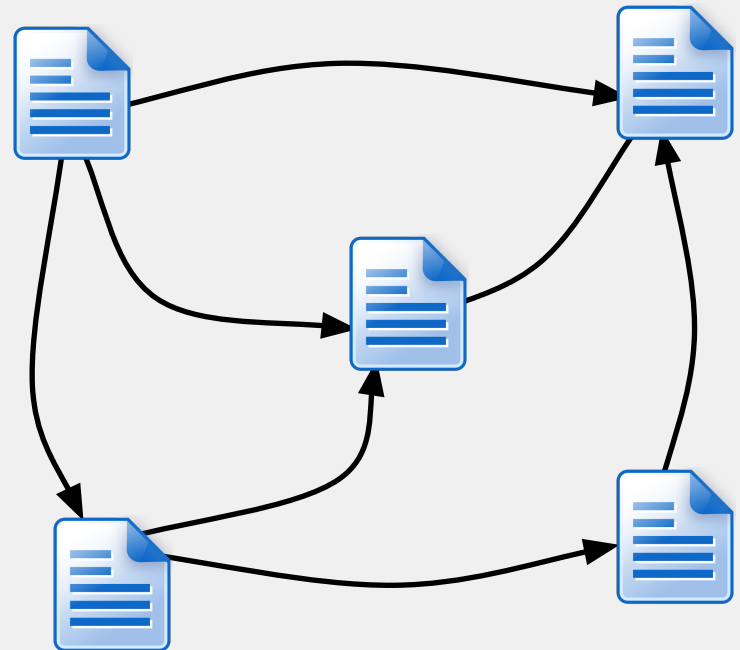
People and follows relationships...

RESTRICTED MODELING

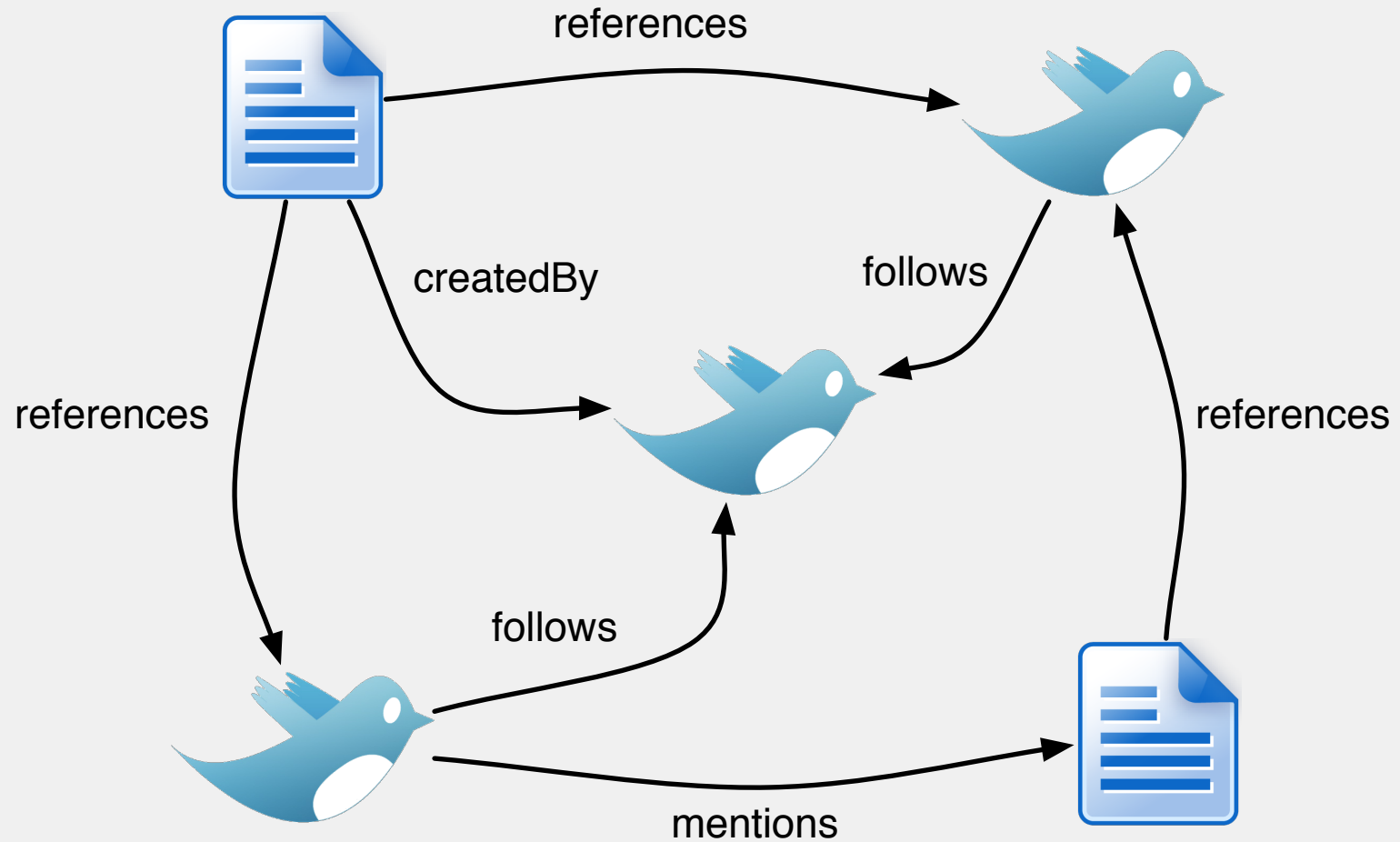


People and follows relationships...

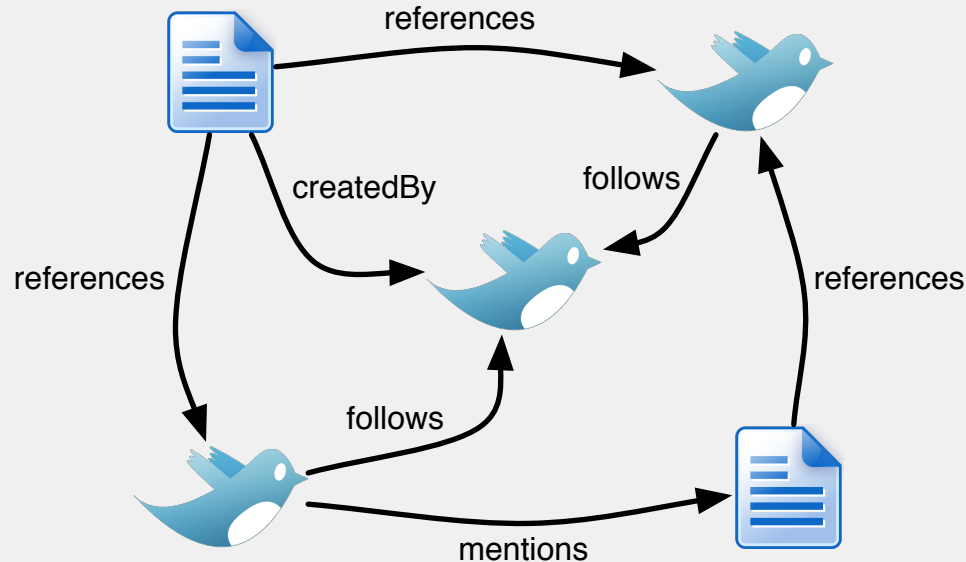
...xor webpages and citations.



AN INTEGRATED MODEL IS TYPICALLY DESIRED



AN INTEGRATED MODEL IS USEFUL



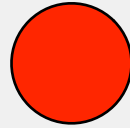
- Allows for more interesting/novel algorithms.
(beyond "textbook" graph algorithms)
- Allows for a universal model of things and their relationships.
(a single, unified model of a domain of interest)

THE PROPERTY GRAPH

$$G = (V, E, \lambda)$$

Current Popular Graph Structure

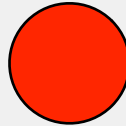
- * Directed, attributed, edge-labeled graph
- * Multi-relational graph with key/value pairs on the elements



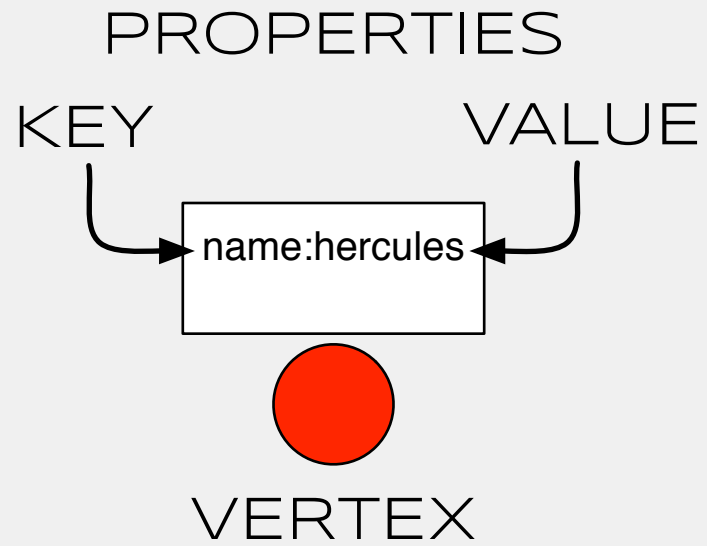
VERTEX

PROPERTIES

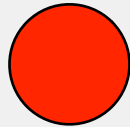
name:hercules

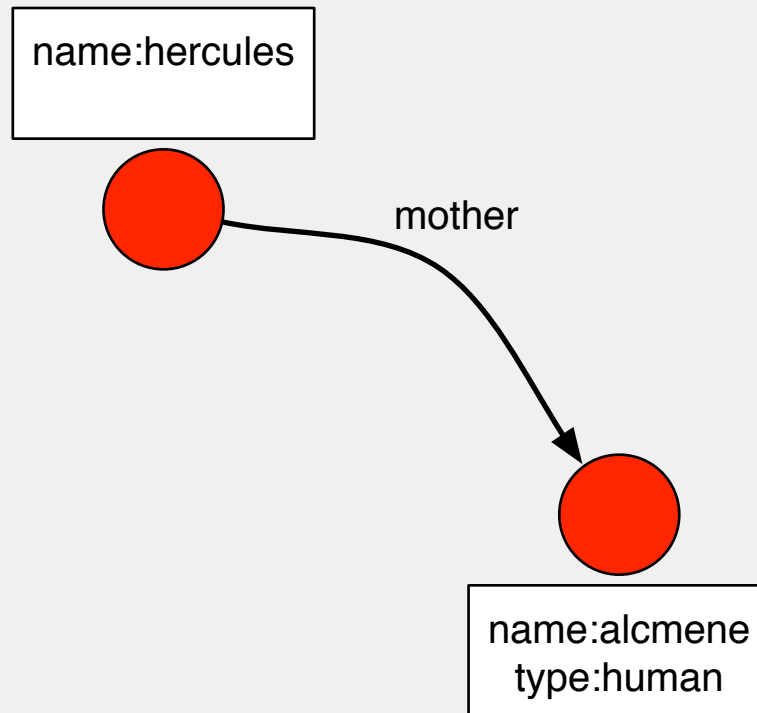


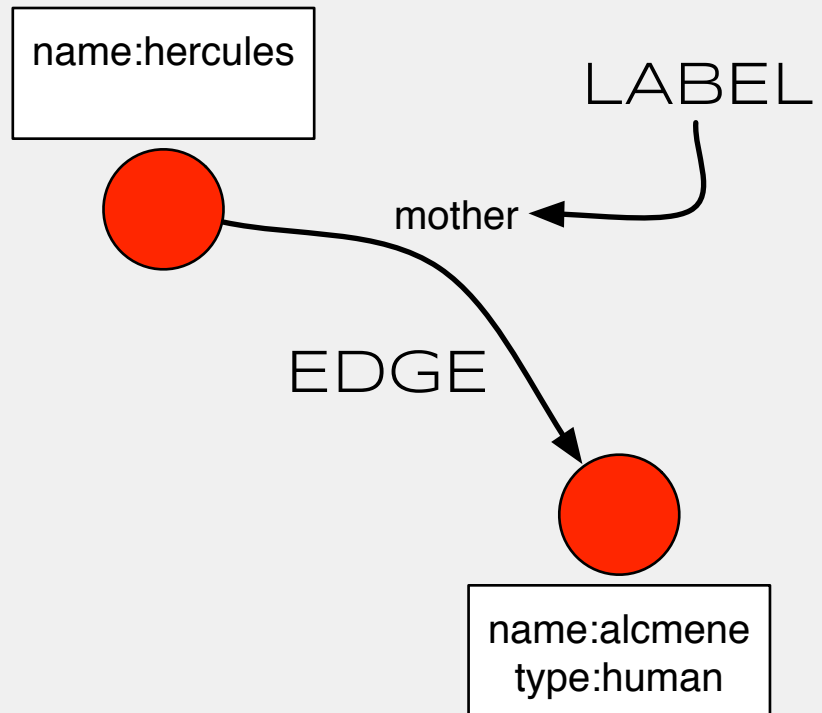
VERTEX

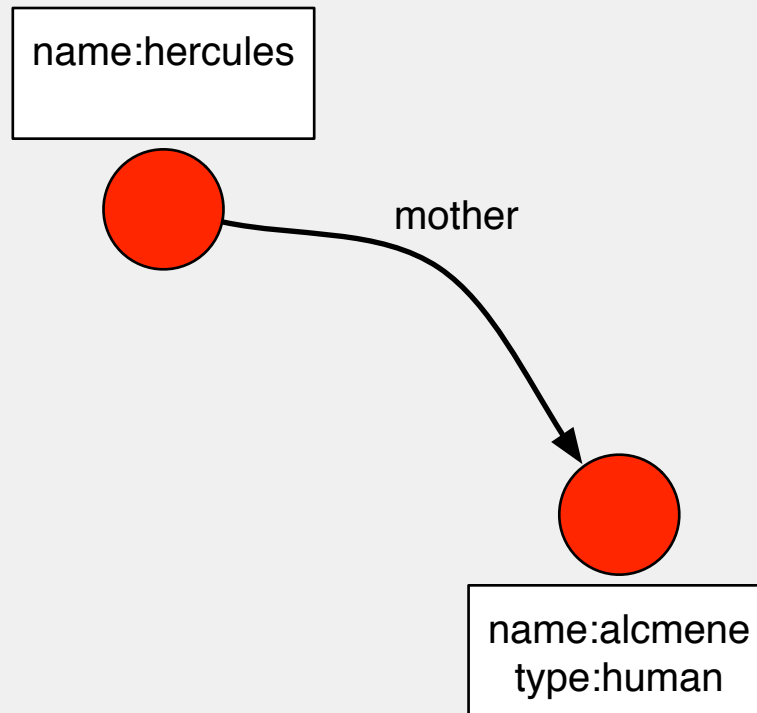


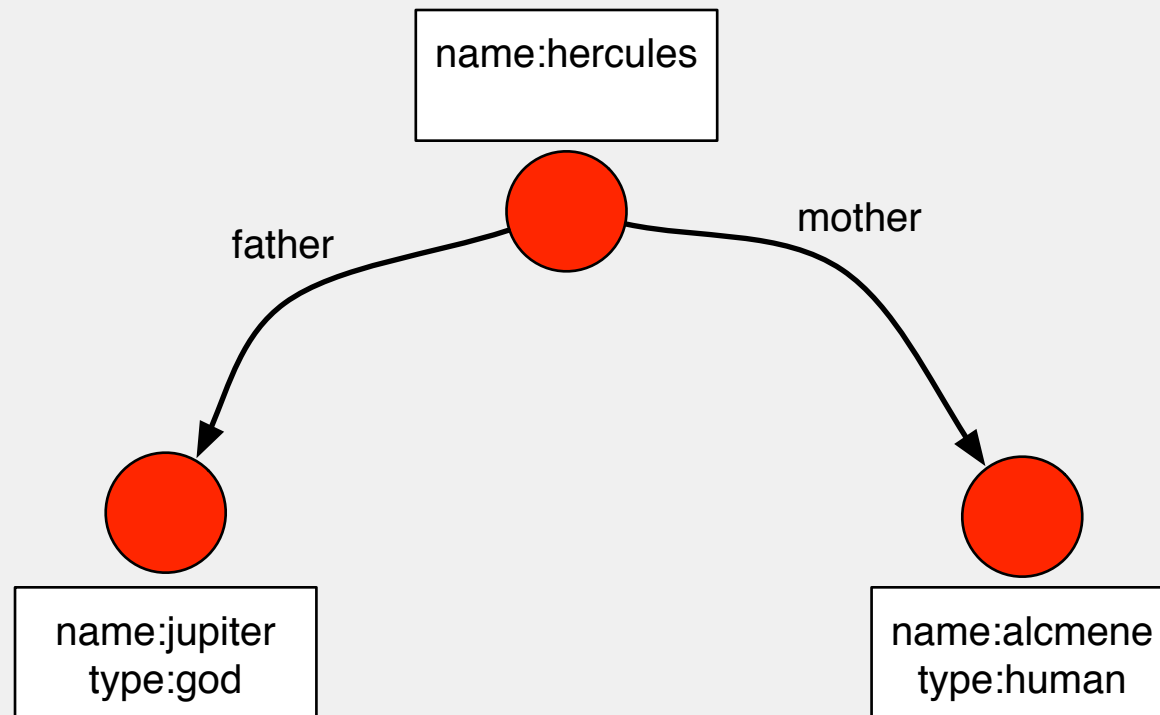
name:hercules





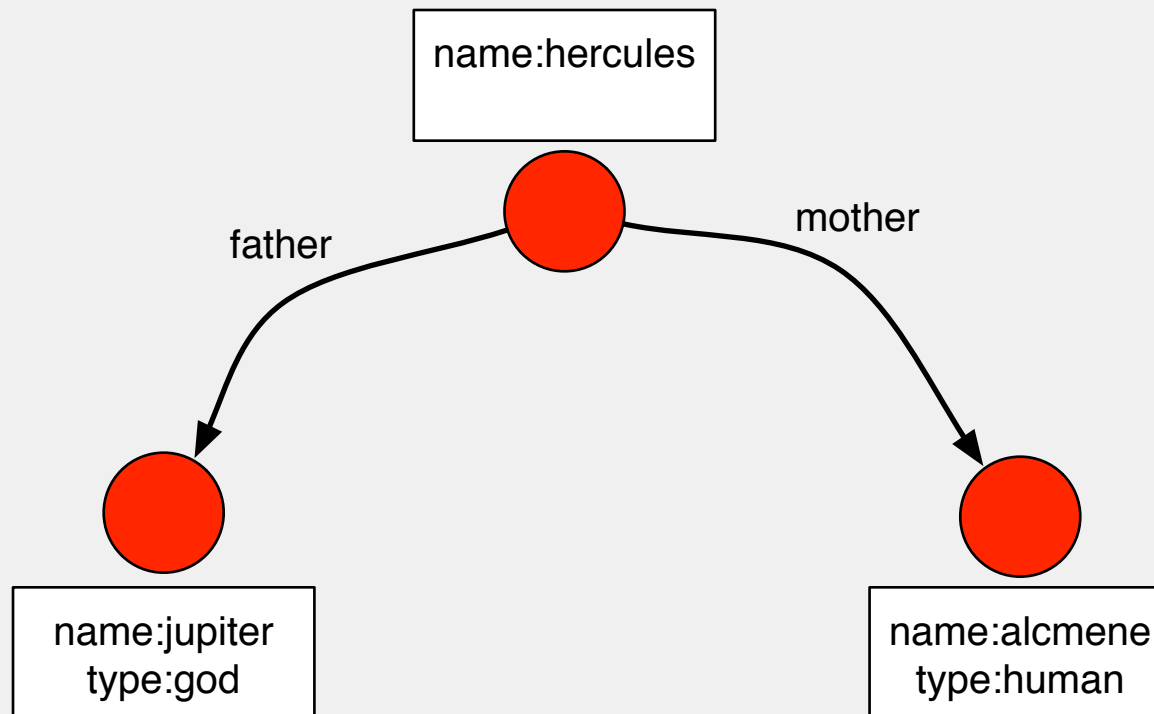


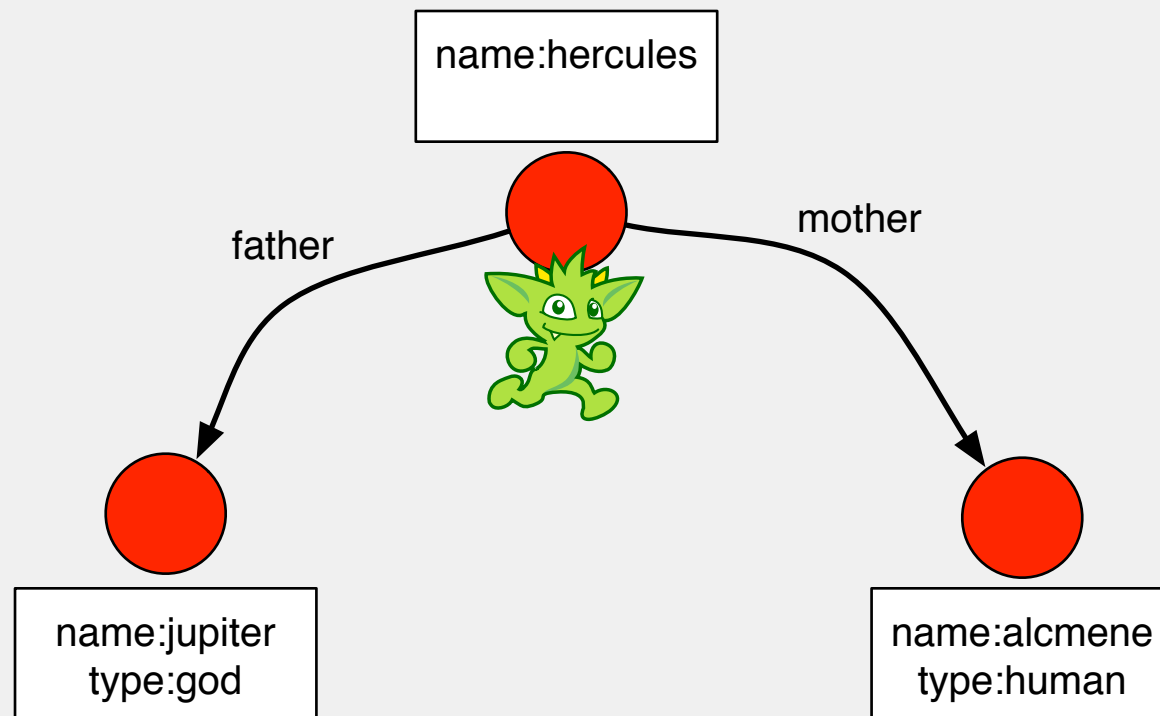




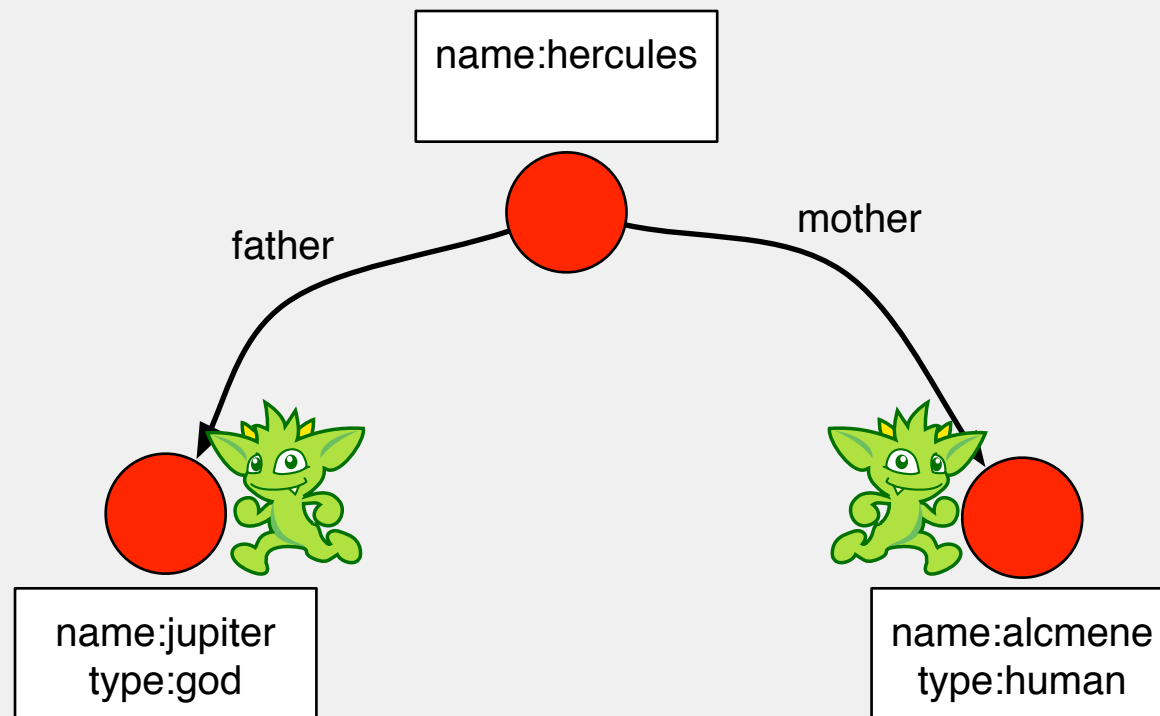
IS HERCULES A DEMIGOD?

DEMIGOD = HALF HUMAN + HALF GOD



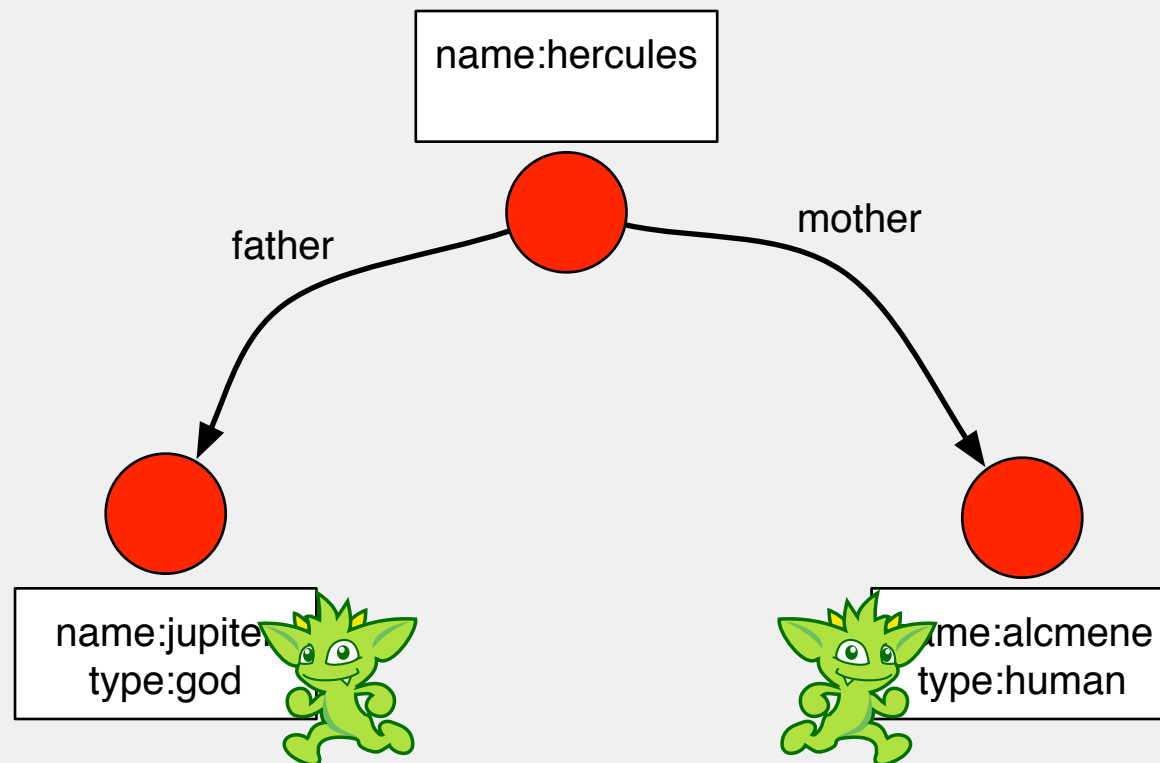


```
gremlin> hercules  
==>v[0]
```



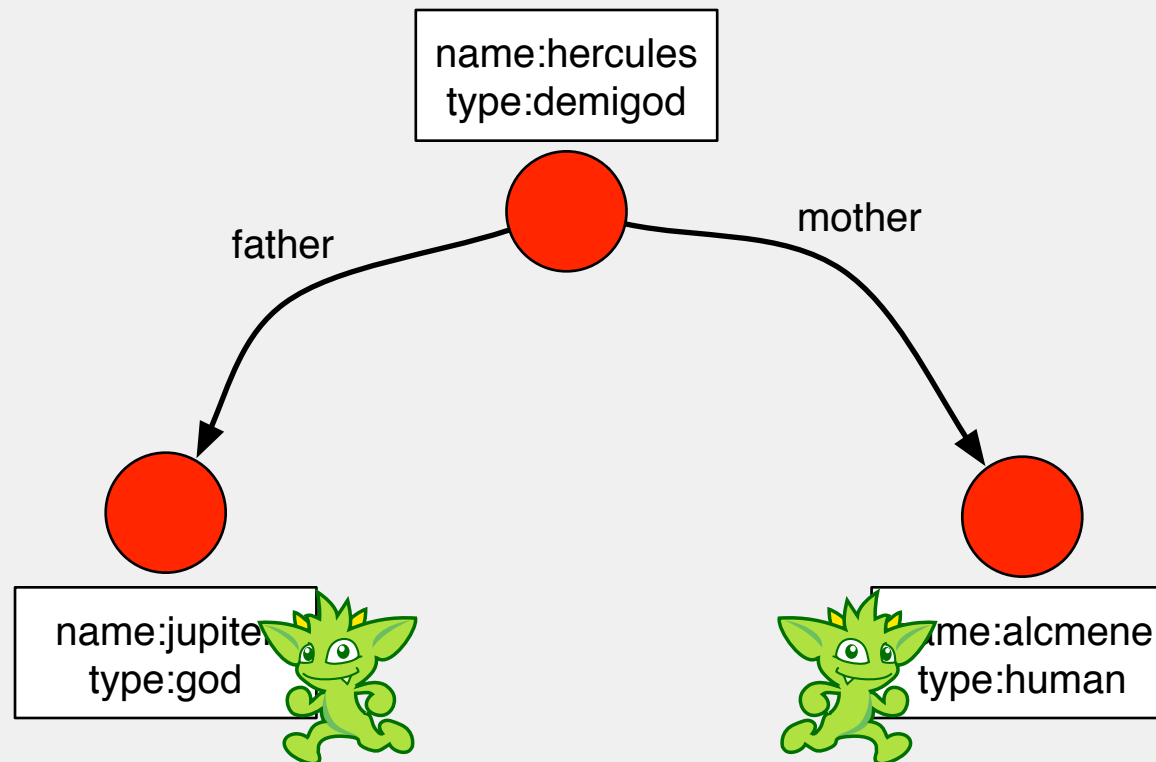
```
gremlin> hercules.out('mother','father')  
==>v[1]  
==>v[2]
```


DEMIGOD = HALF HUMAN + HALF GOD



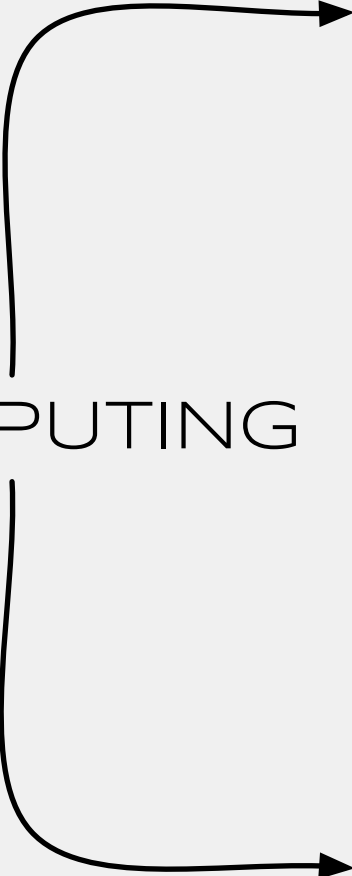
```
gremlin> hercules.out('mother','father').type
==>human
==>god
```

DEMIGOD = HALF HUMAN + HALF GOD



```
gremlin> hercules.type = 'demigod'  
==>demigod
```

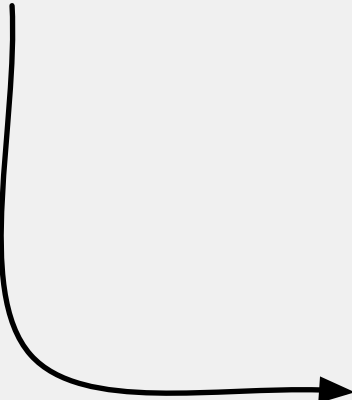
COMPUTING



PROCESS

STRUCTURE

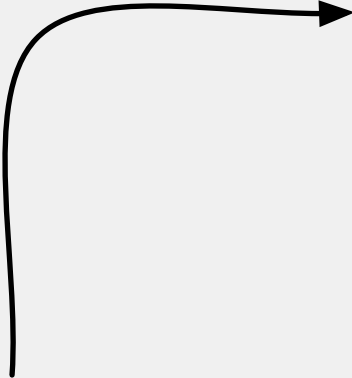
COMPUTING



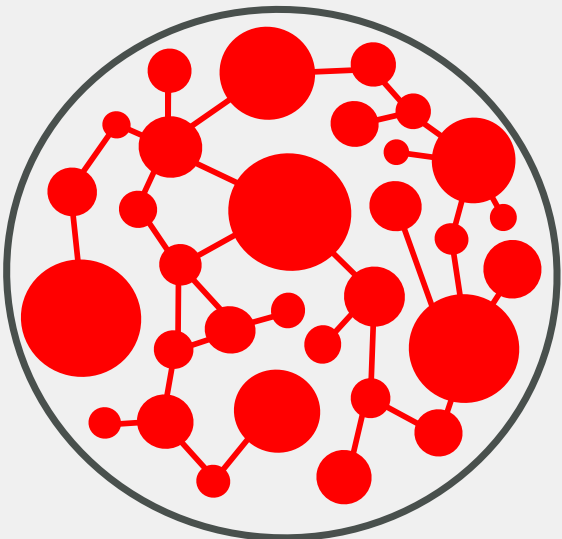
PROCESS



TRAVERSAL

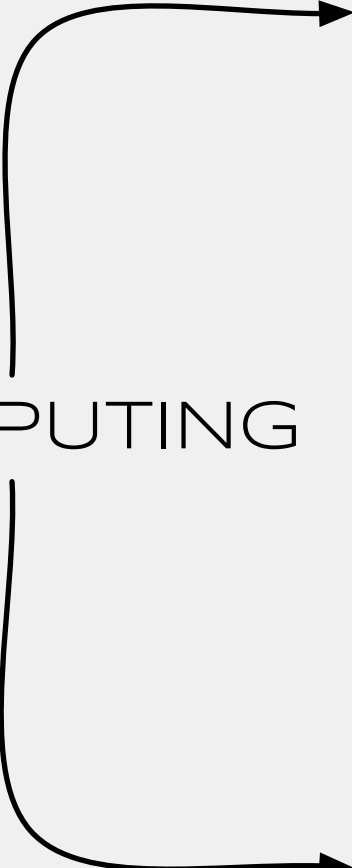


STRUCTURE



GRAPH

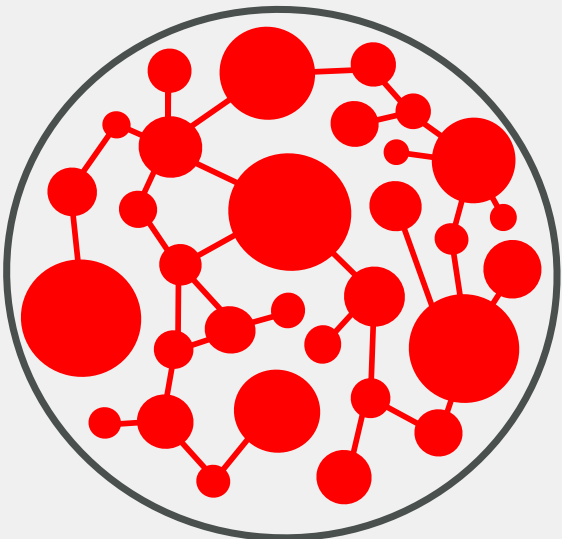
COMPUTING



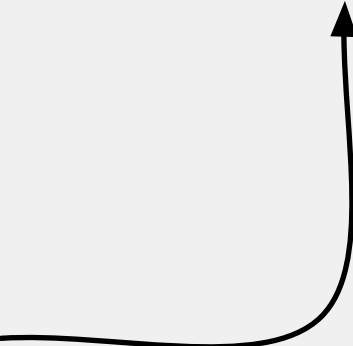
PROCESS



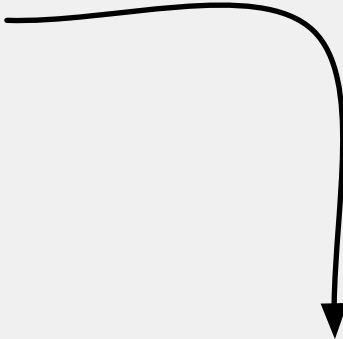
STRUCTURE



TRAVERSAL



GRAPH



GRAPH-BASED
COMPUTING

WHY GRAPH-BASED COMPUTING?

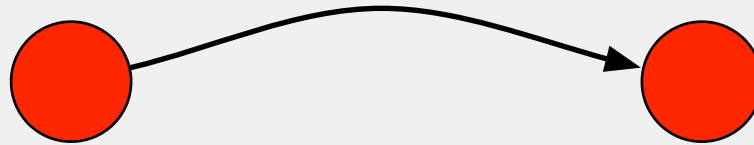
WHY GRAPH-BASED COMPUTING?

INTUITIVE MODELING

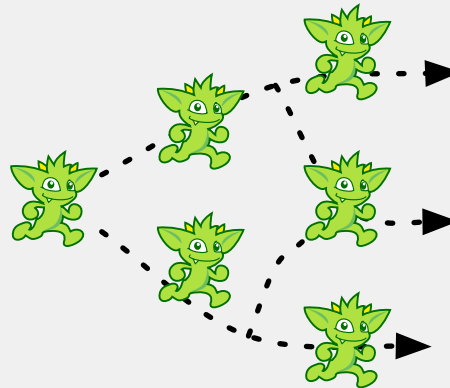


WHY GRAPH-BASED COMPUTING?

INTUITIVE MODELING

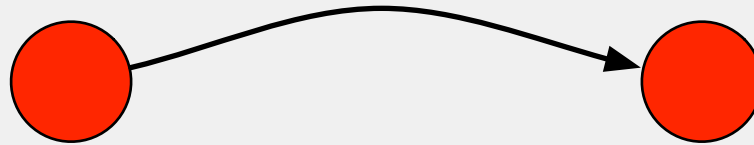


EXPRESSIVE QUERYING

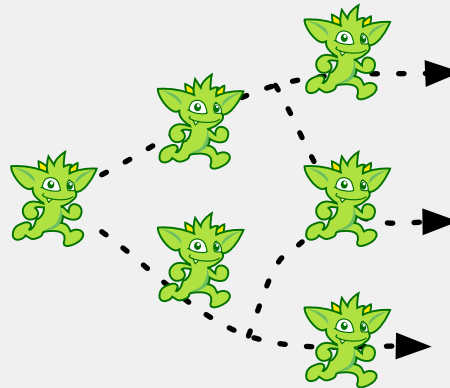


WHY GRAPH-BASED COMPUTING?

INTUITIVE MODELING



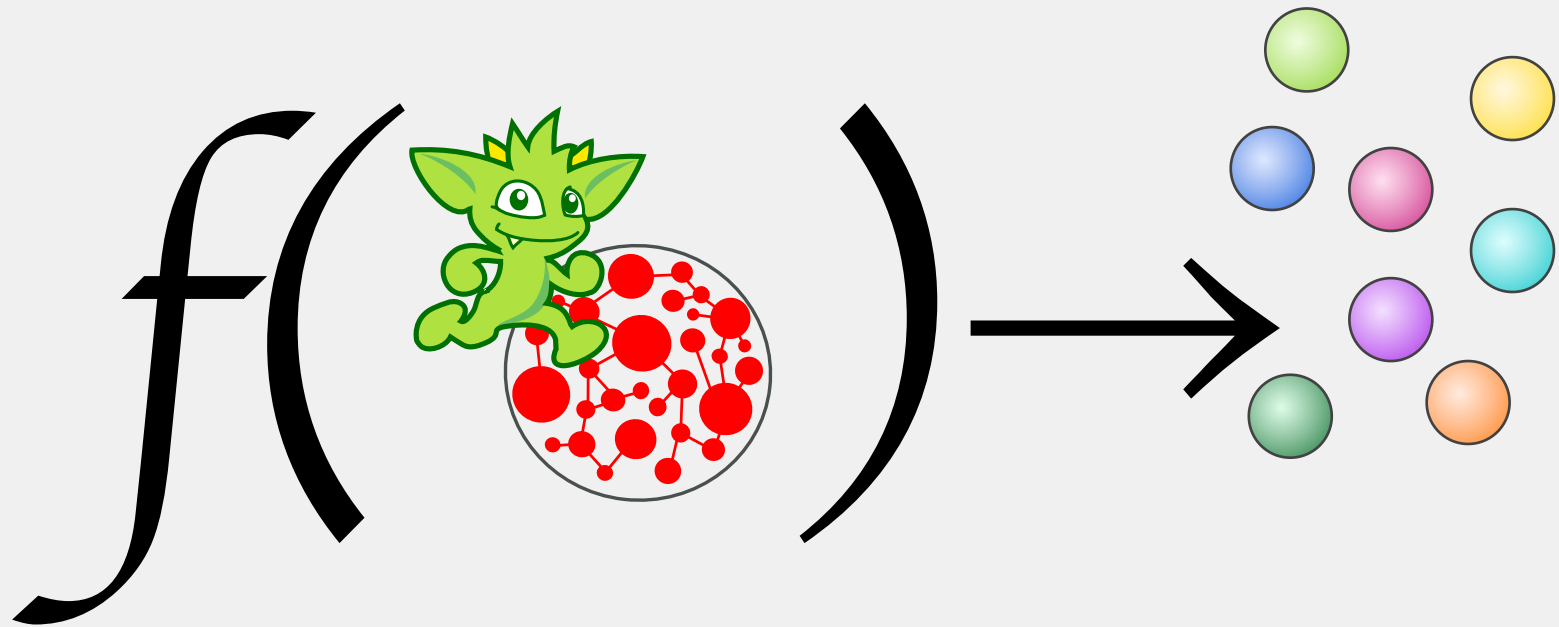
EXPRESSIVE QUERYING



NUMEROUS ANALYSES

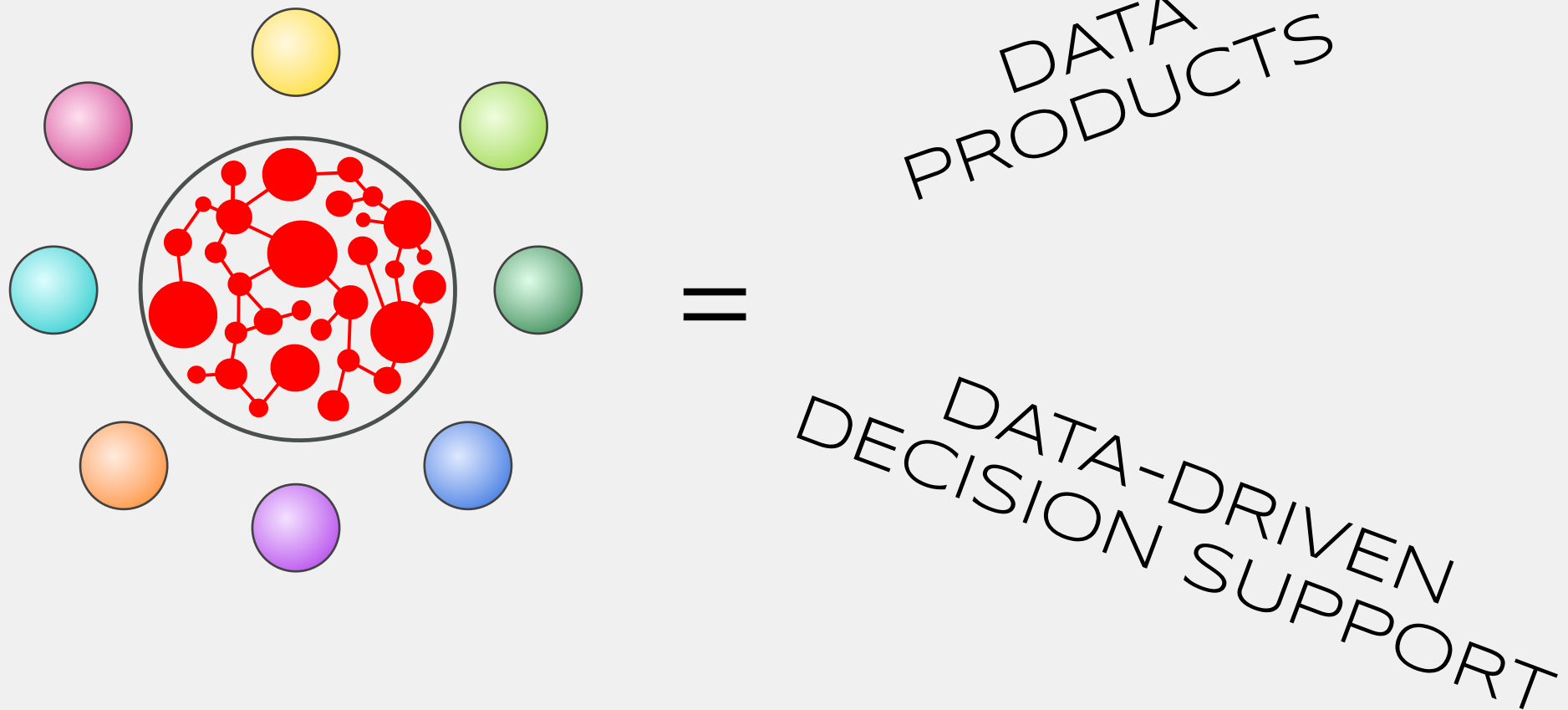


ANALYSES ARE THE EPIPHENOMENA OF TRAVERSAL



WHAT IS THE SIGNIFICANCE OF
GRAPH ANALYSIS?

ANALYSES YIELD INSIGHTS ABOUT THE MODEL



RECOMMENDATION



● People you may know.

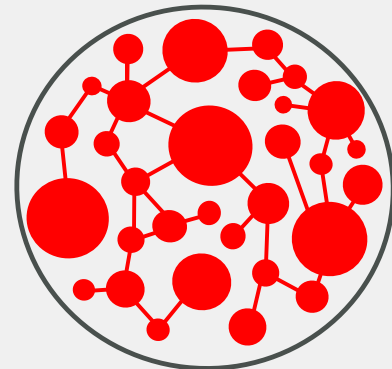
SOCIAL GRAPH

● Products you might like.

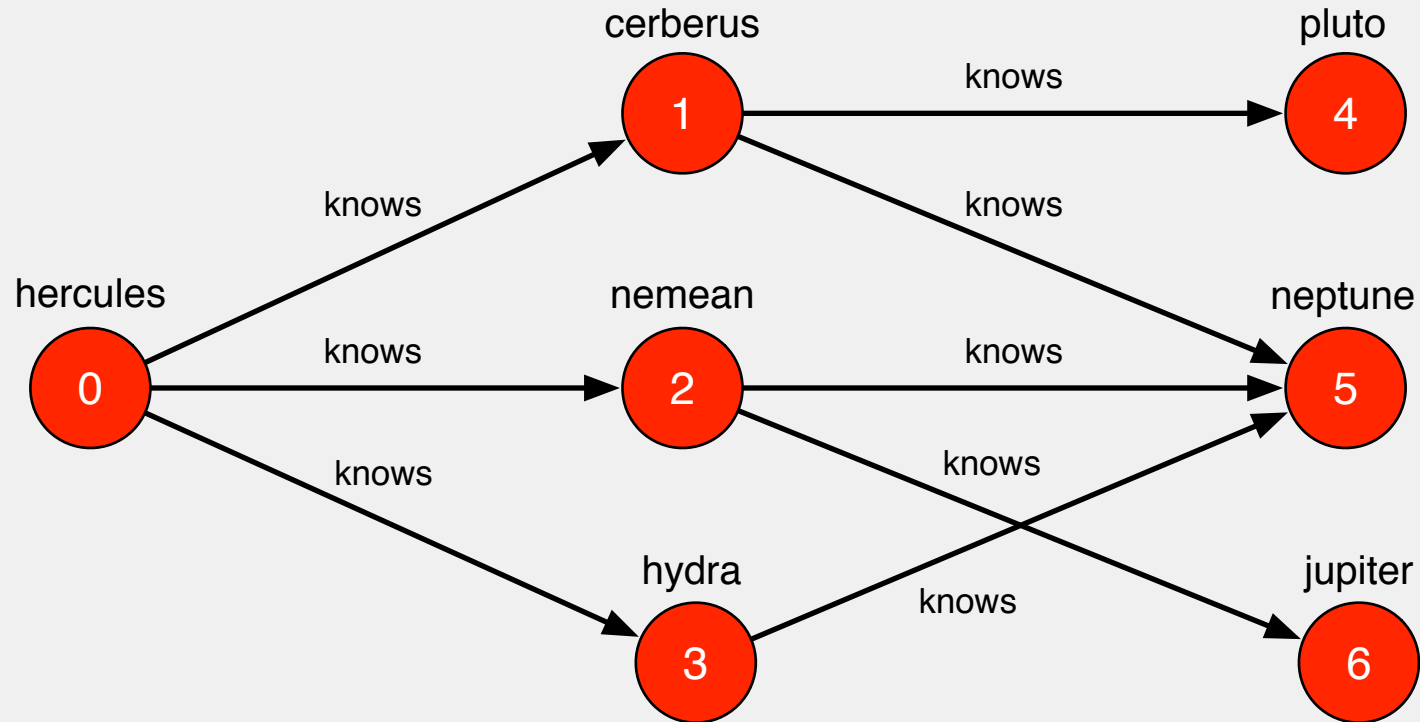
RATINGS GRAPH

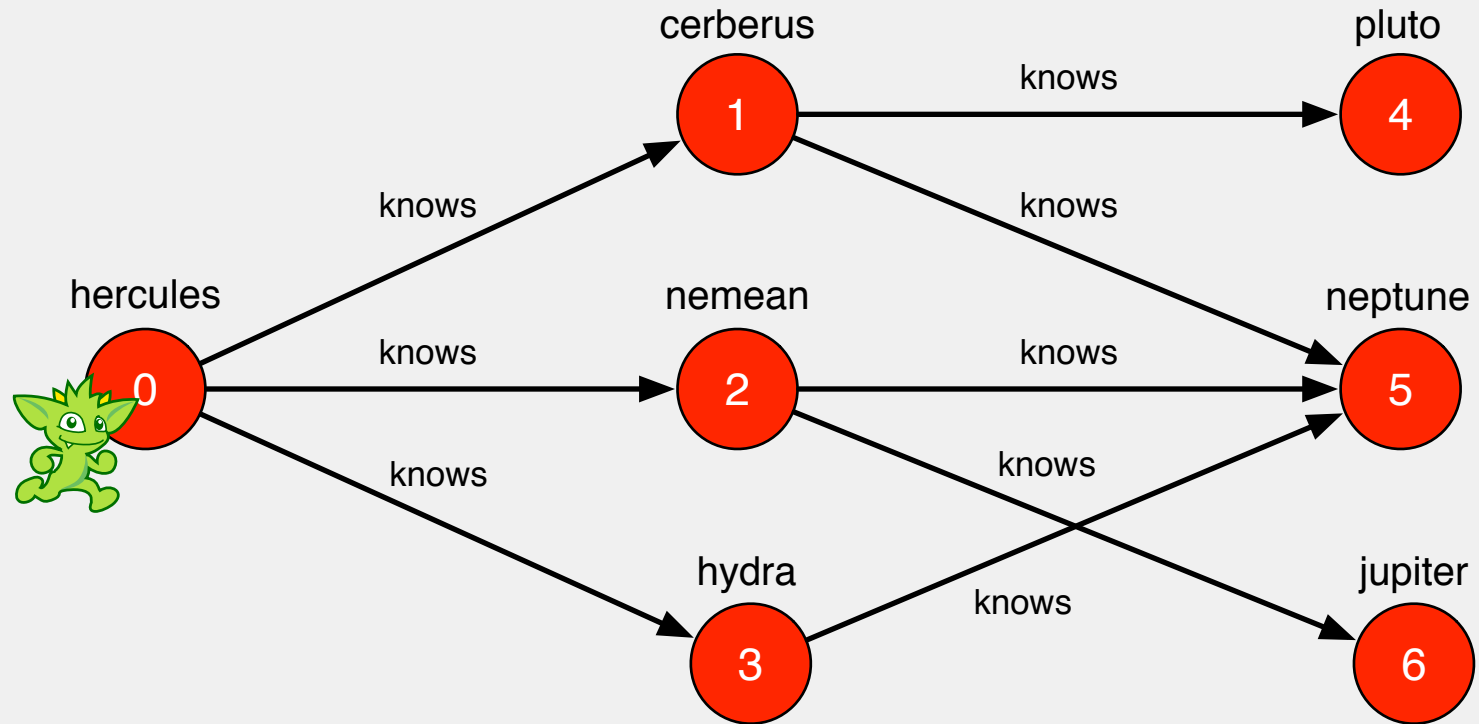
● Movies you should watch and
the friends you should watch them with.

**SOCIAL+RATINGS
GRAPH**

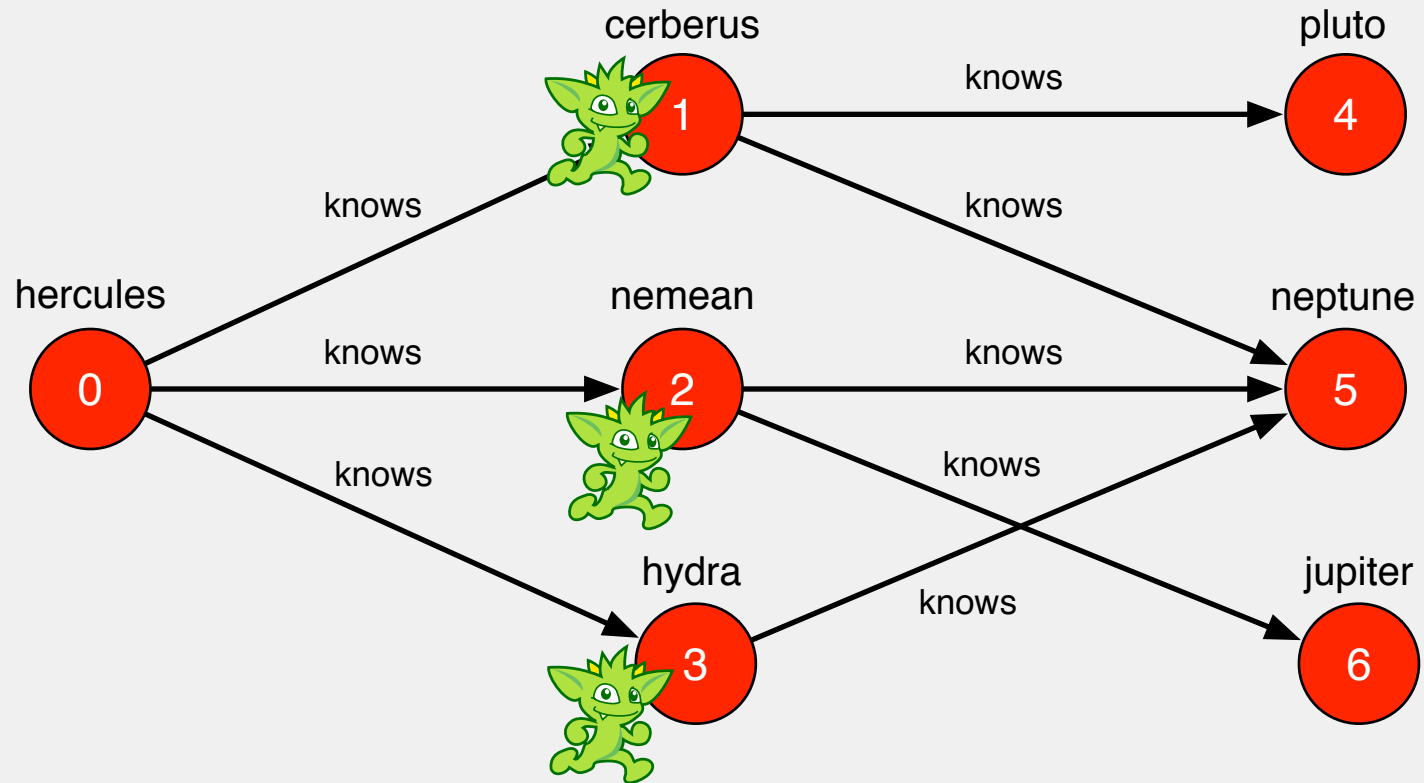


WHO ELSE MIGHT HERCULES KNOW?

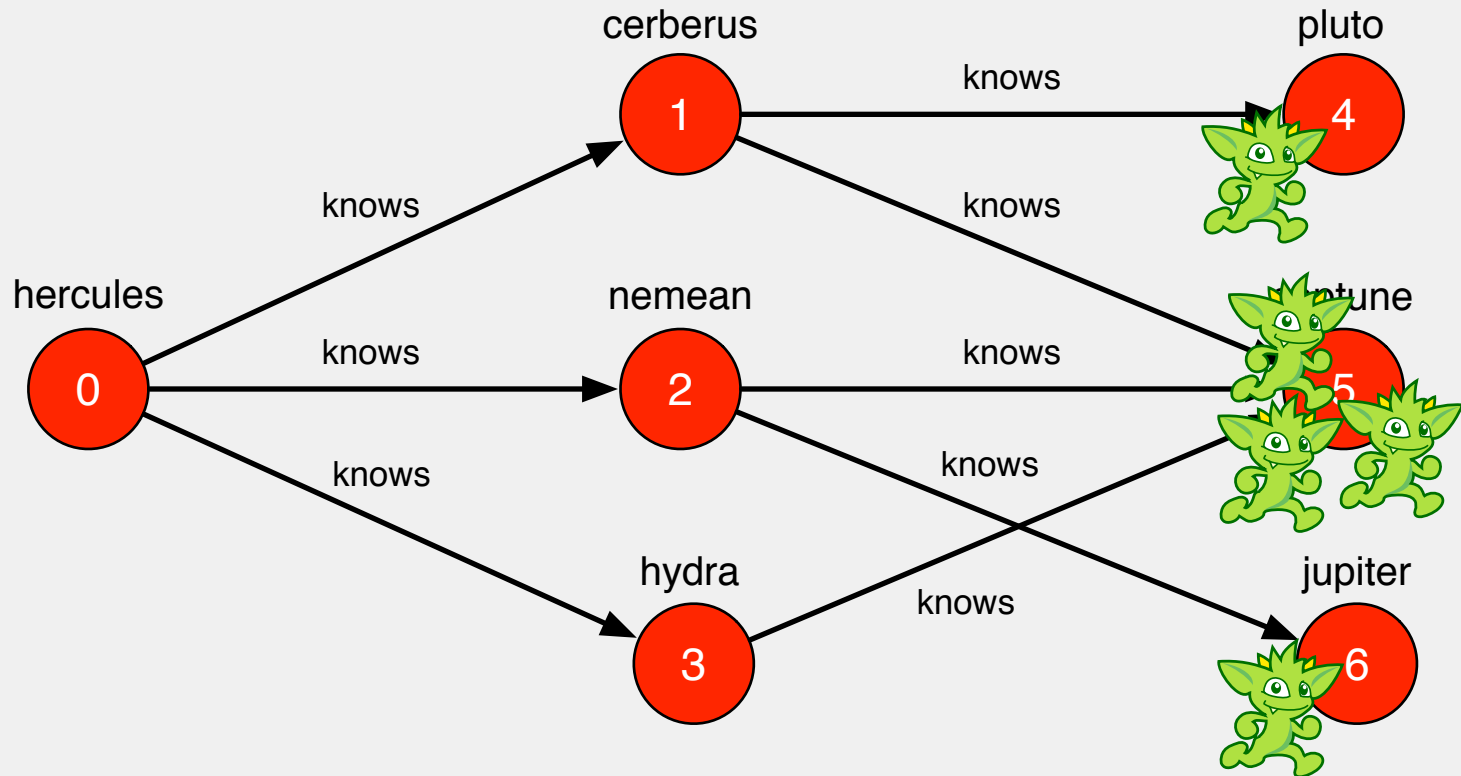




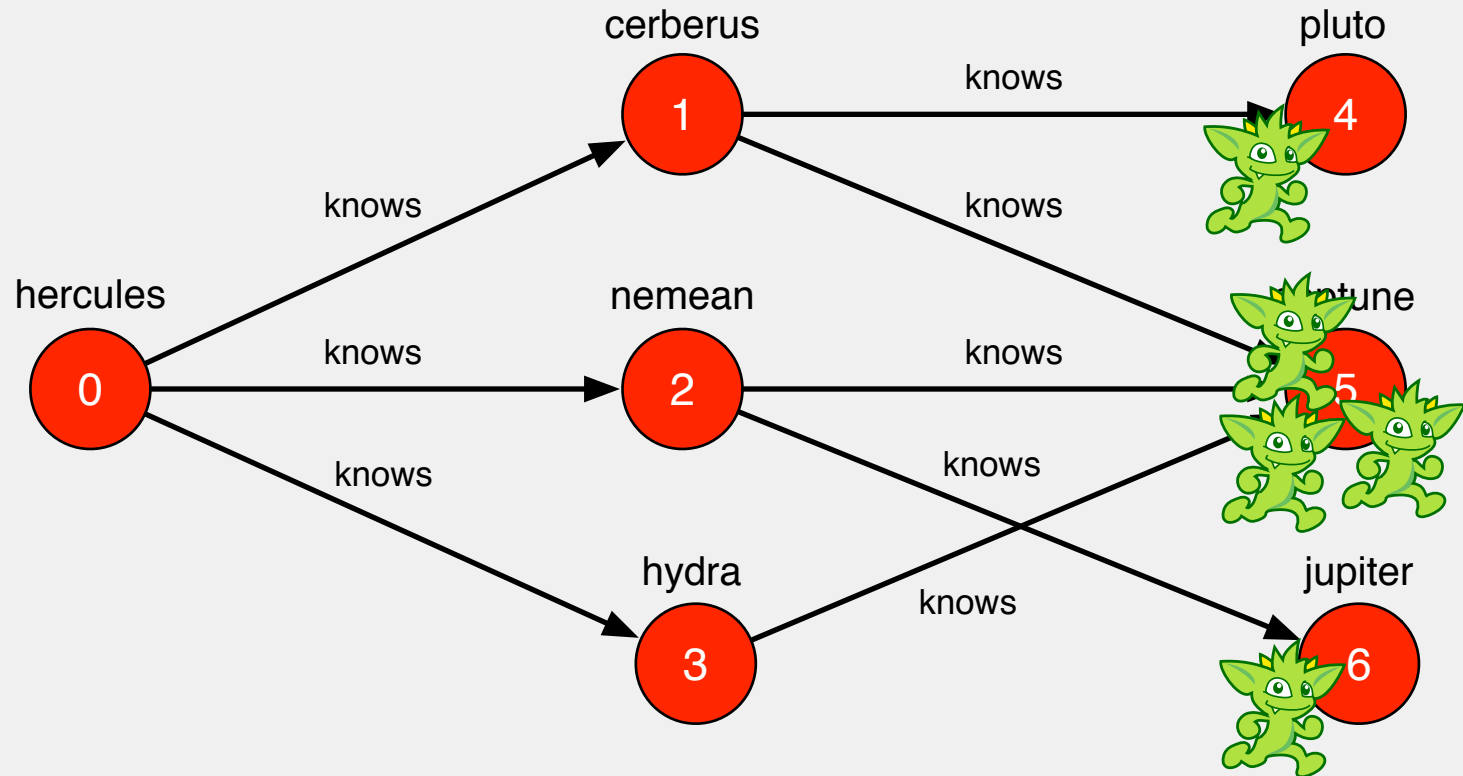
```
gremlin> hercules  
==>v[0]
```



```
gremlin> hercules.out('knows')  
==>v[1]  
==>v[2]  
==>v[3]
```

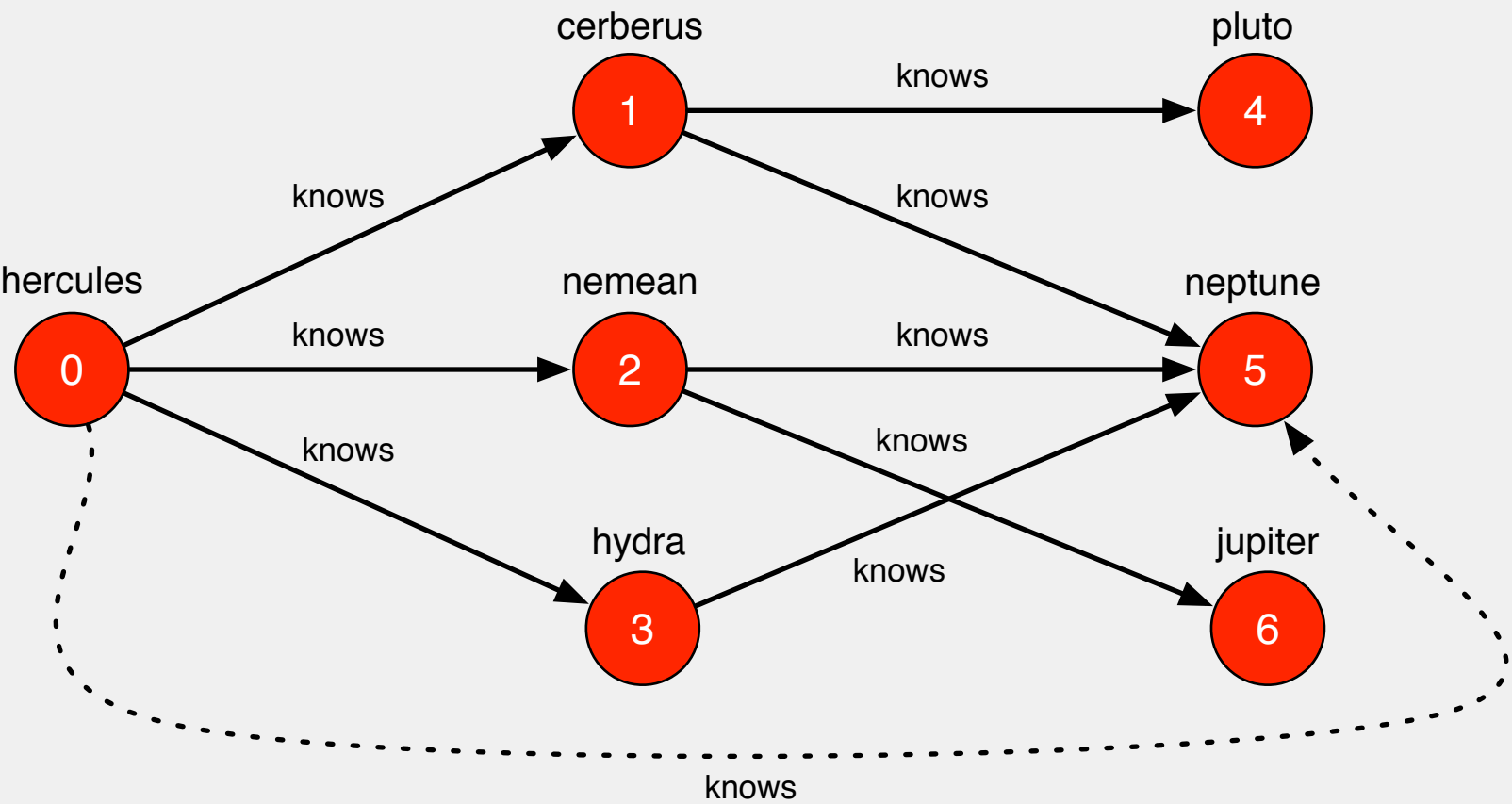



```
gremlin> hercules.out('knows').out('knows')
==>v[4]
==>v[5]
==>v[5]
==>v[6]
==>v[5]
```

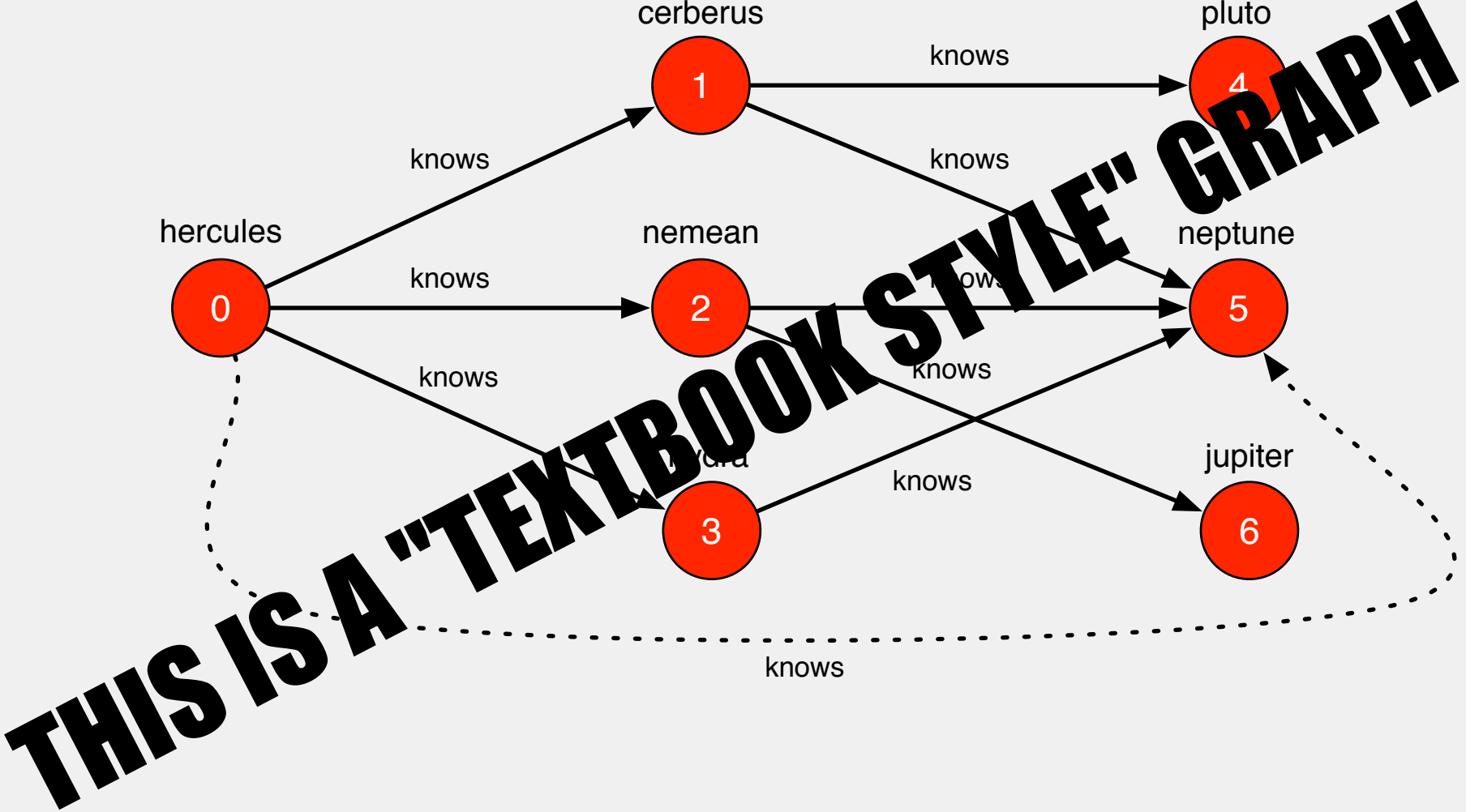


```
gremlin> hercules.out('knows').out('knows').groupCount.cap  
==>v[4]=1  
==>v[5]=3  
==>v[6]=1
```

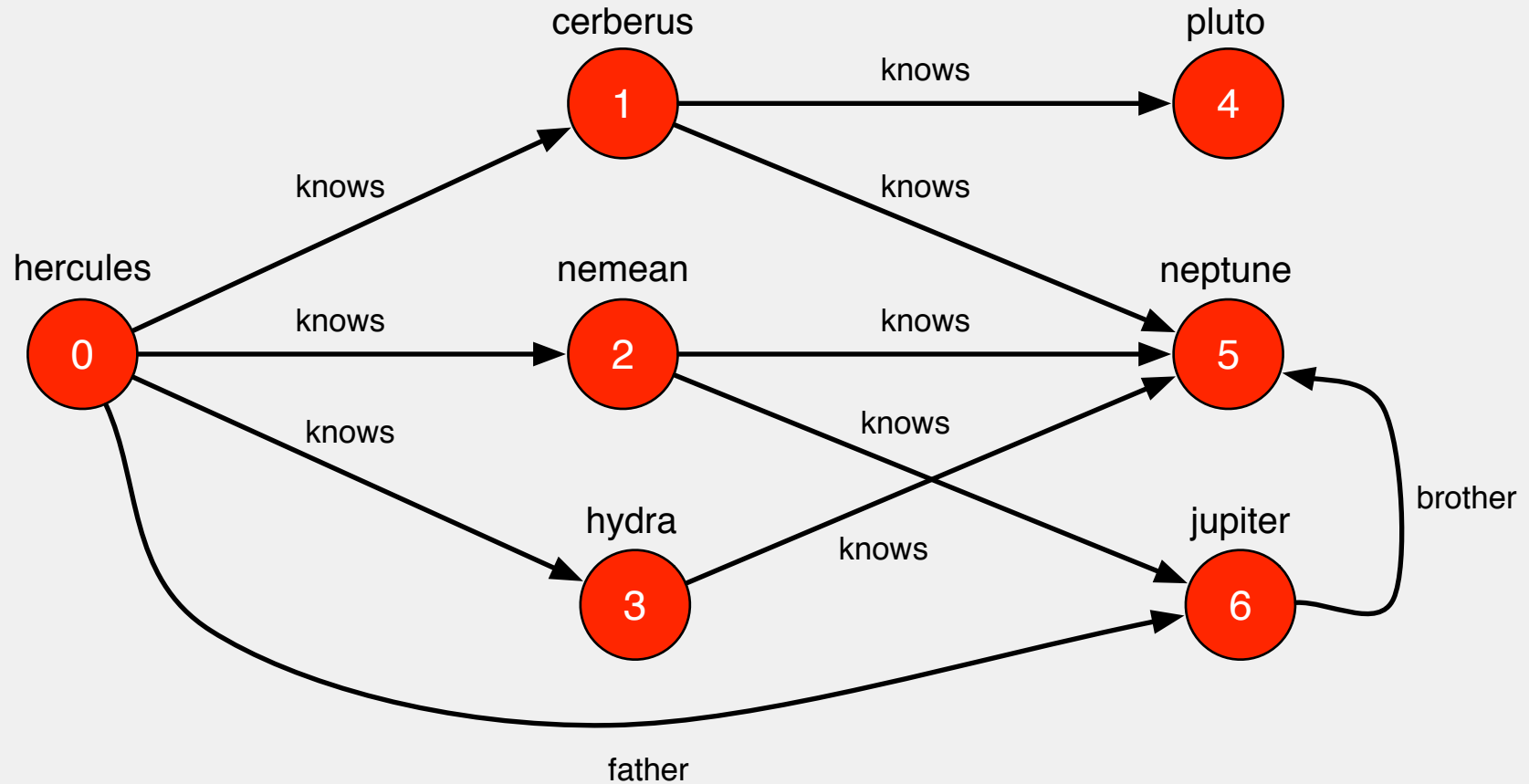
HERCULES PROBABLY KNOWS NEPTUNE



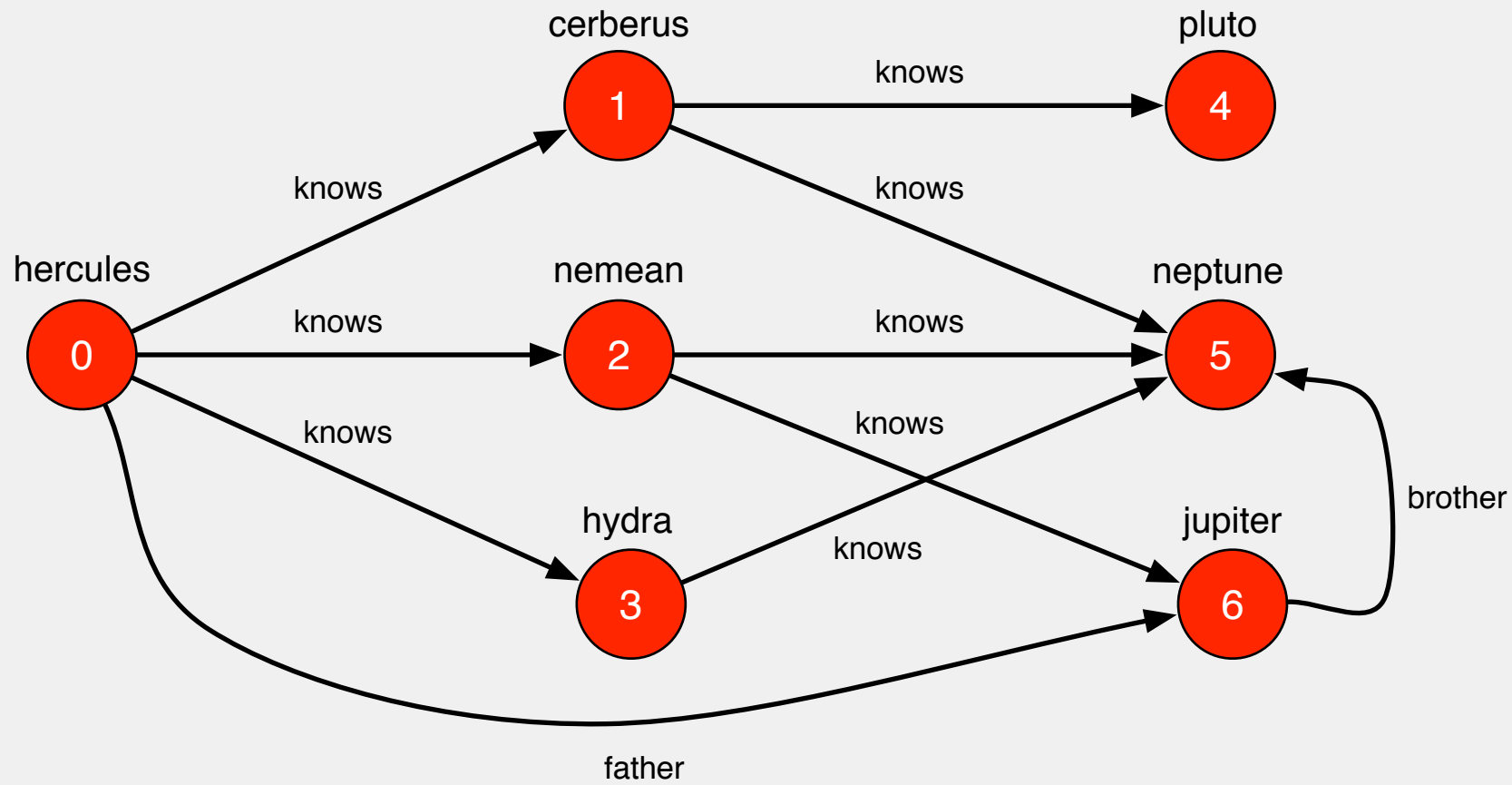
HERCULES PROBABLY KNOWS NEPTUNE

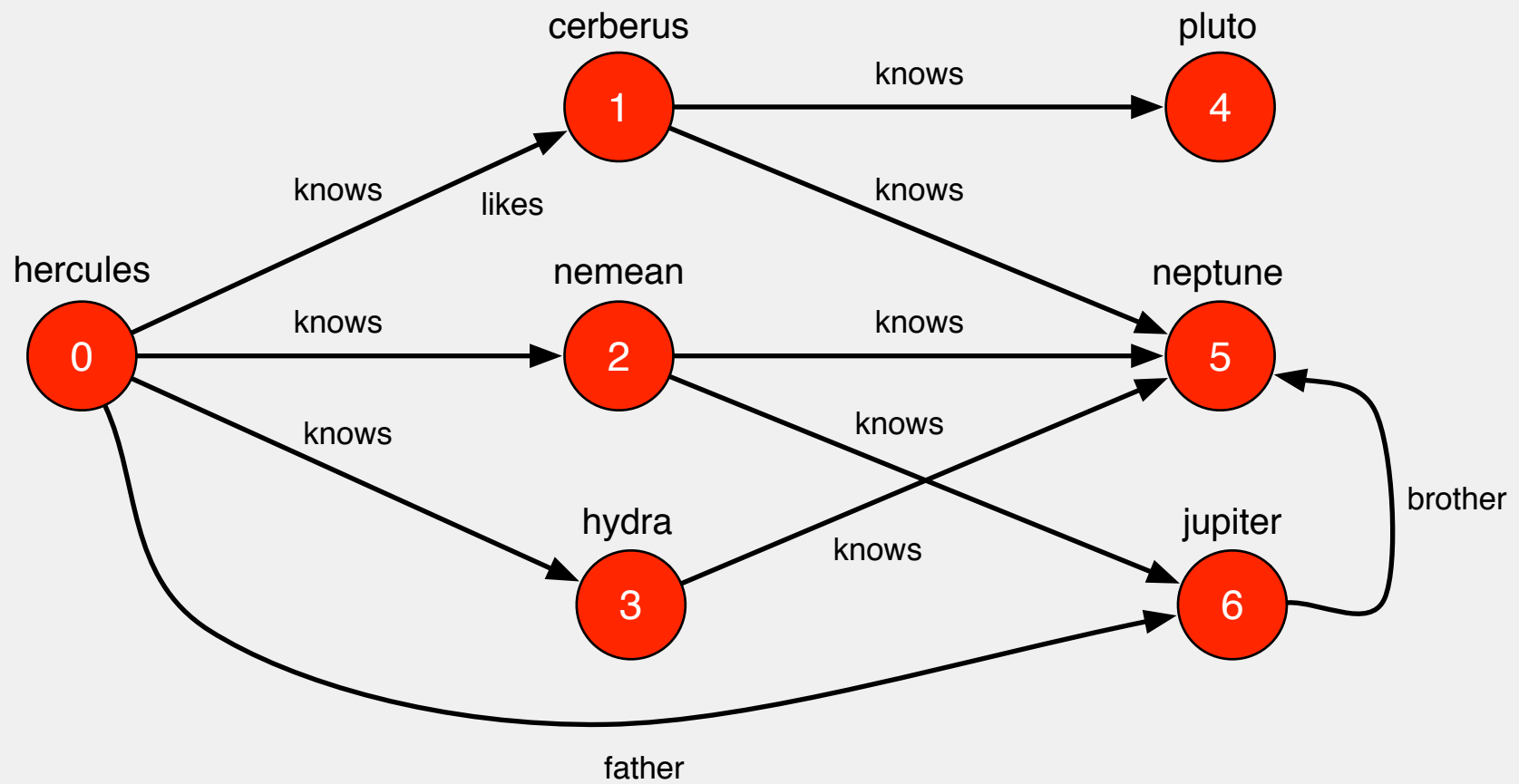


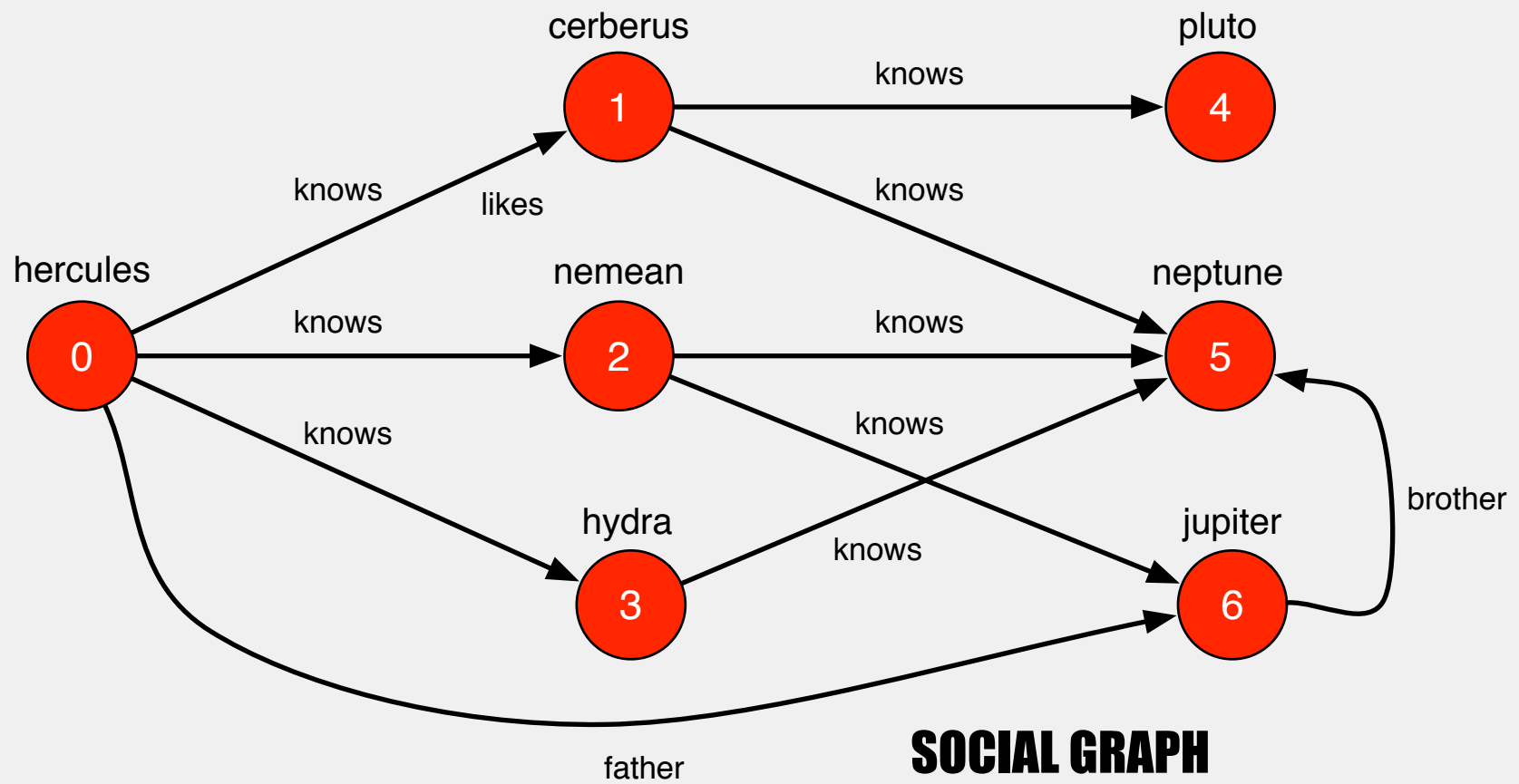
HERCULES PROBABLY KNOWS NEPTUNE

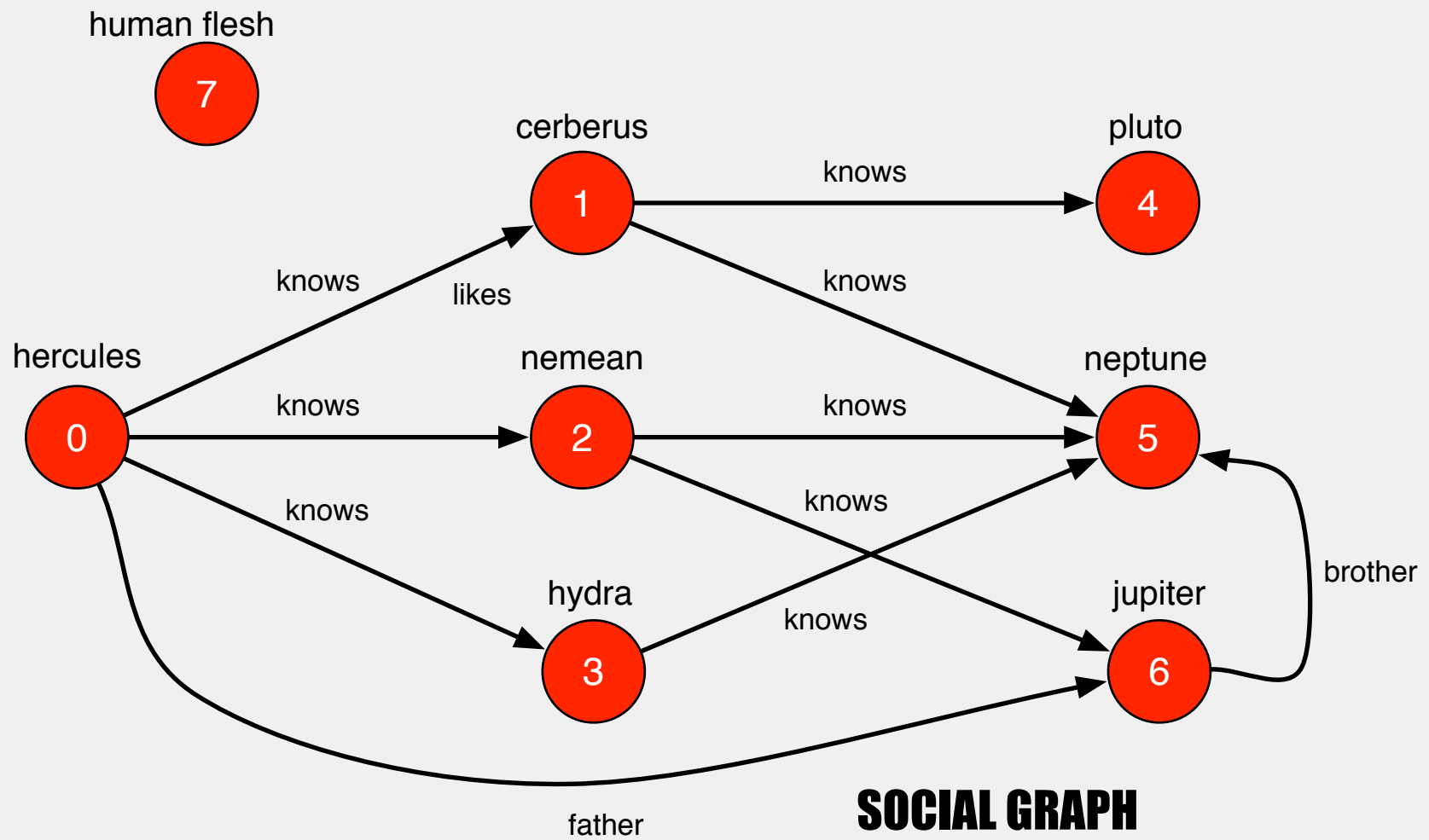


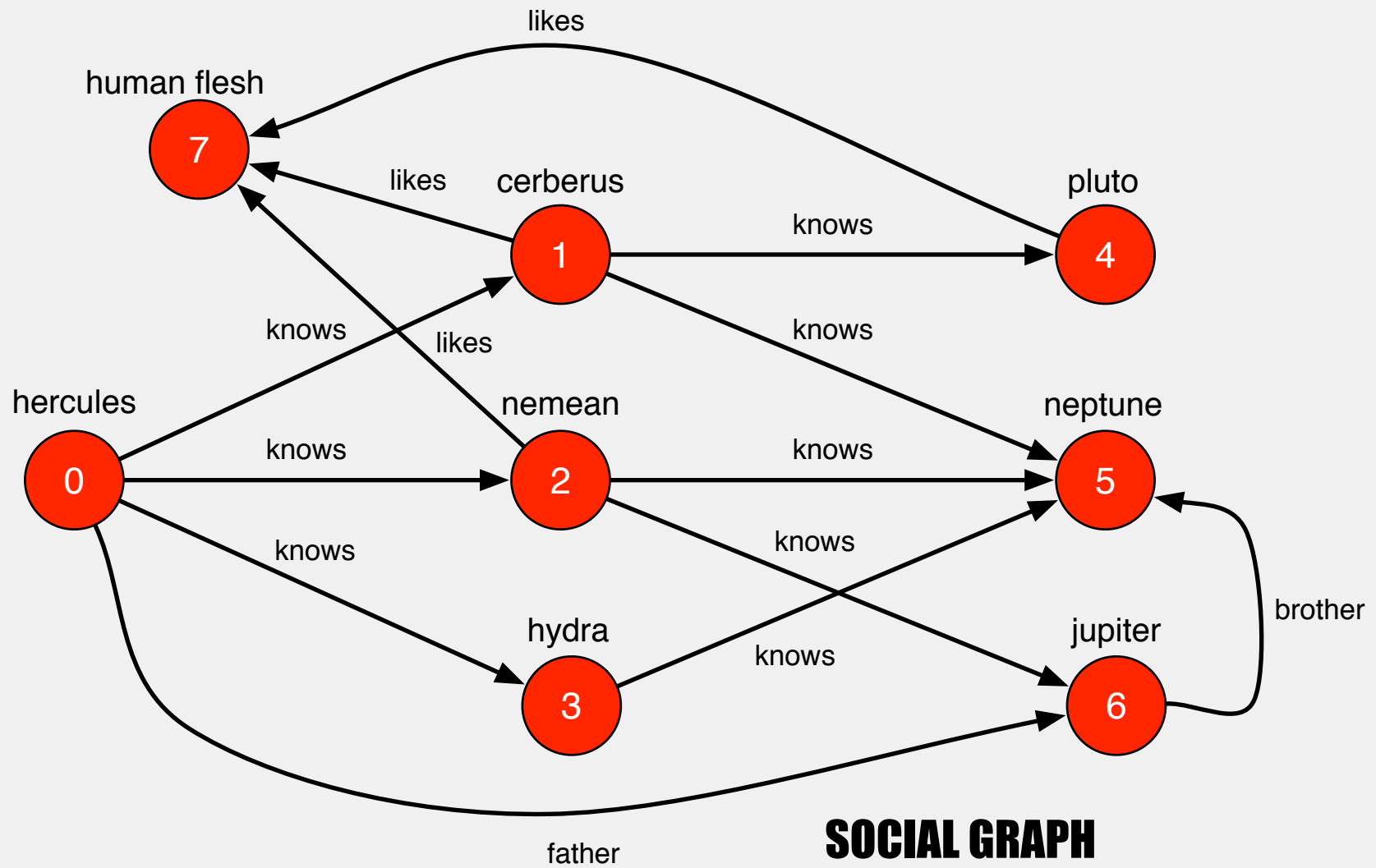
...PROBABLY MORE SO WHEN OTHER
TYPES OF EDGES ARE ANALYZED

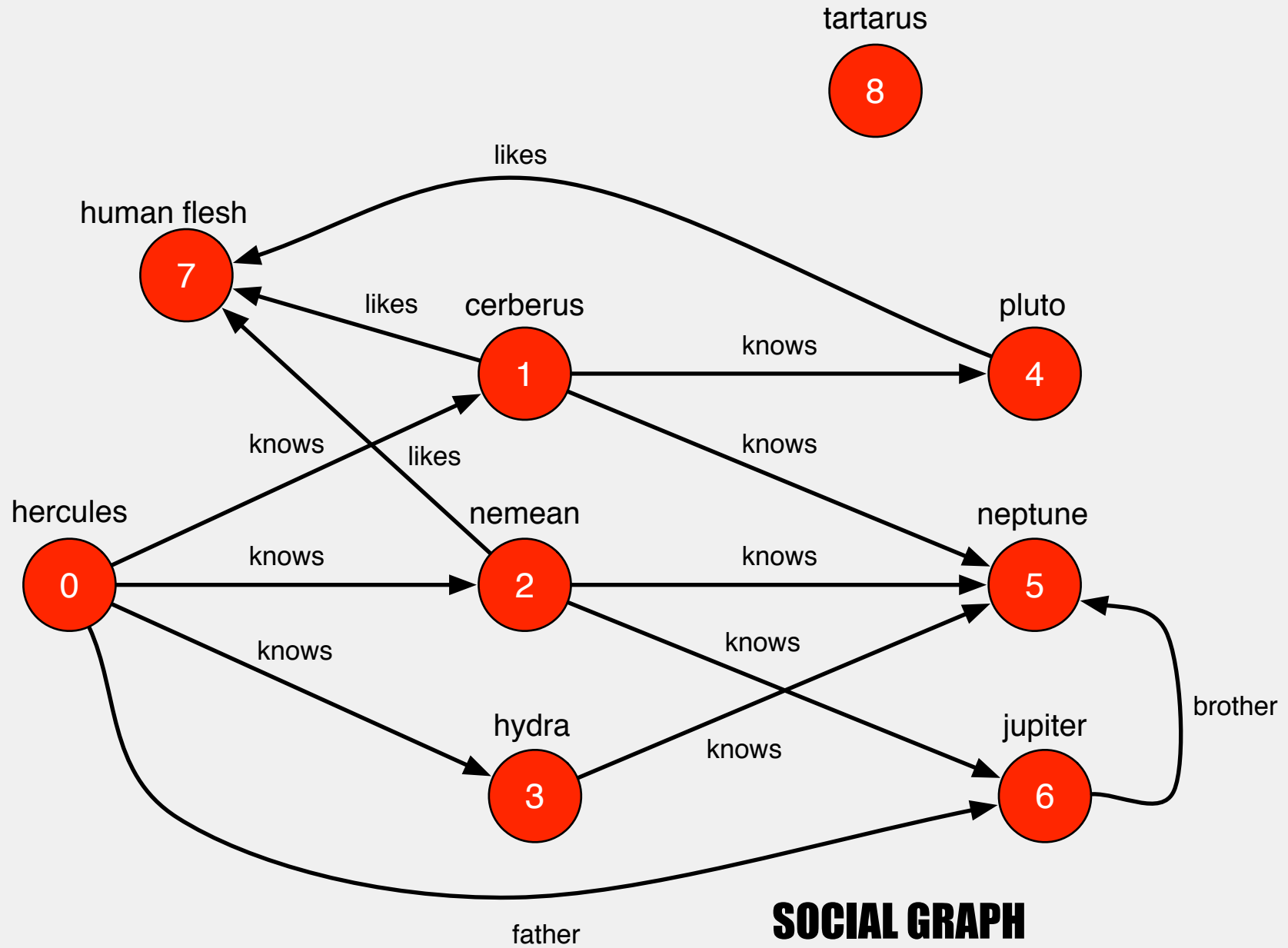


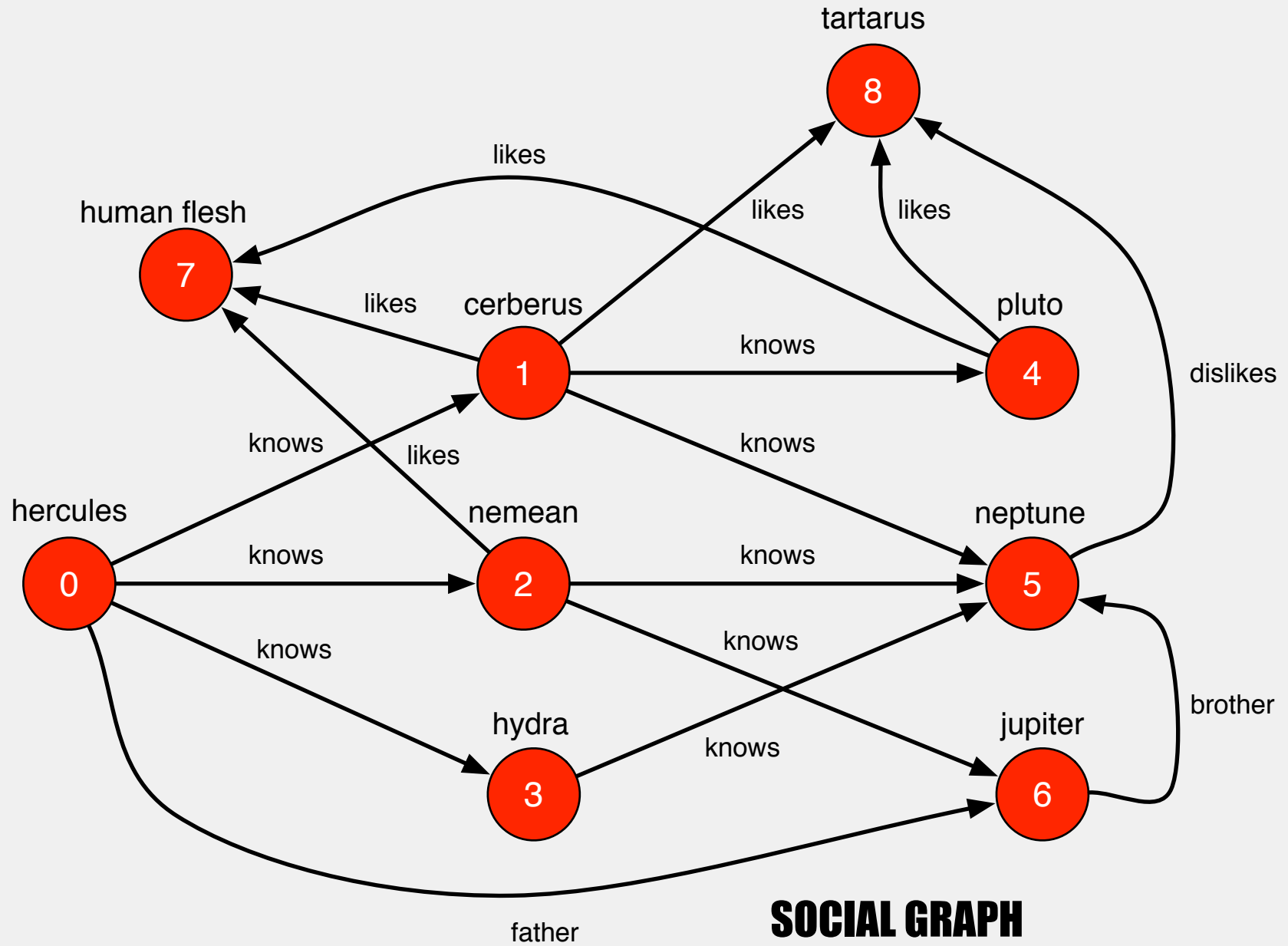




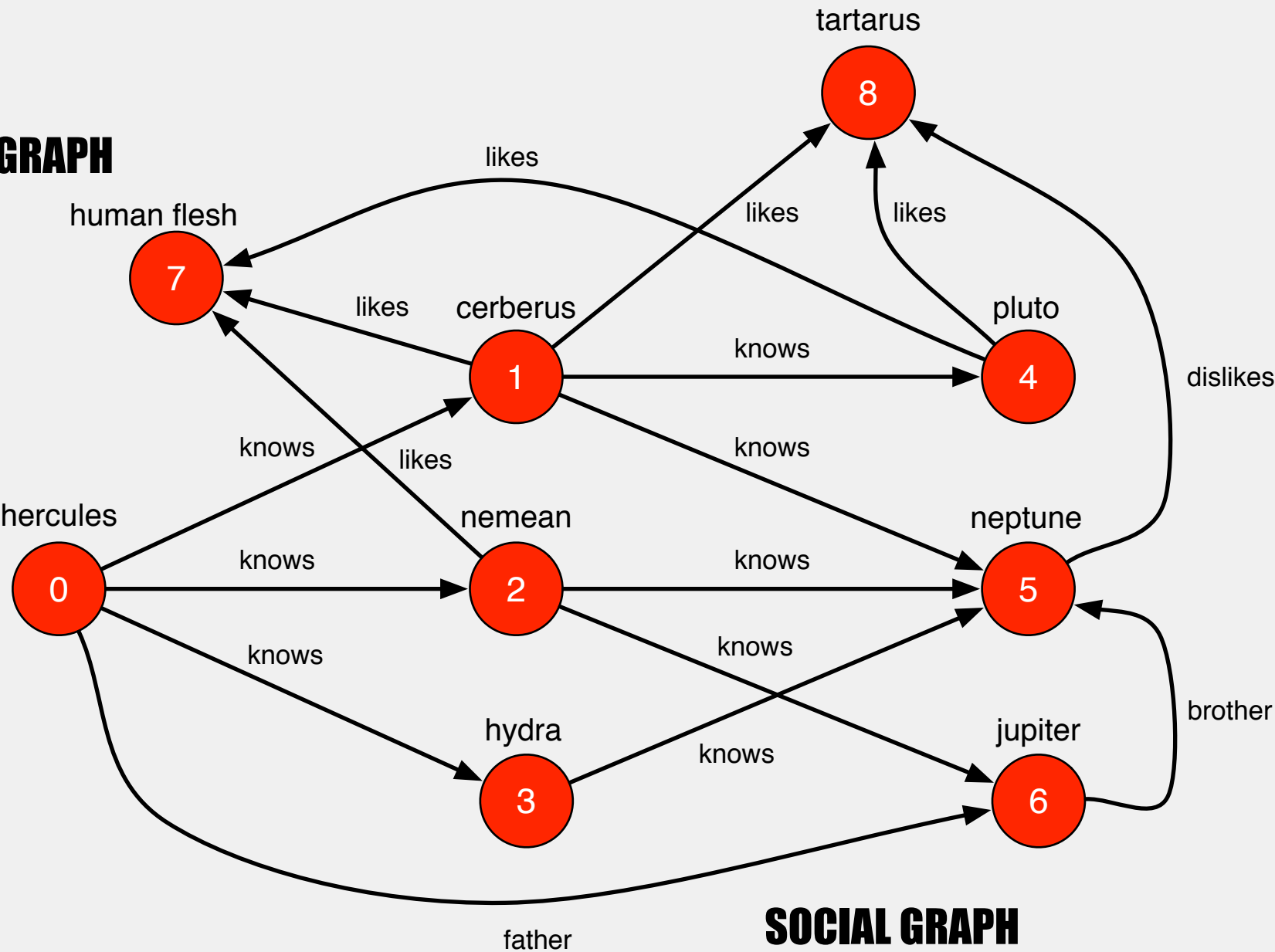




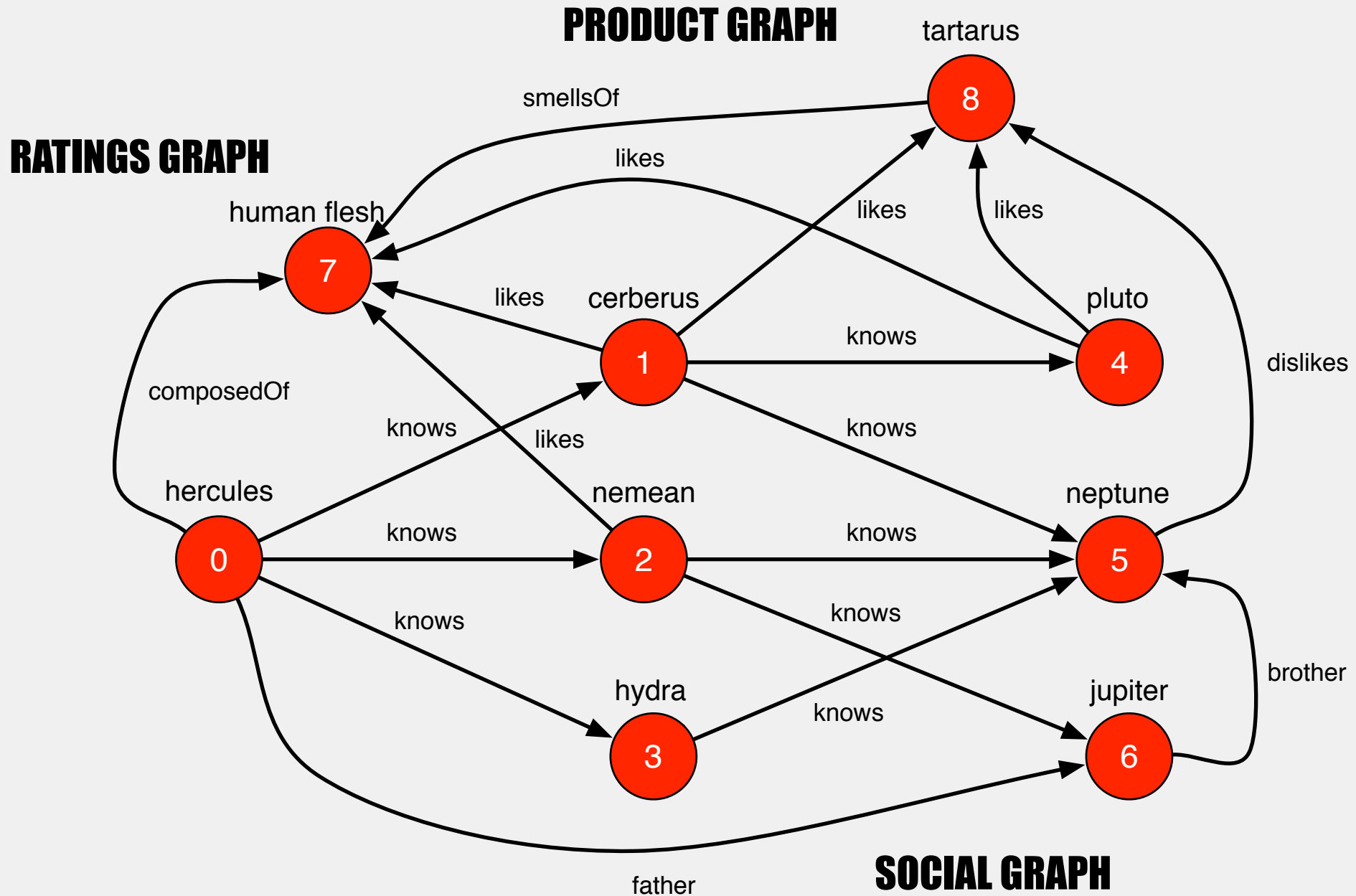




RATINGS GRAPH



NEMEAN MIGHT LIKE TARTARUS



* Collaborative Filtering + Content-Based Recommendation

PATH FINDING



● How is this person related to this film?

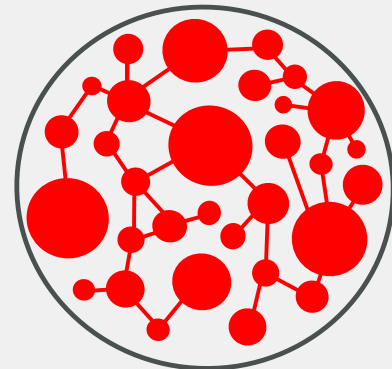
MOVIE GRAPH

● Which authors of this book also wrote a New York Times bestseller?

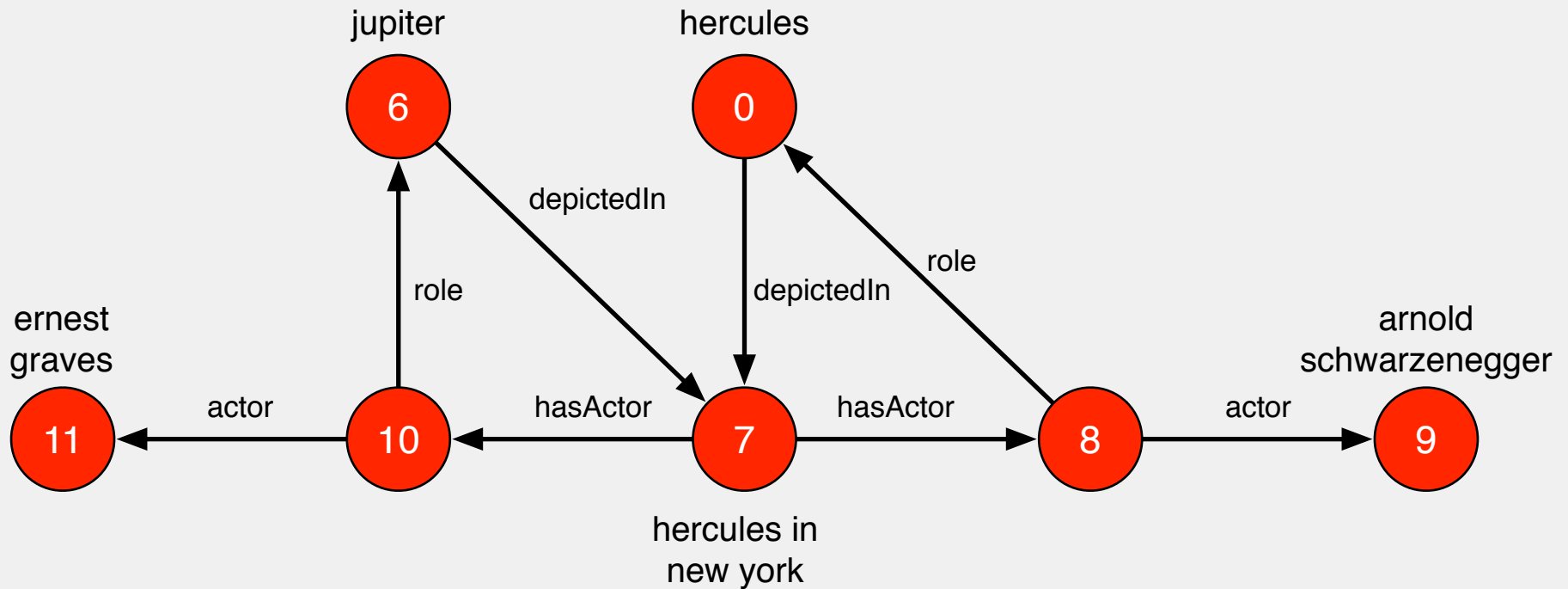
BOOK GRAPH

● Which movies are based on a book by a New York Times bestseller?

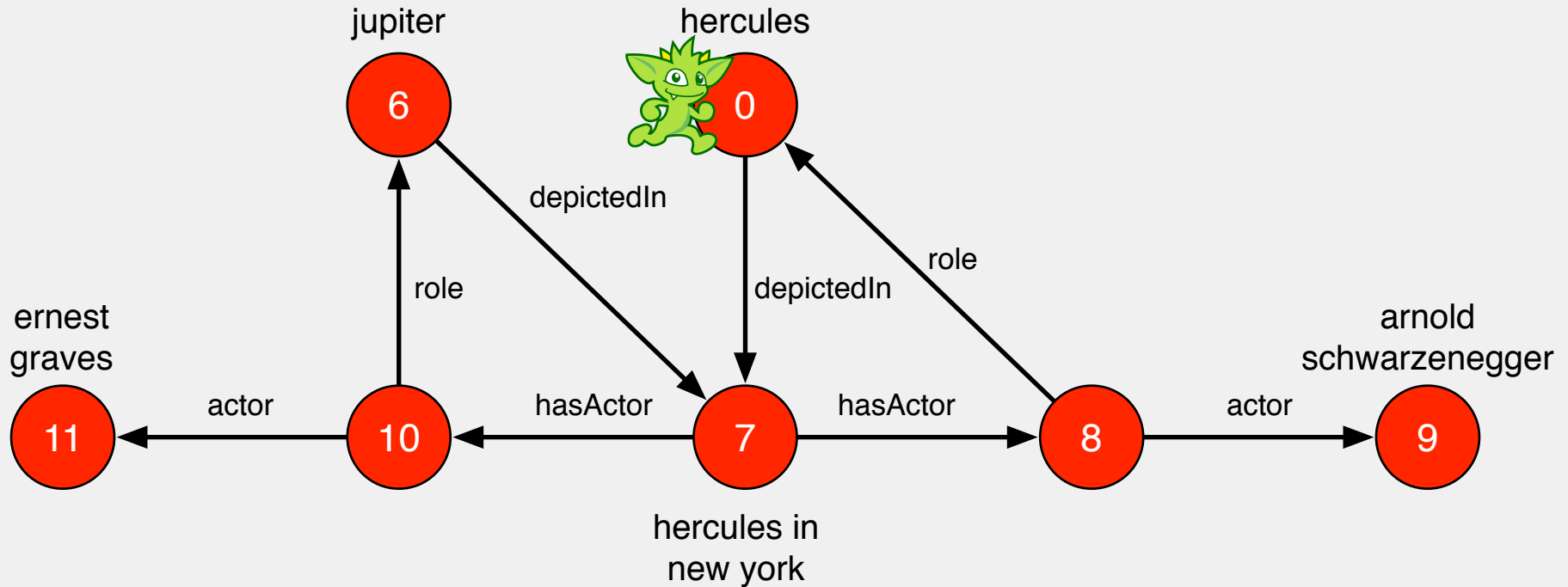
**MOVIE+BOOK
GRAPH**



WHO PLAYED HERCULES IN WHAT MOVIE?

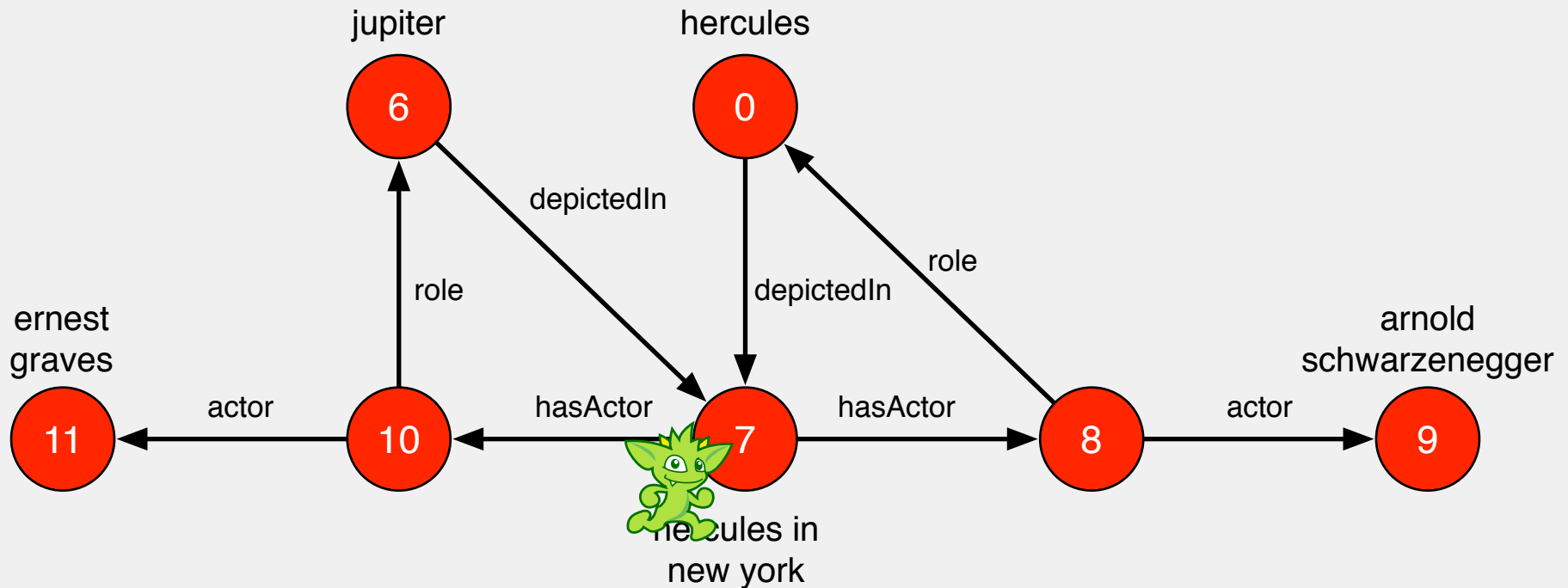


WHO PLAYED HERCULES IN WHAT MOVIE?



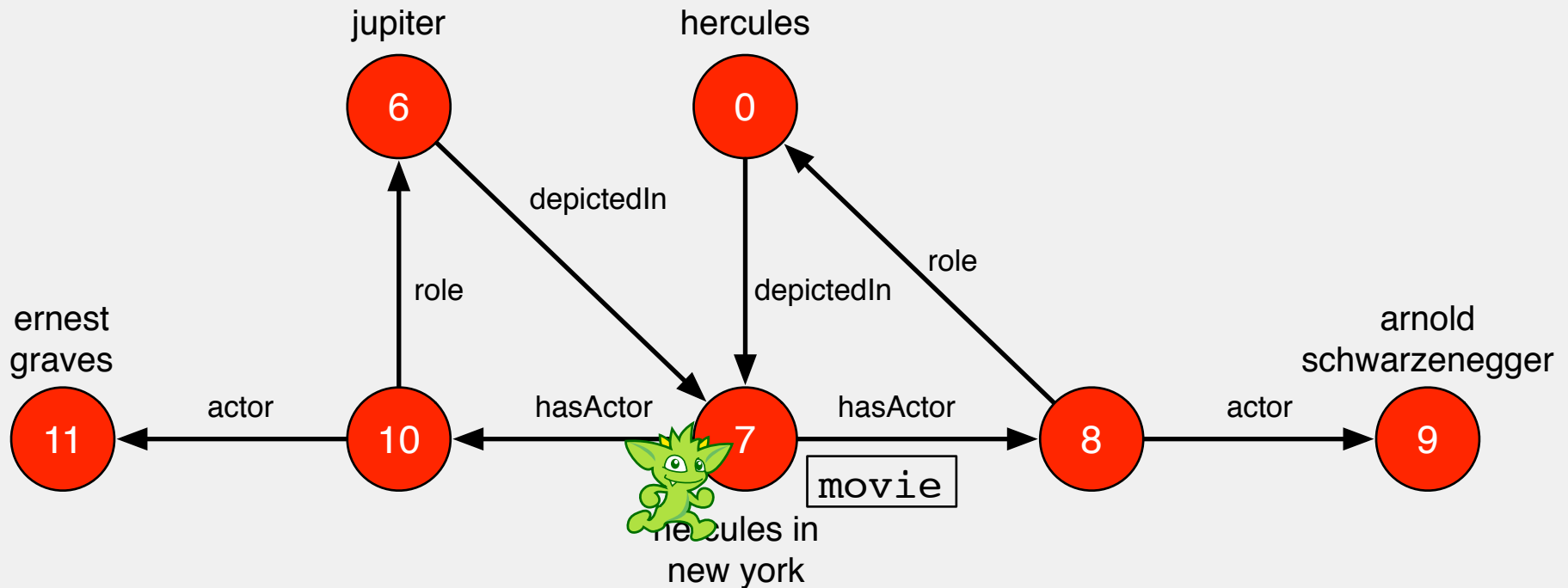
```
gremlin> hercules  
==>v[0]
```

WHO PLAYED HERCULES IN WHAT MOVIE?



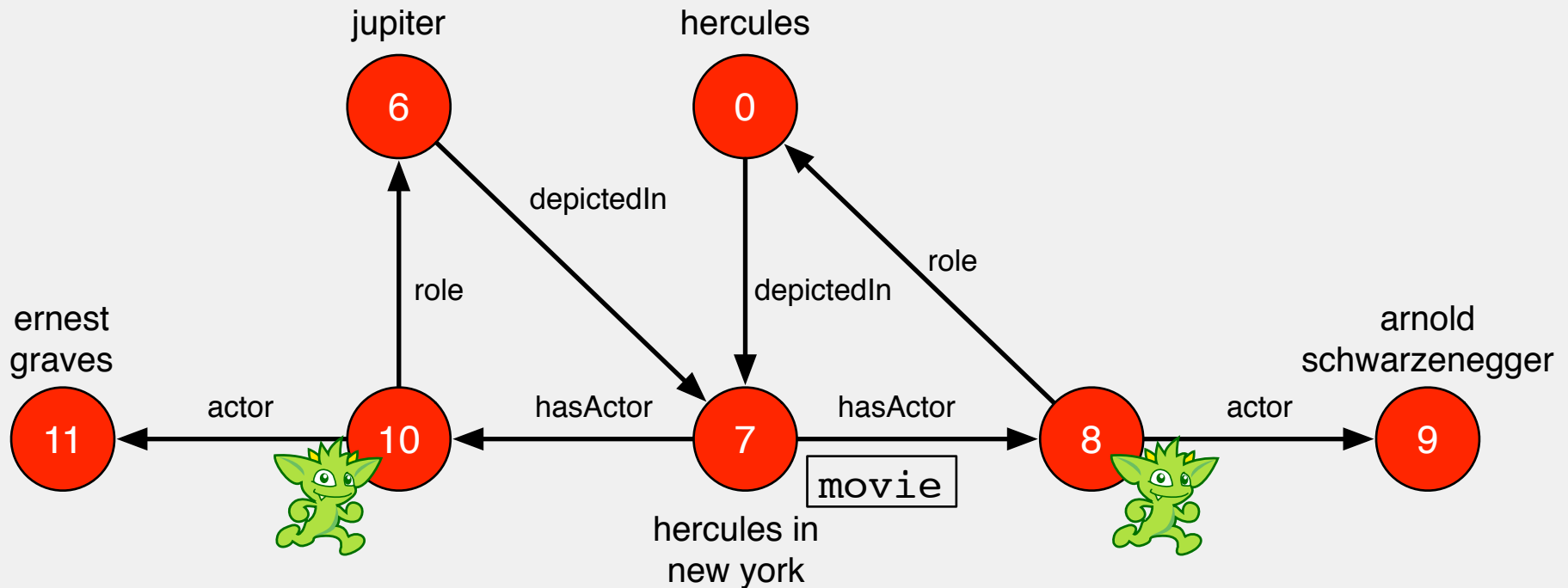
```
gremlin> hercules.out('depictedIn')  
==>v[7]
```

WHO PLAYED HERCULES IN WHAT MOVIE?



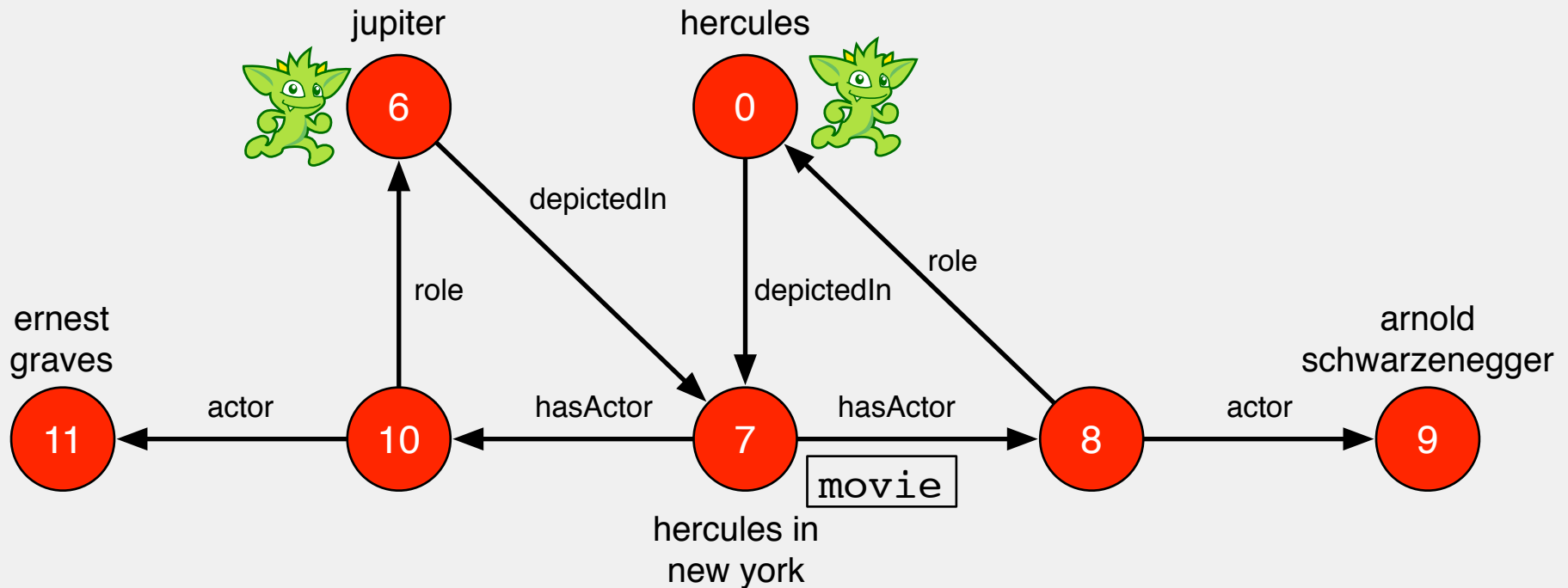
```
gremlin> hercules.out('depictedIn').as('movie')  
==>v[7]
```

WHO PLAYED HERCULES IN WHAT MOVIE?



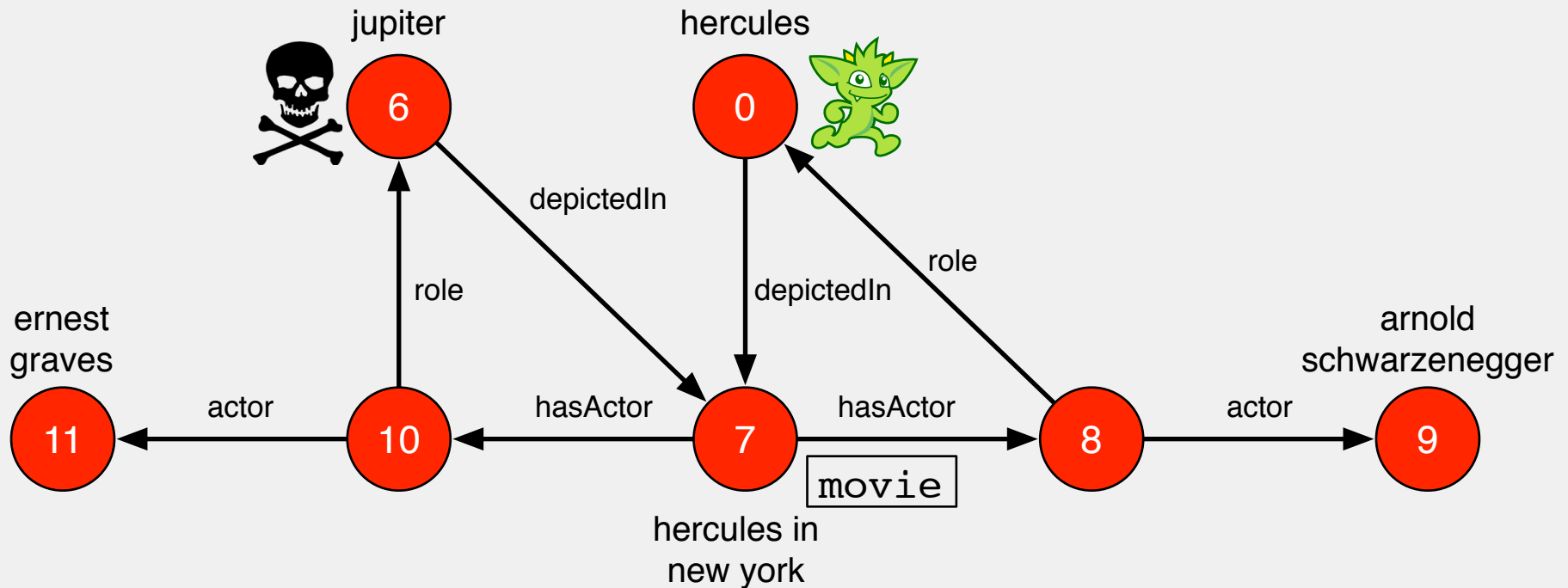
```
gremlin> hercules.out('depictedIn').as('movie').out('hasActor')  
==>v[8]  
==>v[10]
```

WHO PLAYED HERCULES IN WHAT MOVIE?



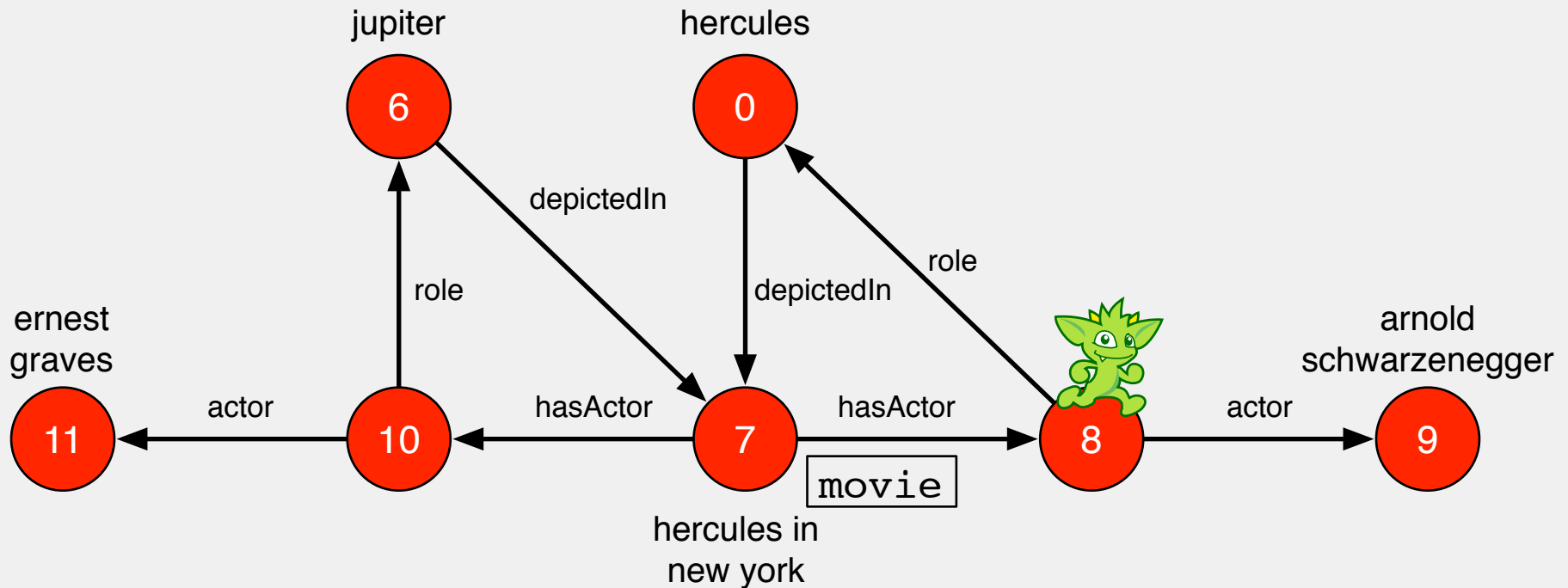
```
gremlin> hercules.out('depictedIn').as('movie').out('hasActor')  
          .out('role')  
==>v[0]  
==>v[6]
```

WHO PLAYED HERCULES IN WHAT MOVIE?



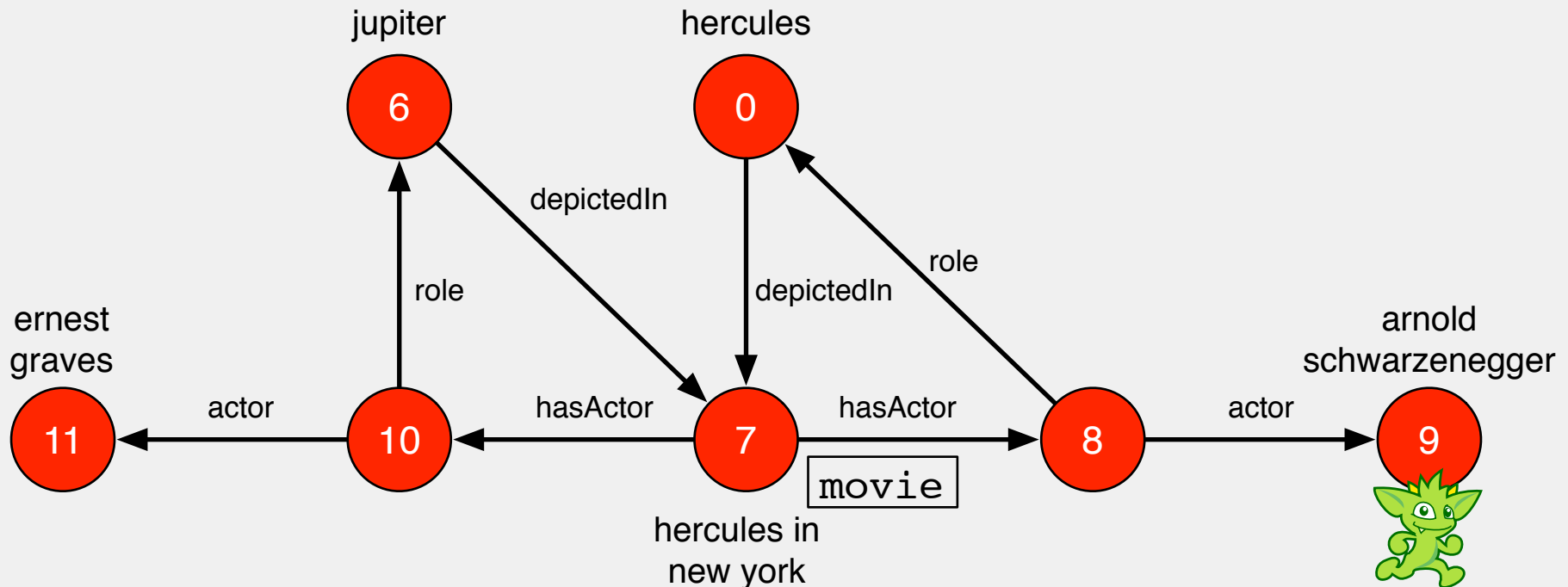
```
gremlin> hercules.out('depictedIn').as('movie').out('hasActor')  
        .out('role').retain(hercules)  
==>v[0]
```

WHO PLAYED HERCULES IN WHAT MOVIE?



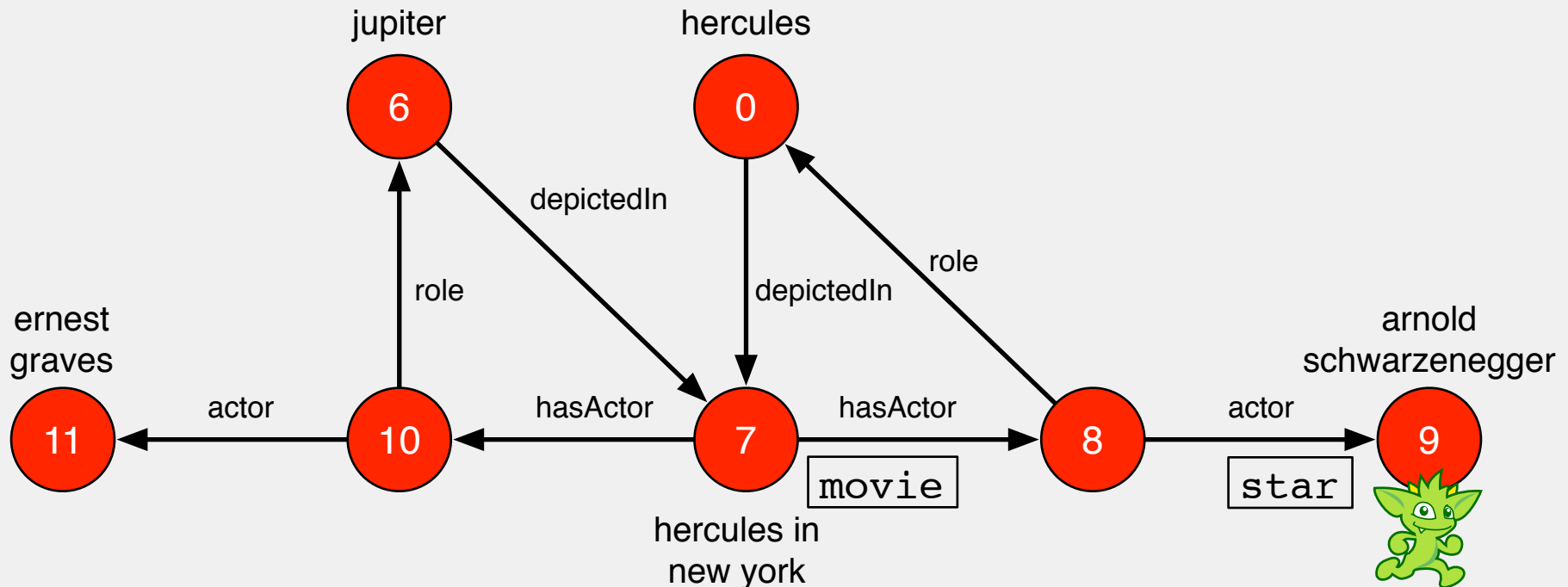
```
gremlin> hercules.out('depictedIn').as('movie').out('hasActor')  
        .out('role').retain(hercules).back(2)  
==>v[8]
```

WHO PLAYED HERCULES IN WHAT MOVIE?



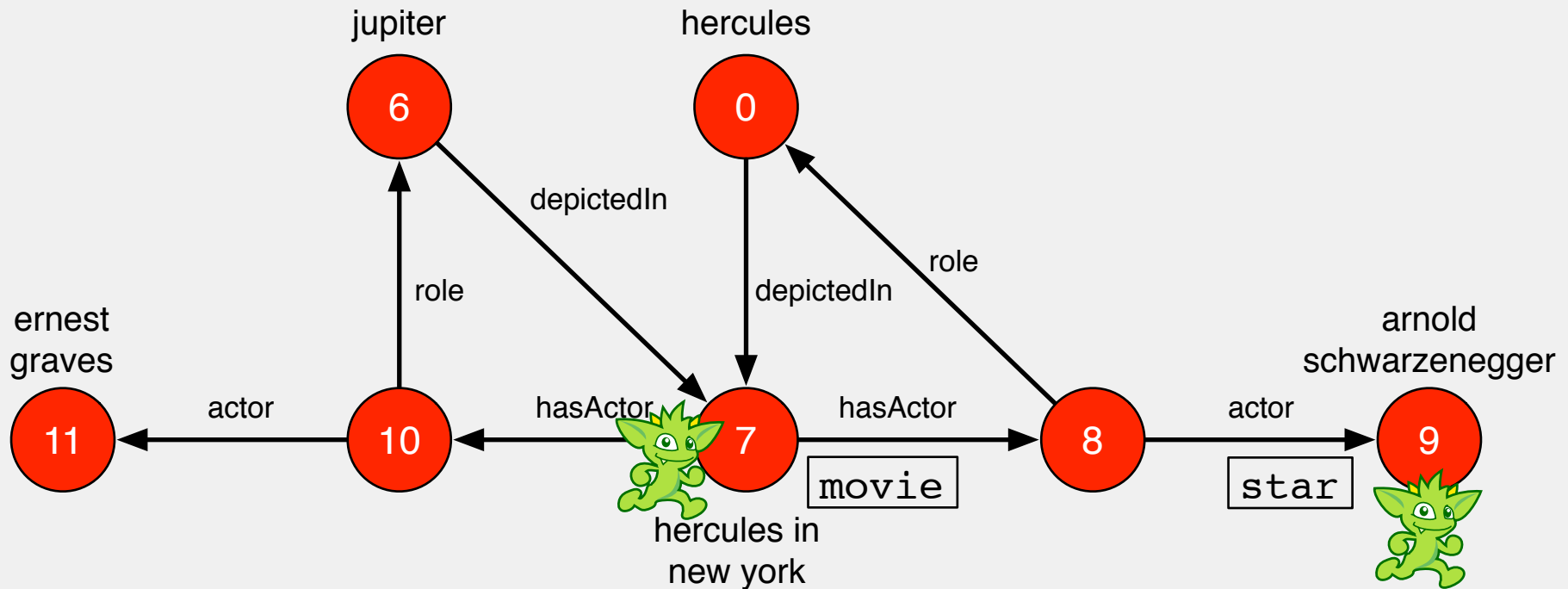
```
gremlin> hercules.out('depictedIn').as('movie').out('hasActor')  
        .out('role').retain(hercules).back(2).out('actor')  
==>v[9]
```


WHO PLAYED HERCULES IN WHAT MOVIE?



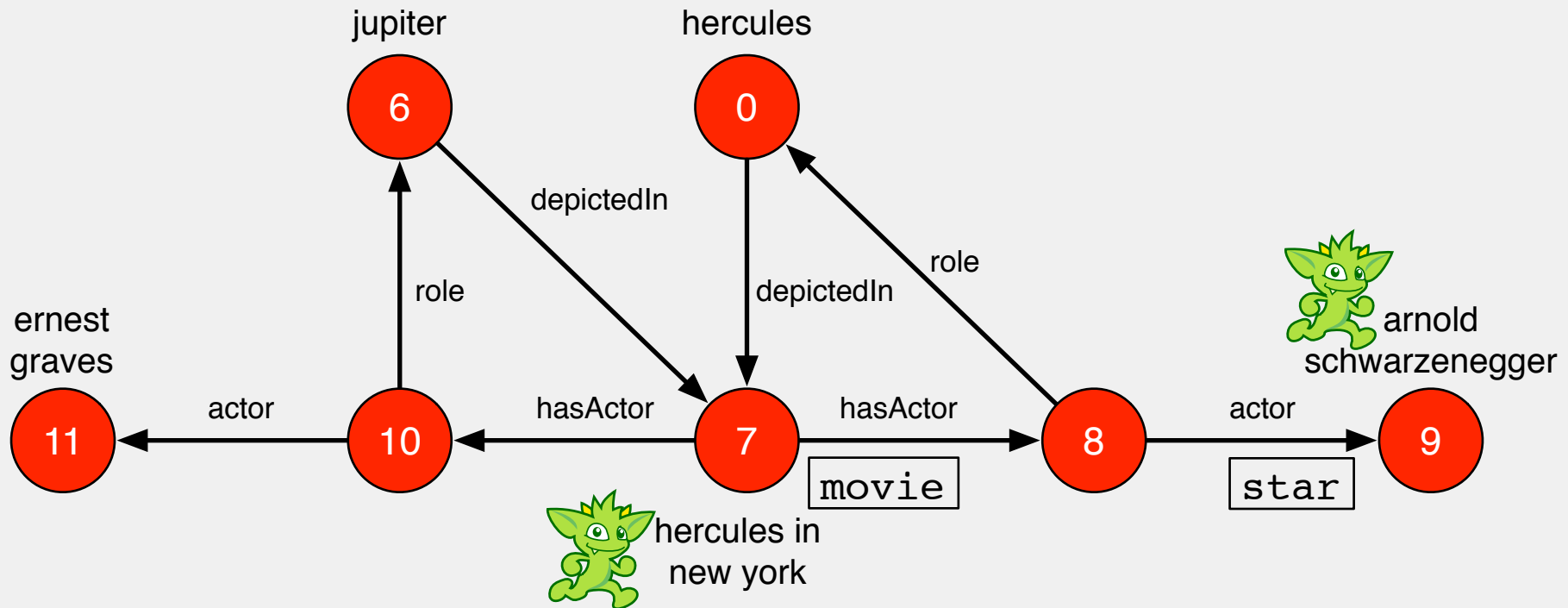
```
gremlin> hercules.out('depictedIn').as('movie').out('hasActor')  
        .out('role').retain(hercules).back(2).out('actor')  
        .as('star')  
==>v[9]
```

WHO PLAYED HERCULES IN WHAT MOVIE?

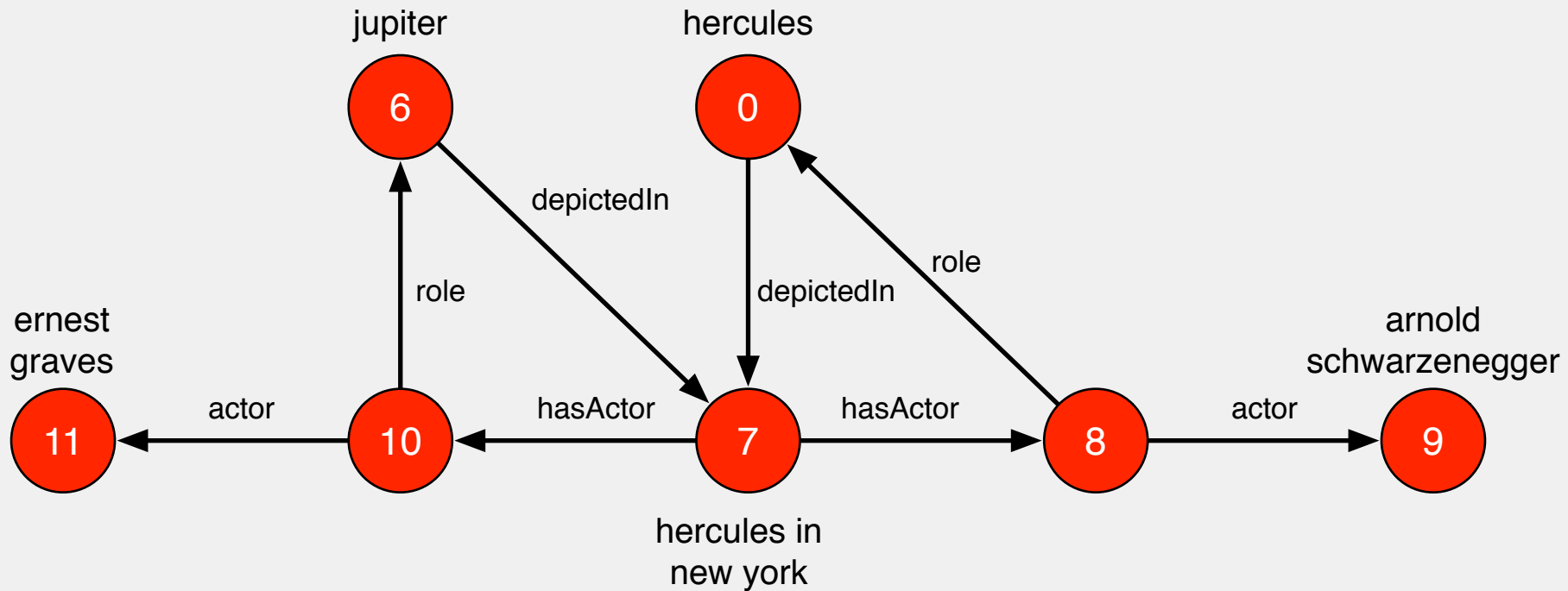


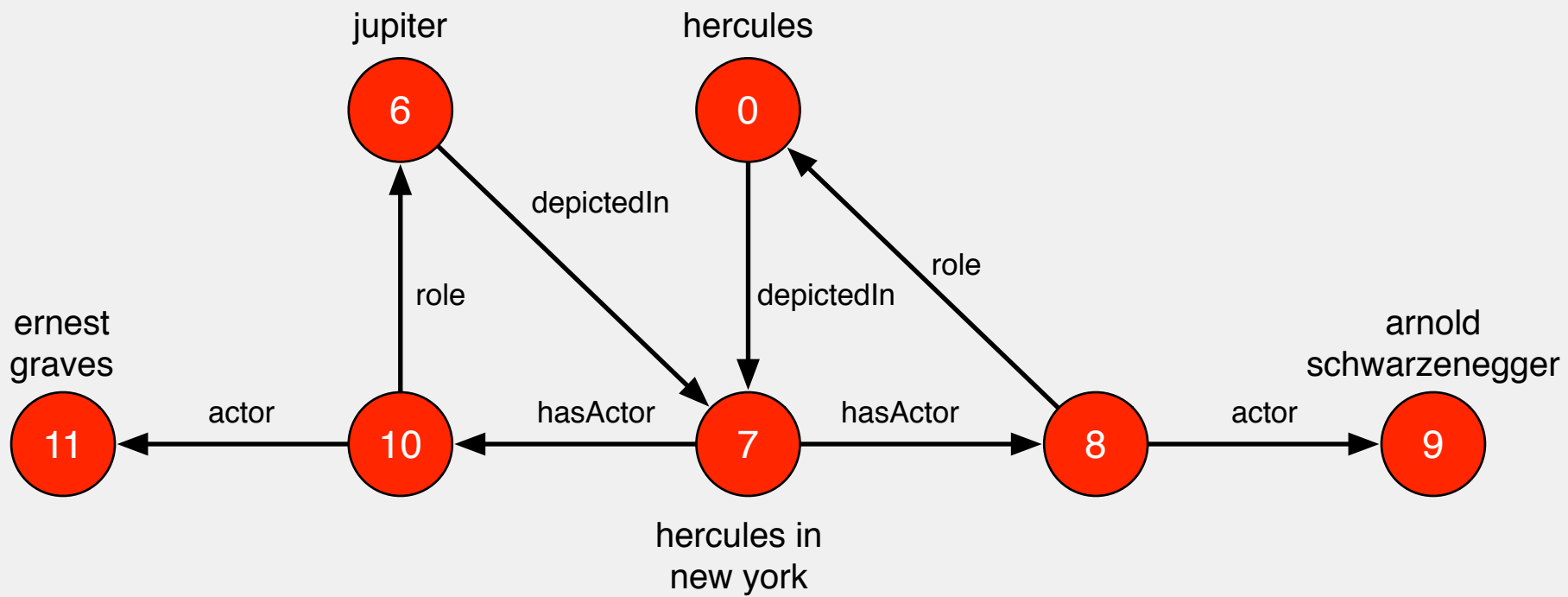
```
gremlin> hercules.out('depictedIn').as('movie').out('hasActor')  
        .out('role').retain(hercules).back(2).out('actor')  
        .as('star').select  
==>[movie:v[7], star:v[9]]
```

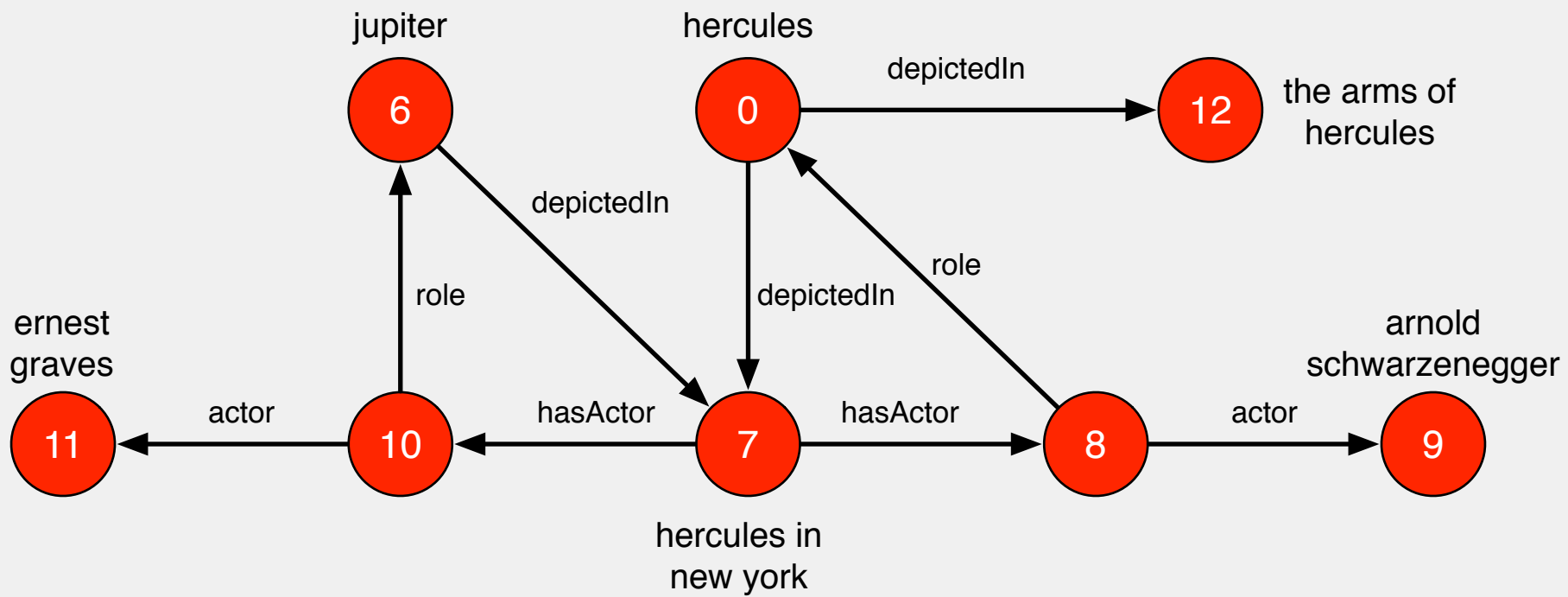
WHO PLAYED HERCULES IN WHAT MOVIE?

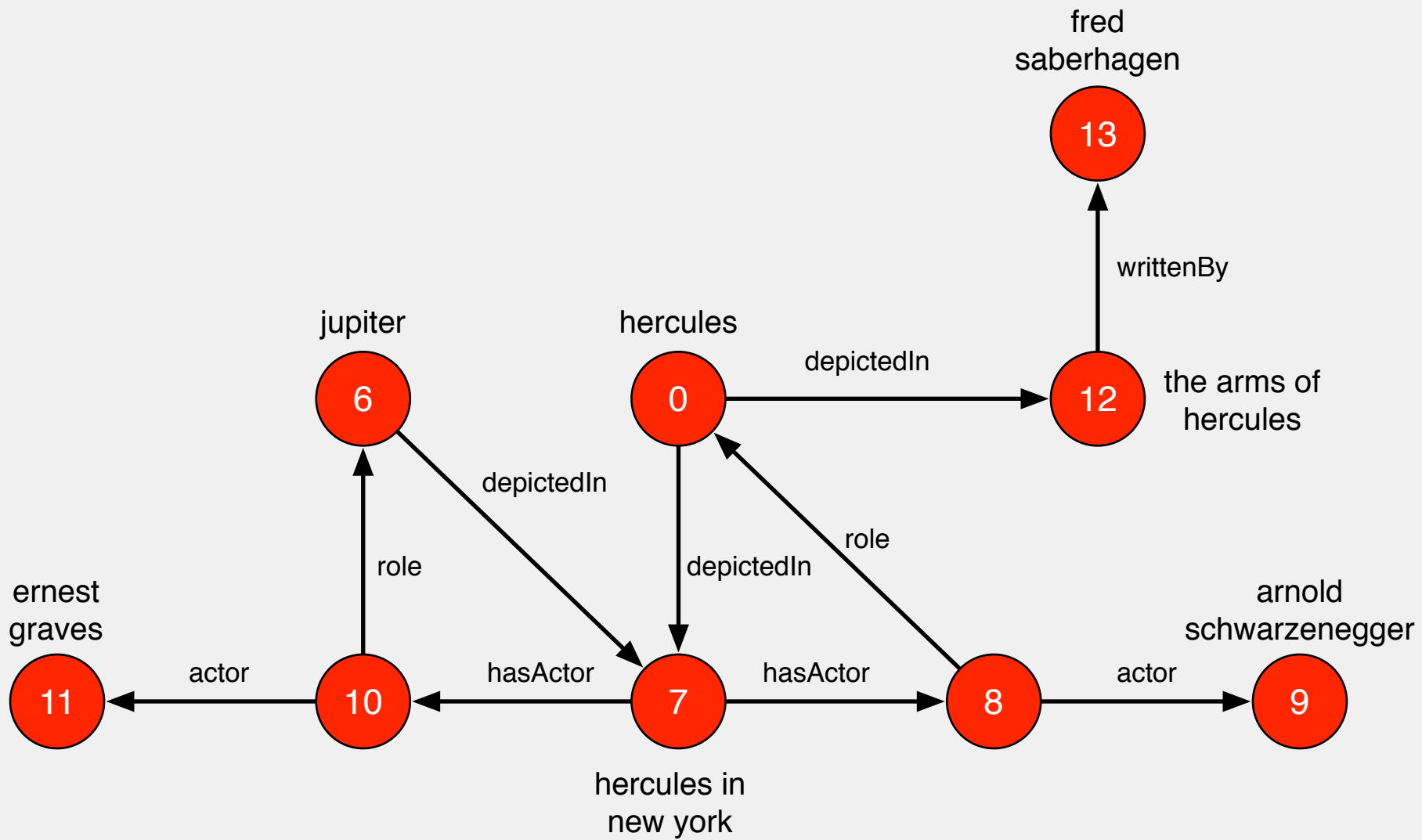


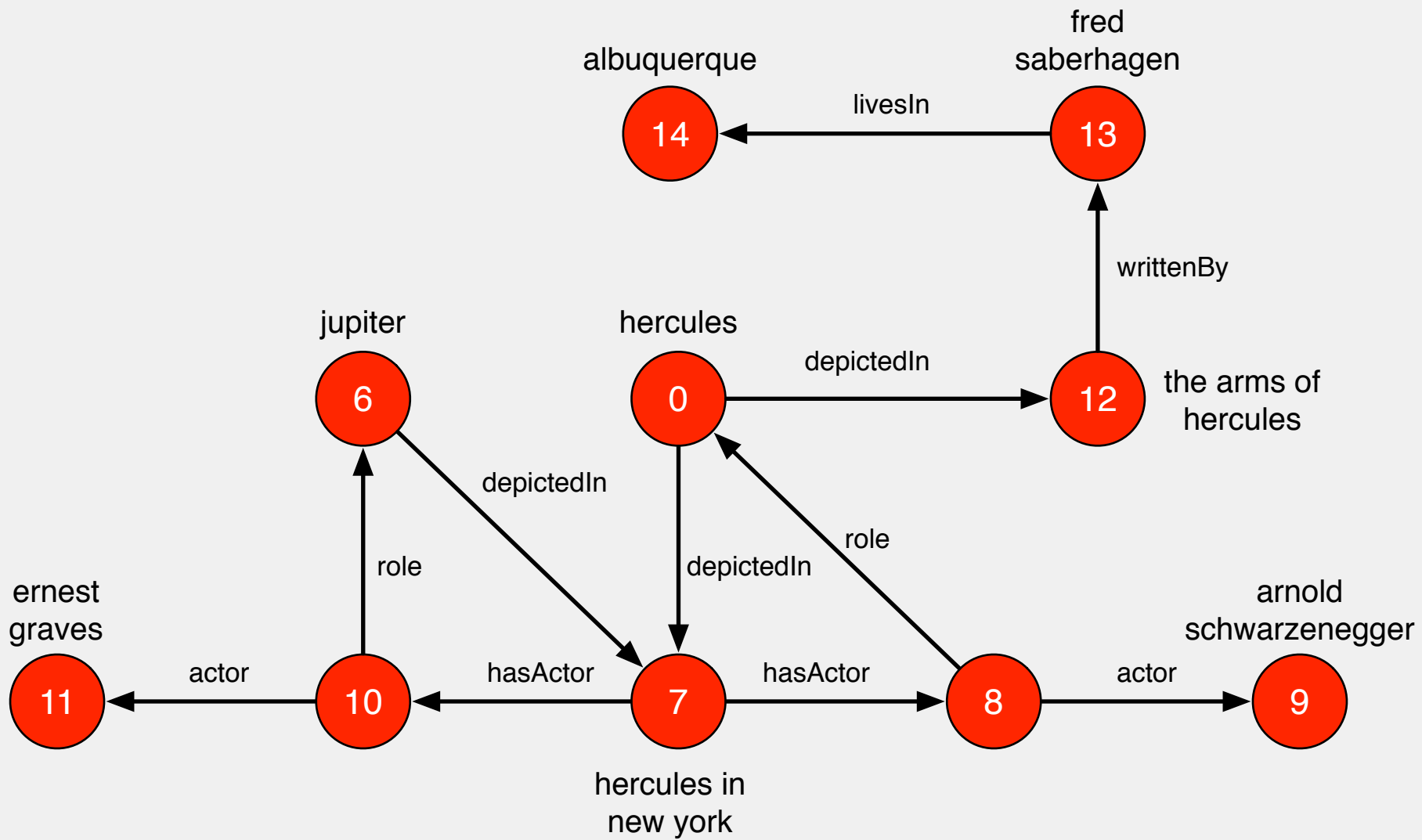
```
gremlin> hercules.out('depictedIn').as('movie').out('hasActor')  
        .out('role').retain(hercules).back(2).out('actor')  
        .as('star').select{it.name}  
==>[movie:hercules in new york, star:arnold schwarzenegger]
```

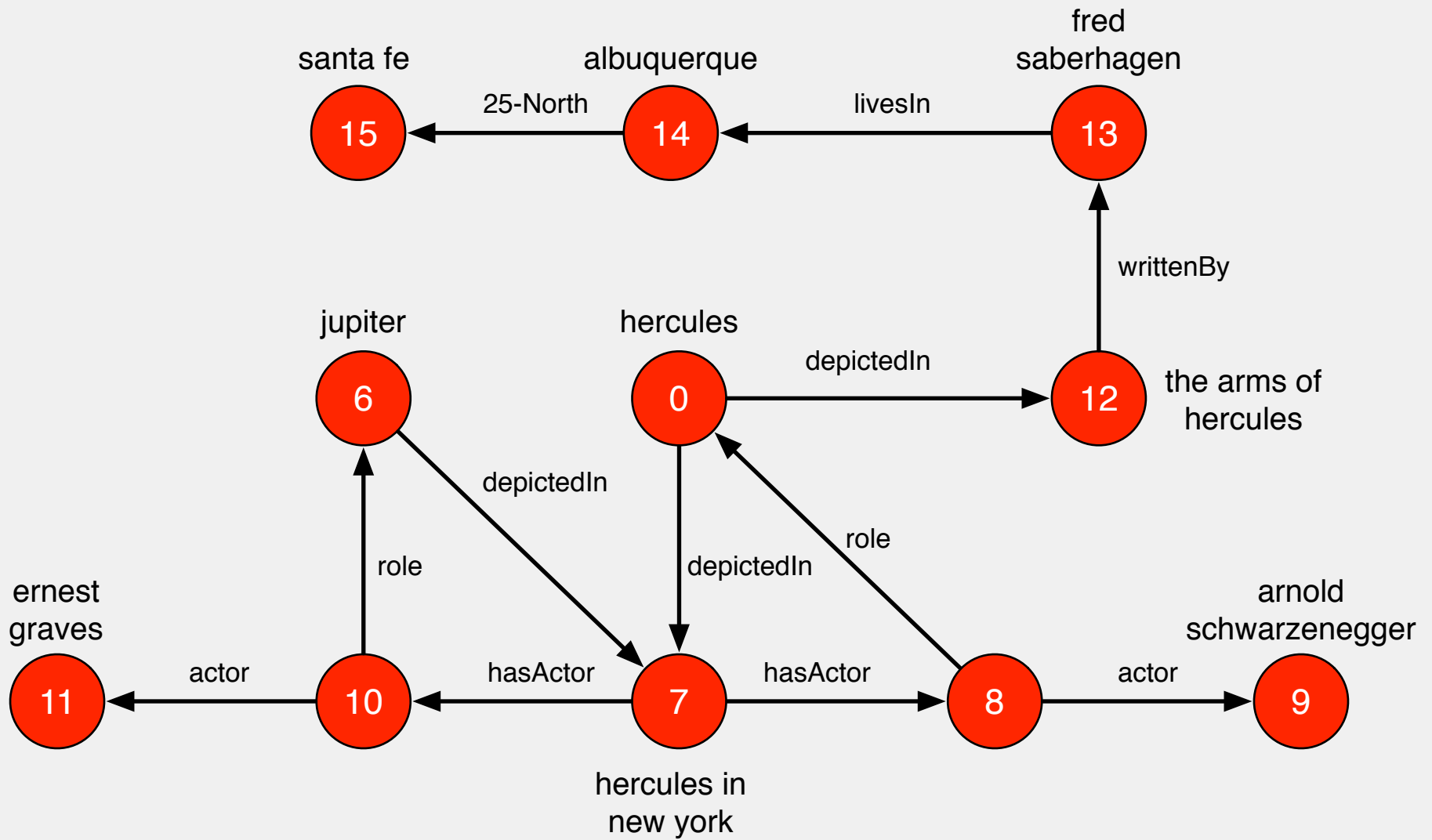


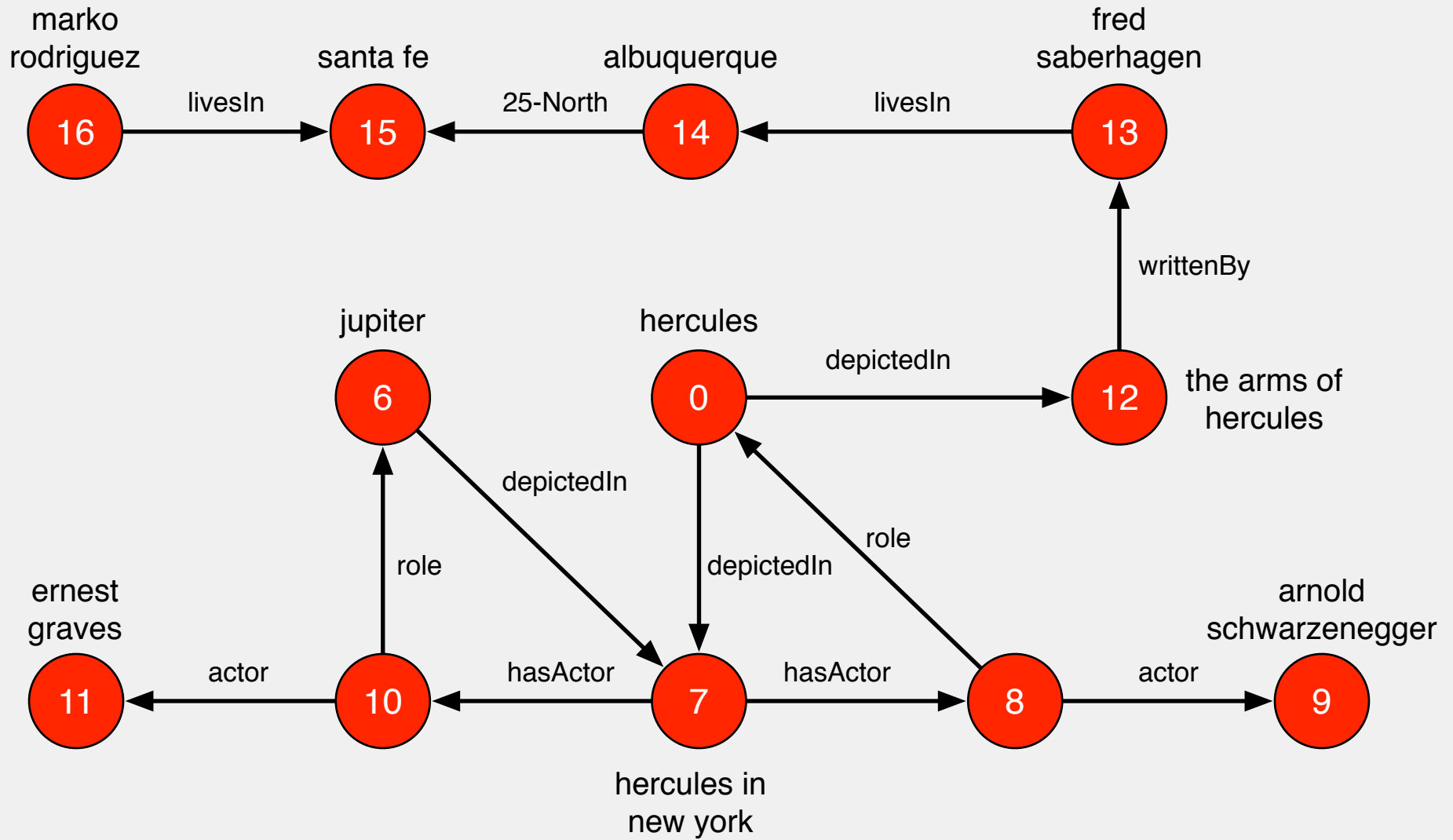


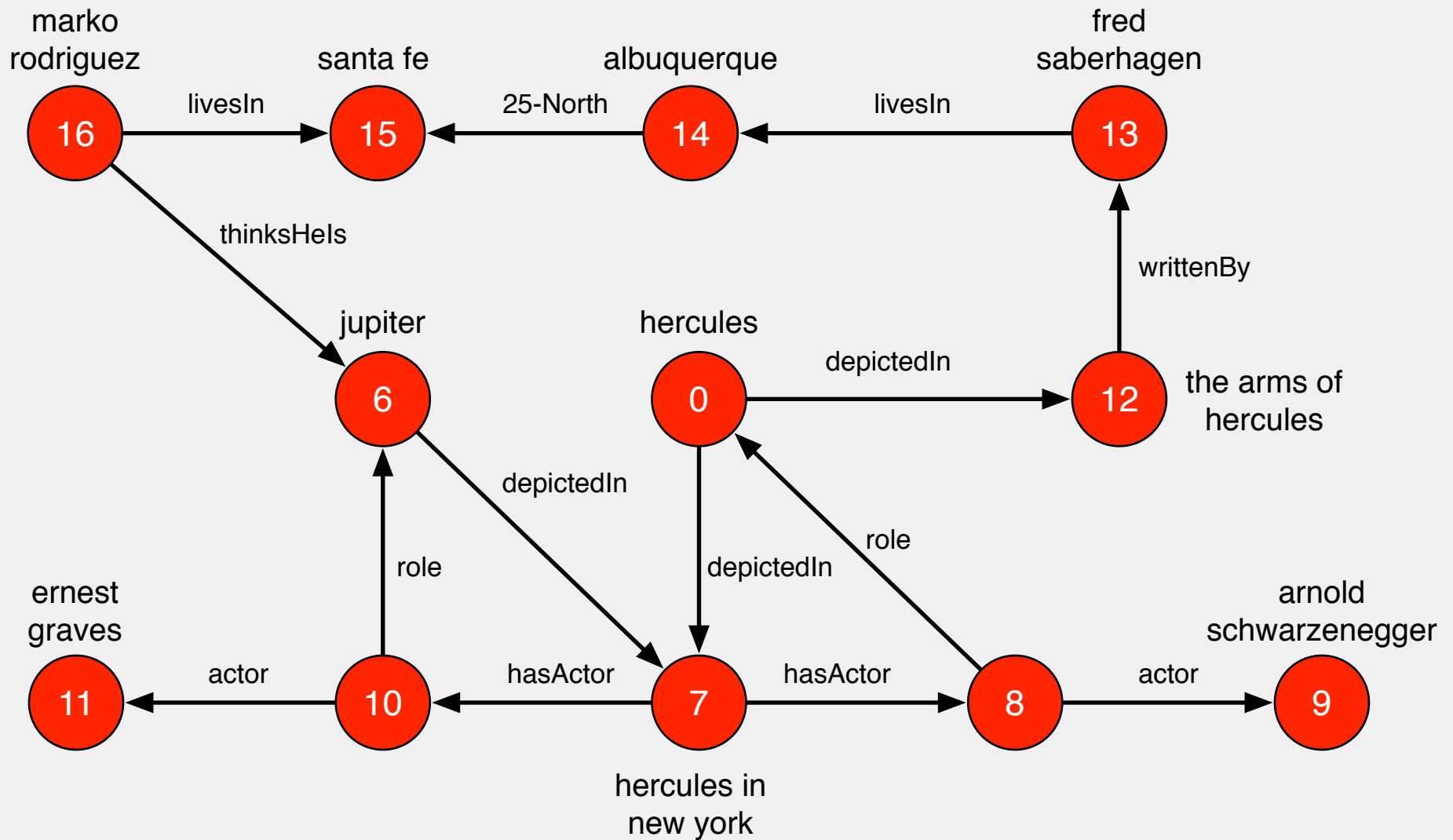




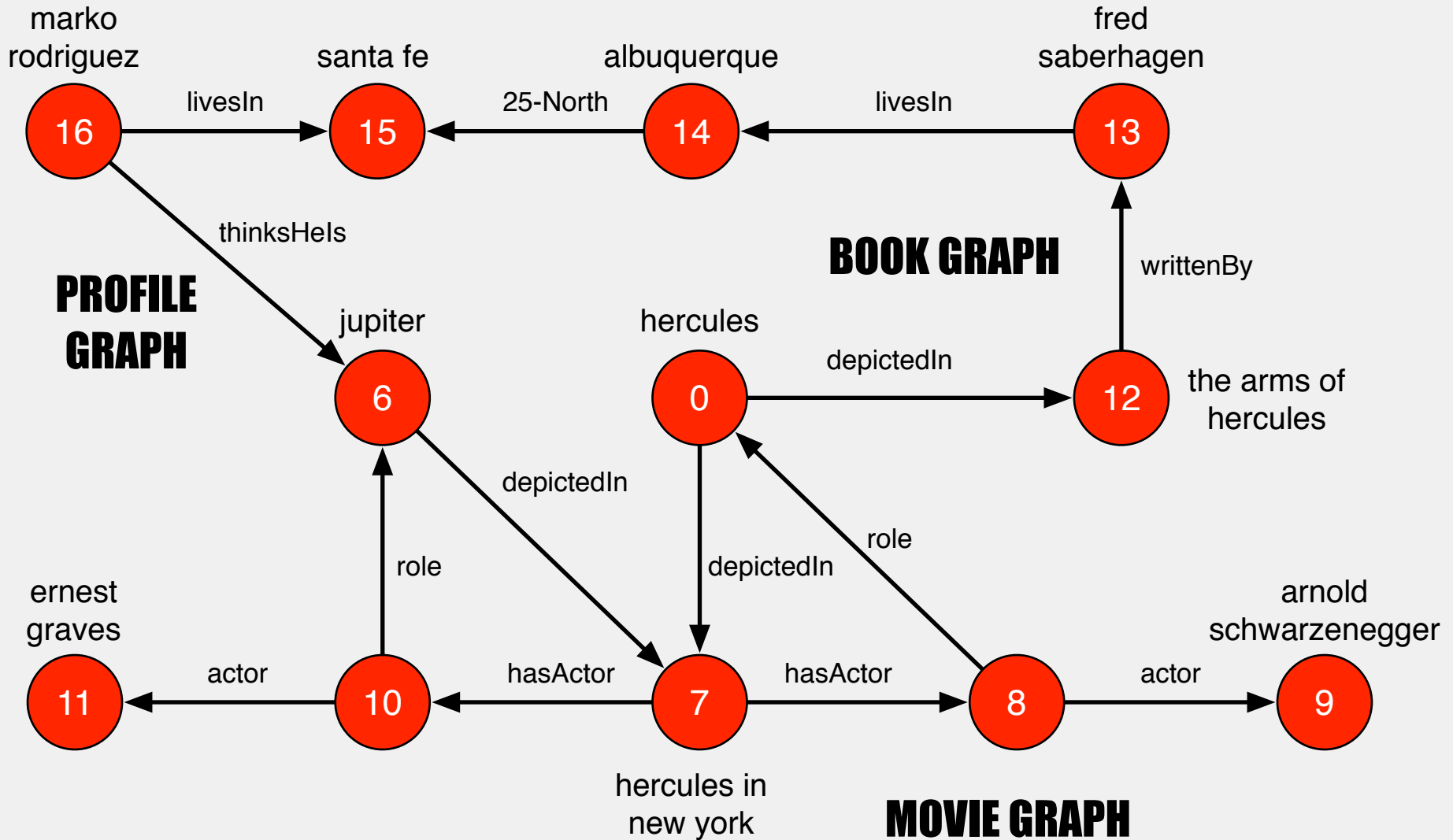








TRANSPORTATION GRAPH

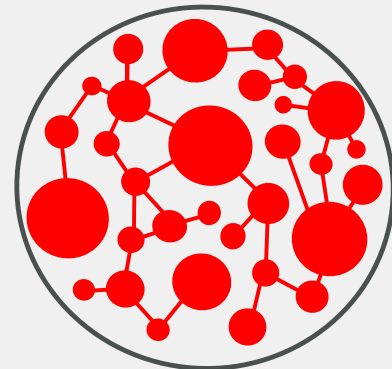


SOCIAL INFLUENCE



- Who are the most influential people in java, mathematics, art, surreal art, politics, ...?
- Which region of the social graph will propagate this advertisement this furthest?
- Which 3 experts should review this submitted article?
- Which people should I talk to at the upcoming conference and what topics should I talk to them about?

SOCIAL + COMMUNICATION + EXPERTISE + EVENT GRAPH



PATTERN IDENTIFICATION

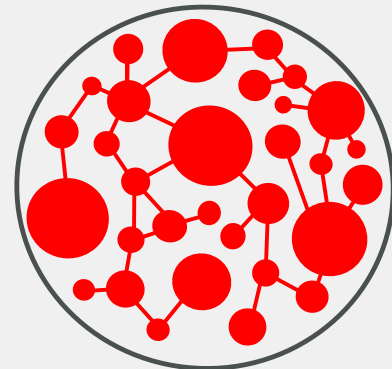


- This connectivity pattern is a sign of financial fraud.
When this motif is found, a red flag will be raised.

TRANSACTION GRAPH

- Healthy discourse is typified by a discussion board with a branch factor in this range and a concept clique score in this range.

DISCUSSION GRAPH



KNOWLEDGE DISCOVERY

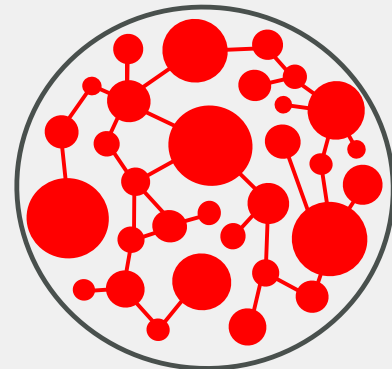


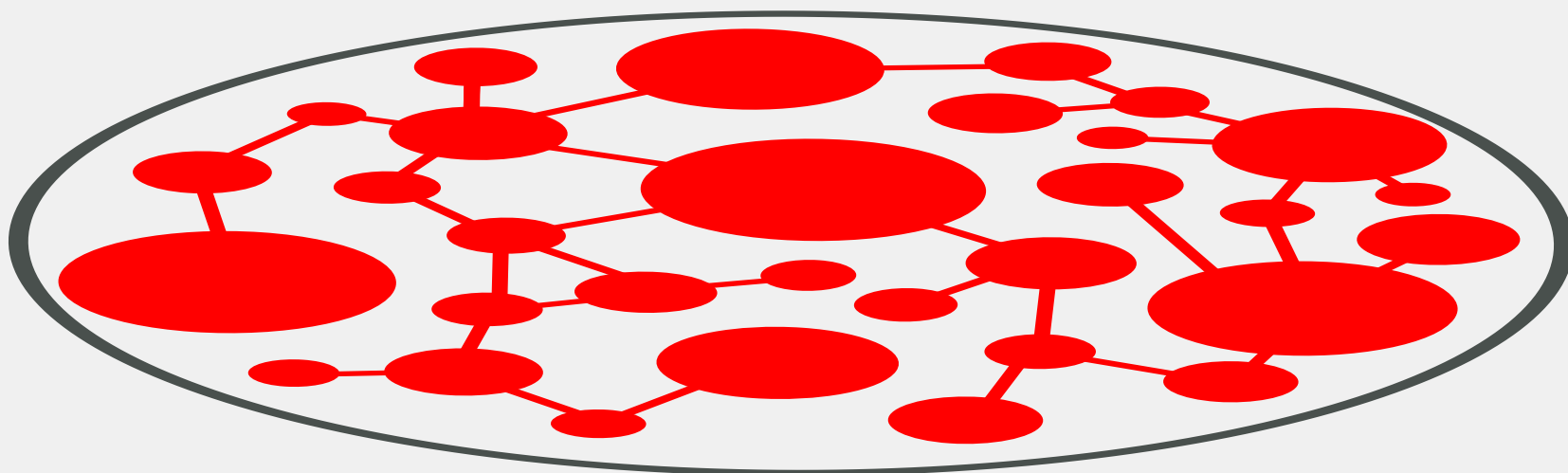
- The terms "ice", "fans", "stanley cup," are classified as "sports"

WIKIPEDIA GRAPH

- Given that all identified birds fly,
it can be deduced that all birds fly.
If contrary evidence is provided,
then this "fact" can be retracted.

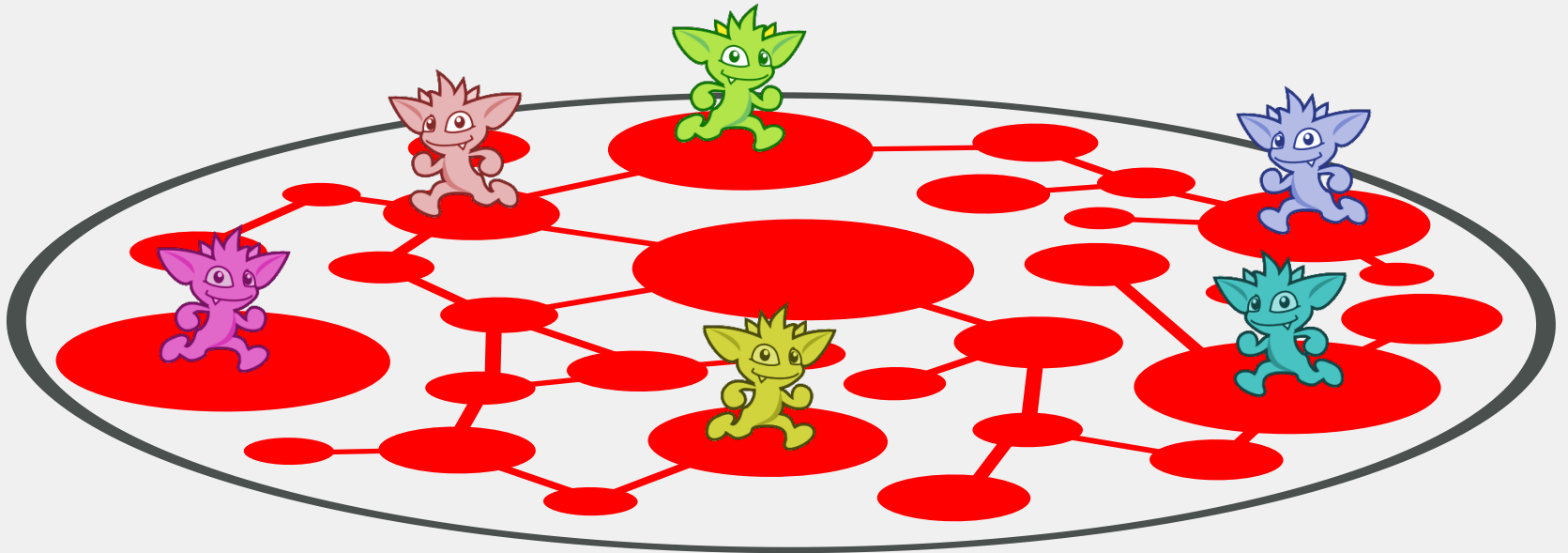
EVIDENTIAL LOGIC GRAPH





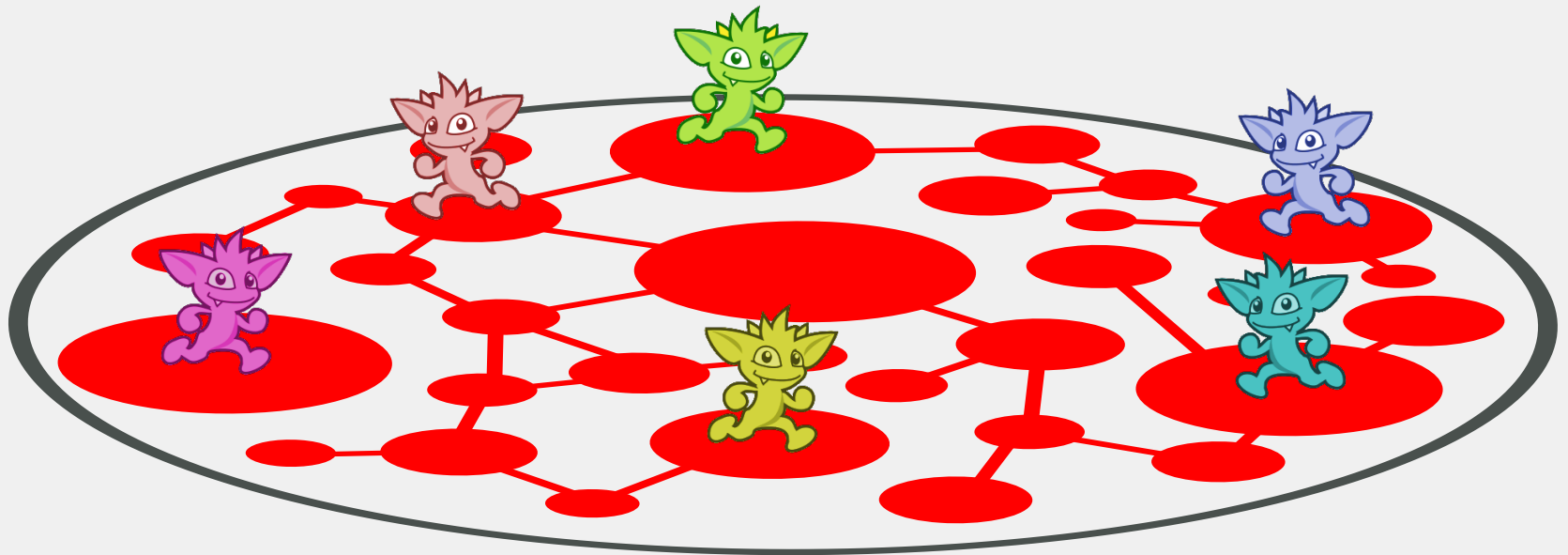
WORLD MODEL

WORLD PROCESSES



WORLD MODEL

WORLD PROCESSES



WORLD MODEL

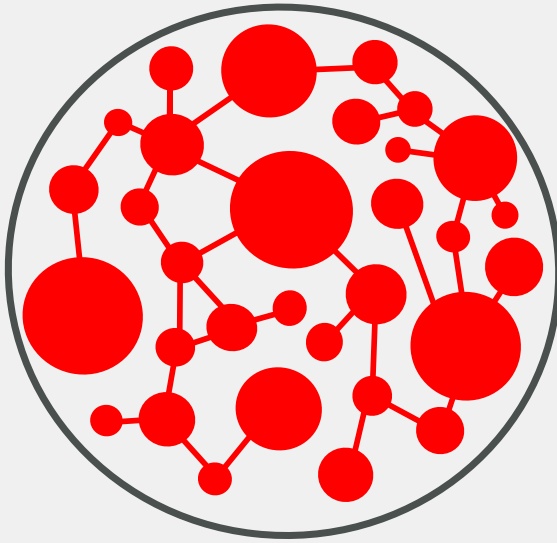
A single world model and various types of traversers moving through that model to solve problems.

COMPUTING

PROCESS



STRUCTURE



TRAVERSAL

GRAPH-BASED
COMPUTING

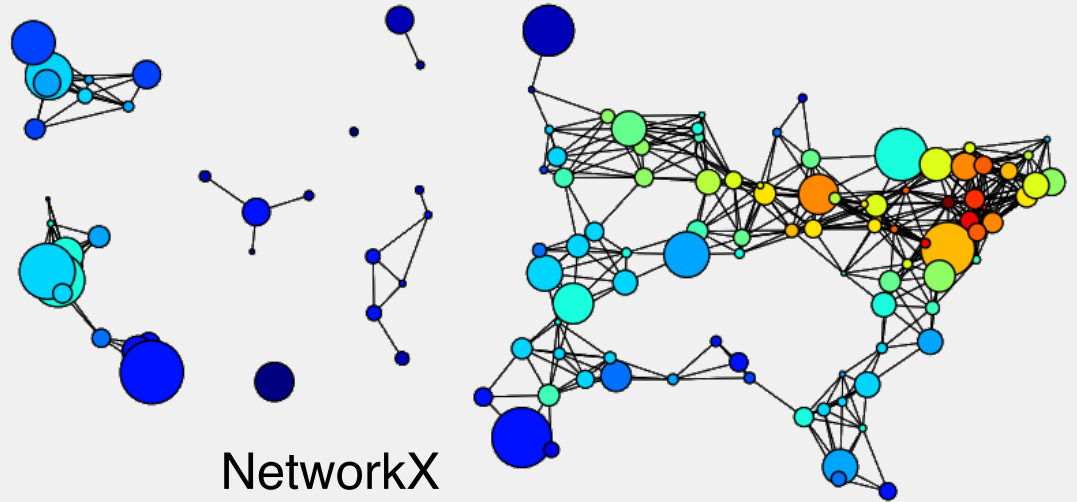
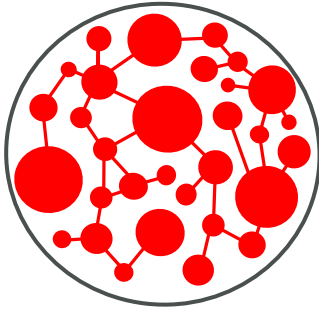
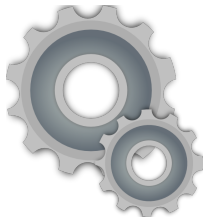
GRAPH

GRAPH COMPUTING ENGINES

MEMORY-BASED GRAPHS

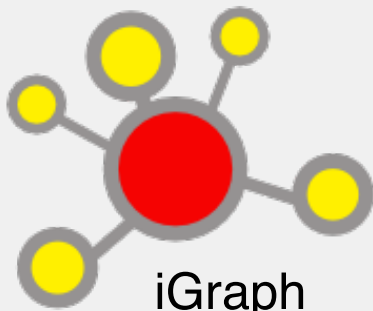
Graph Framework

Application



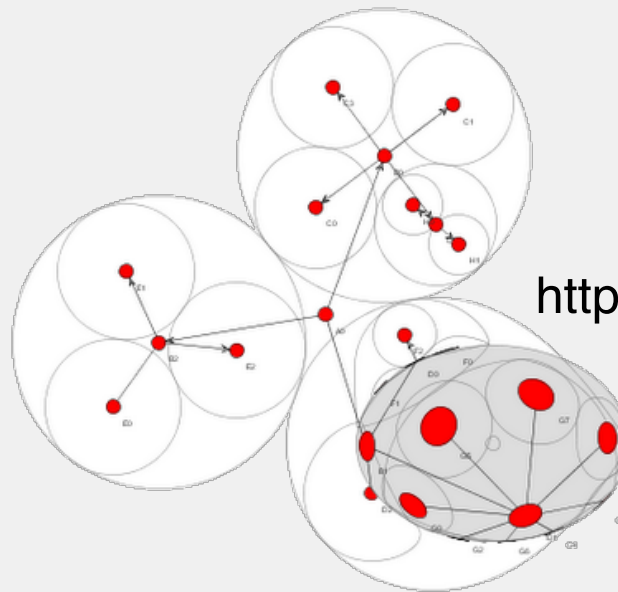
NetworkX

<http://networkx.lanl.gov/>



iGraph

<http://igraph.sourceforge.net/>

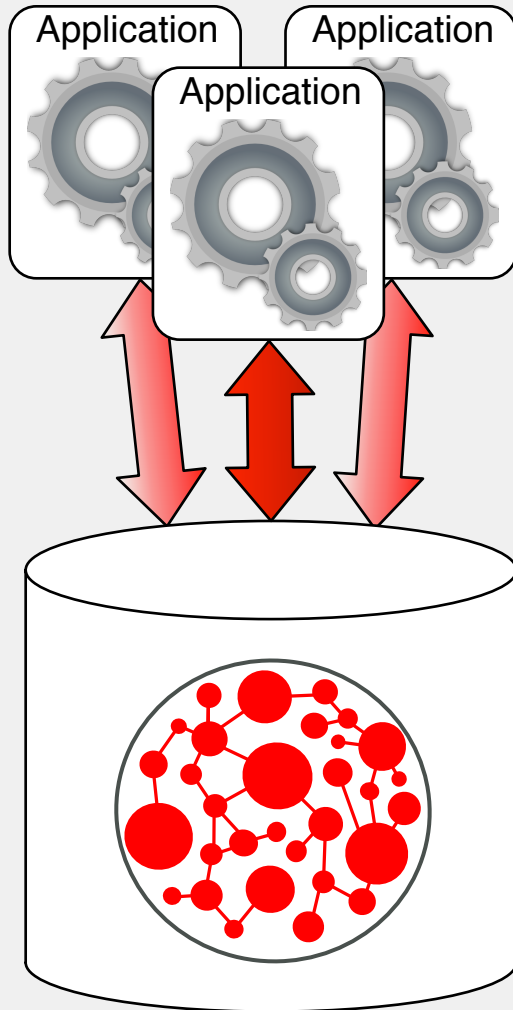


JUNG

<http://jung.sourceforge.net/>

DISK-BASED GRAPHS

Graph Database



Neo4j
<http://neo4j.org/>



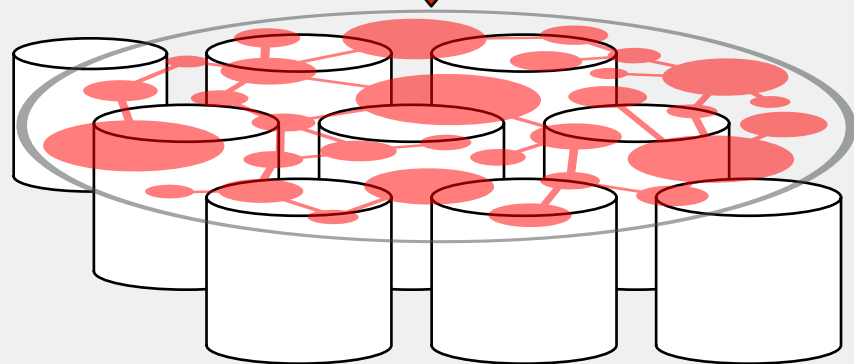
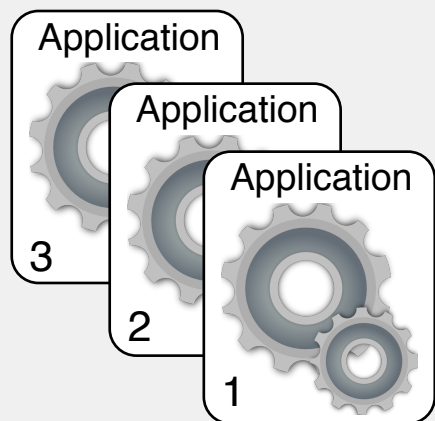
InfiniteGraph
<http://objectivity.com>



<http://www.sparsity-technologies.com/dex>

CLUSTER-BASED GRAPHS

Bulk Synchronous Parallel Processing



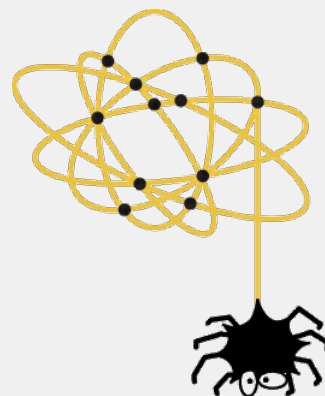
Hama

<http://incubator.apache.org/hama/>



Giraph

<http://incubator.apache.org/giraph/>



GOLDENORB

GoldenOrb

<http://goldenorbos.org/>

* In the same spirit as Google's Pregel

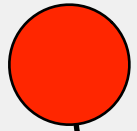


MEMORY-BASED GRAPHS

Graph size is constrained by local machine's RAM.

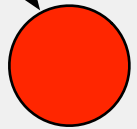
Rich graph algorithm and visualization packages.

Oriented towards "textbook-style" graphs.



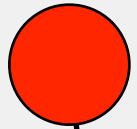
MEMORY-BASED GRAPHS

Graph size is constrained by local machine's RAM.
Rich graph algorithm and visualization packages.
Oriented towards "textbook-style" graphs.



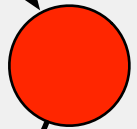
DISK-BASED GRAPHS

Graph size is constrained by local disk.
Optimized for local graph algorithms.
Oriented towards property graphs.



MEMORY-BASED GRAPHS

Graph size is constrained by local machine's RAM.
Rich graph algorithm and visualization packages.
Oriented towards "textbook-style" graphs.



DISK-BASED GRAPHS

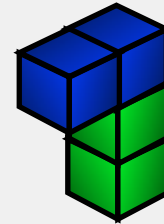
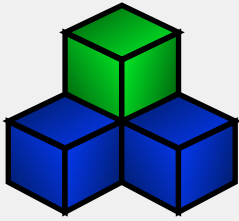
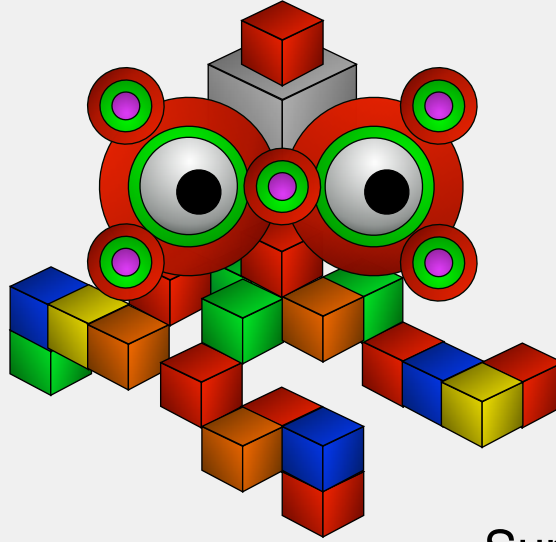
Graph size is constrained by local disk.
Optimized for local graph algorithms.
Oriented towards property graphs.



CLUSTER-BASED GRAPHS

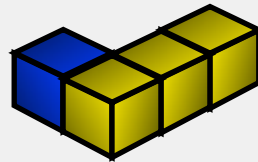
Graph size is constrained to cluster's total RAM.
Optimized for global graph algorithms.
Oriented towards "textbook-style" graphs.

TINKERPOP

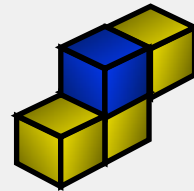
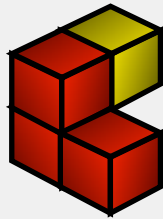


Open source graph product group

Support for various graph vendors



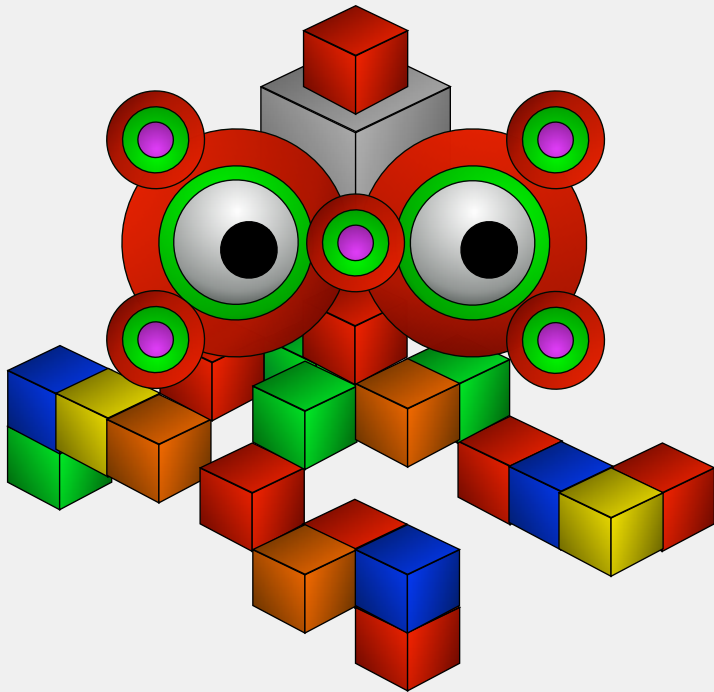
* Encompassing the various graph computing styles



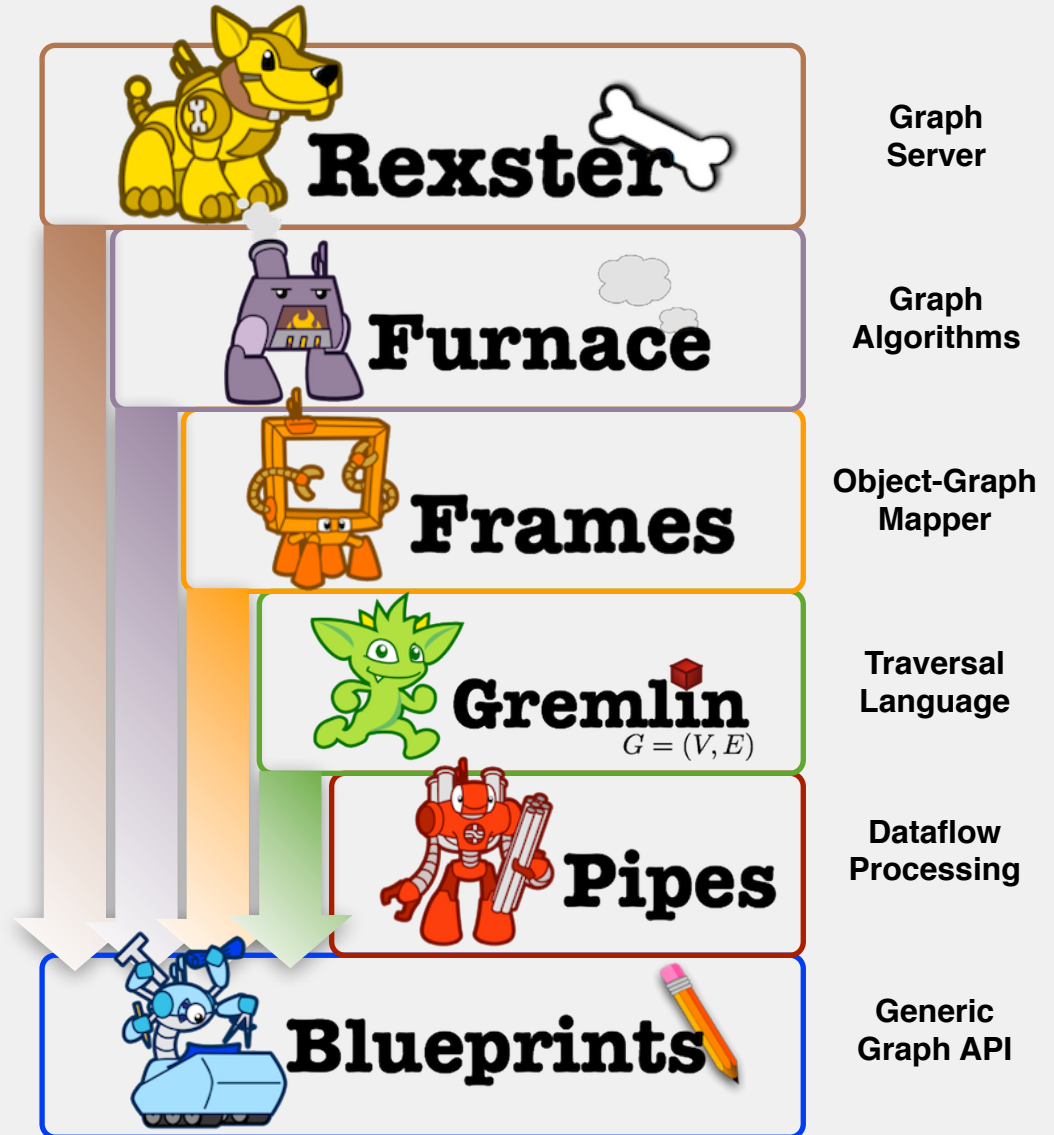
Provides a vendor-agnostic graph framework

Simple, well-defined products

TINKERPOP



<http://tinkerpop.com>



Graph
Server

Graph
Algorithms

Object-Graph
Mapper

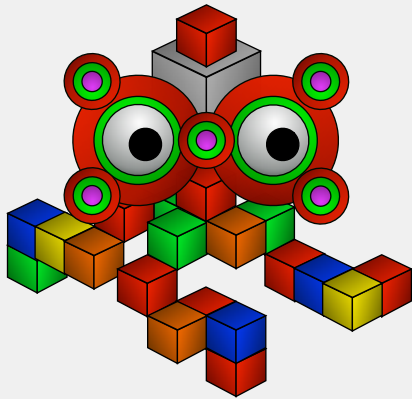
Traversal
Language

Dataflow
Processing

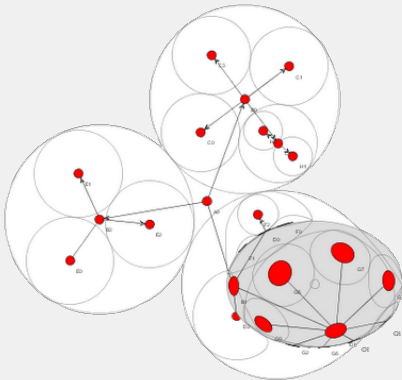
Generic
Graph API

[http://\\${project.name}.tinkerpop.com](http://${project.name}.tinkerpop.com)

TINKERPOP INTEGRATION

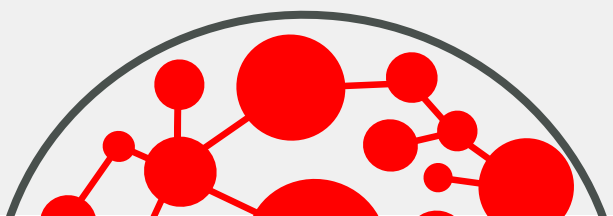


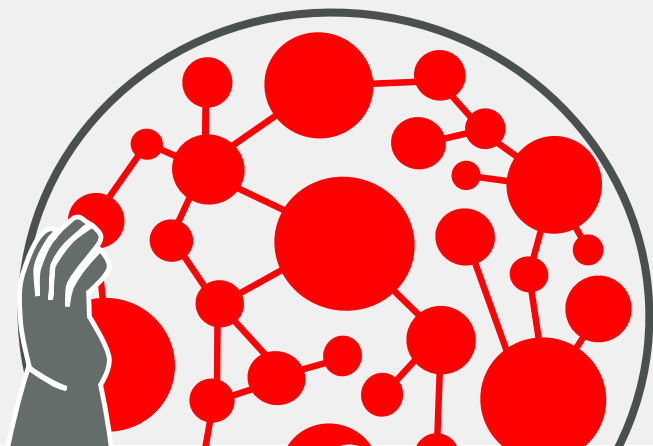
<http://tinkerpop.com>

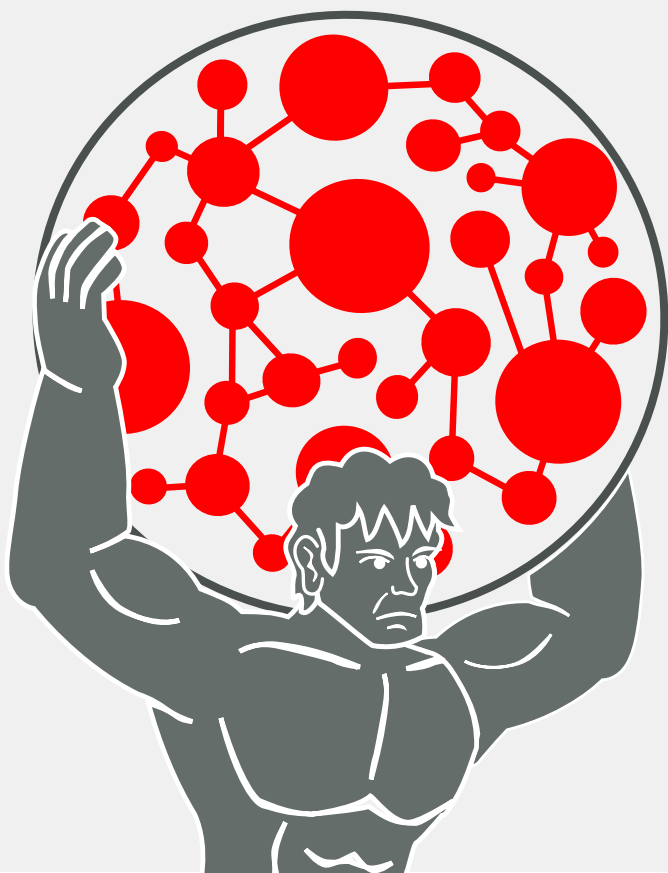


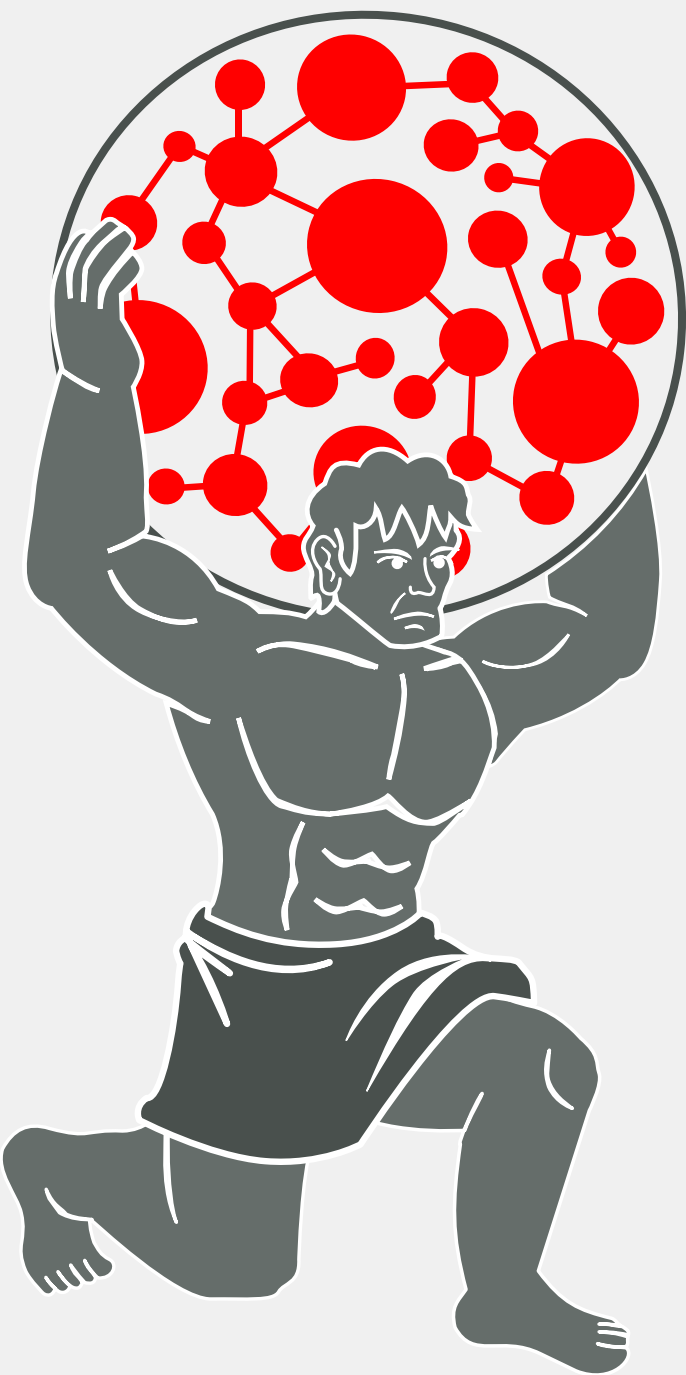
Stardog

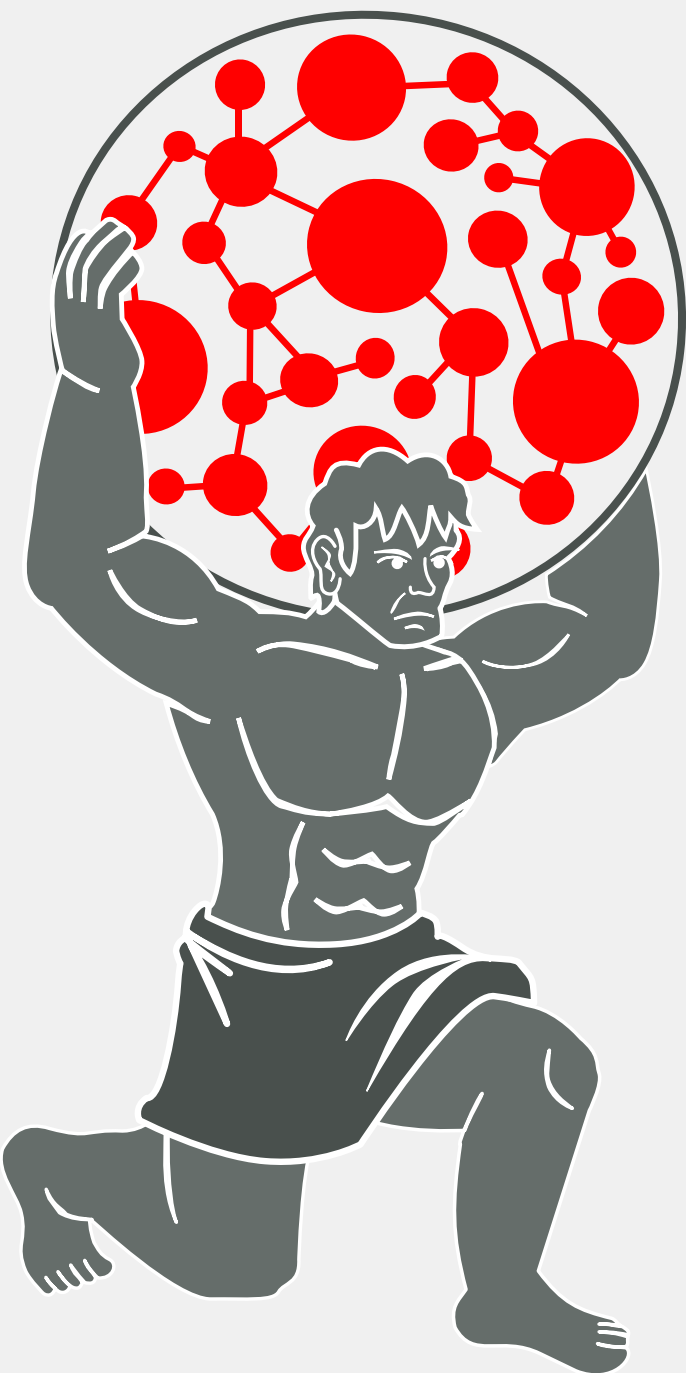
AND NOW
THERE IS ANOTHER...











TITAN

PART 2: INTRODUCTION TO TITAN



MATTHIAS BROECHELER



WHY CREATE TITAN?



A number of Aurelius' clients...

- ...need to represent and process graphs at the 100+ billion edge scale w/ thousands of concurrent transactions.
- ...need both local graph traversals (OLTP) and batch graph processing (OLAP).
- ...desire a free, open source distributed graph database.

TITAN'S KEY FEATURES



Titan provides...

- ..."infinite size" graphs and "unlimited" users by means of a distributed storage engine.
- ...real-time local traversals (OLTP) and support for global batch processing via Hadoop (OLAP).
- ...distribution via the liberal, free, open source Apache2 license.

matthias\$


```
matthias$ wget http://thinkaurelius/titan.zip
% Total      % Received % Xferd  Average Speed   Time    Time
100 99999      0 99999    0      11078      0 --:--:-- 0:01:01
matthias$
```

```
matthias$ wget http://thinkaurelius/titan.zip
% Total      % Received % Xferd  Average Speed   Time    Time
100 99999      0 99999    0        0  11078      0  --:--:--  0:01:01
matthias$ unzip titan.zip
Archive:  titan.zip
  creating: titan/
  ...
matthias$
```

```
matthias$ wget http://thinkaurelius/titan.zip
  % Total    % Received % Xferd  Average Speed   Time    Time
100 99999      0 99999      0      0  11078      0 --:--:--  0:01:01
matthias$ unzip titan.zip
Archive:  titan.zip
  creating: titan/
  ...
matthias$ cd titan
titan$
```

```
matthias$ wget http://thinkaurelius/titan.zip
% Total      % Received % Xferd  Average Speed   Time    Time
100 99999      0 99999    0        0  11078        0 --:--:--  0:01:01
```

```
matthias$ unzip titan.zip
Archive:  titan.zip
  creating: titan/
  ...
```

```
matthias$ cd titan
titan$ bin/gremlin.sh
```

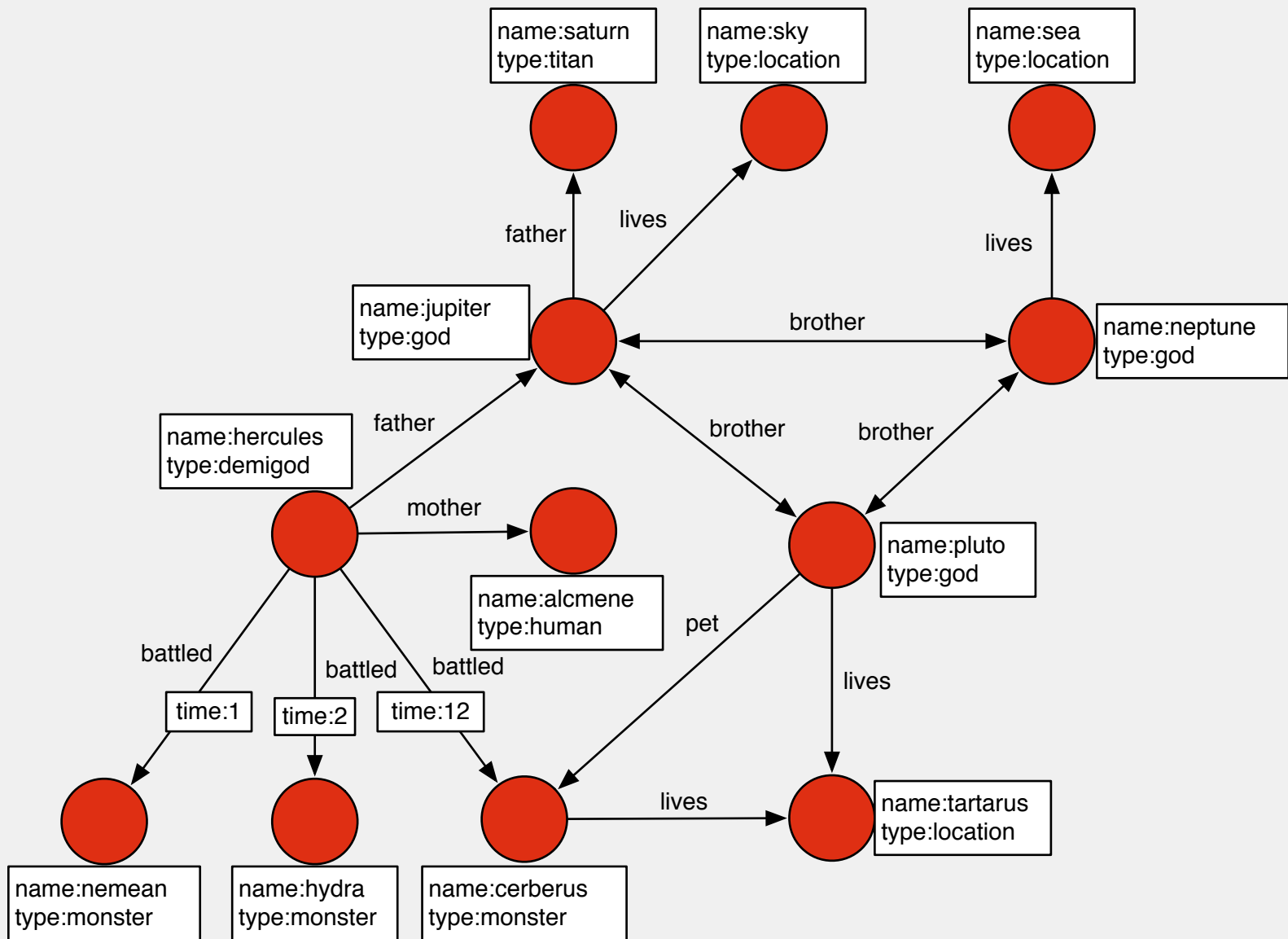
```
      \,,,/
      (o o)
-----o00o-(_)o00o-----
gremlin>
```

```
gremlin> g = TitanFactory.open('/tmp/local-titan')  
==>titangraph[local:/tmp/local-titan]
```

```
gremlin> g = TitanFactory.open('/tmp/local-titan')  
==>titangraph[local:/tmp/local-titan]
```

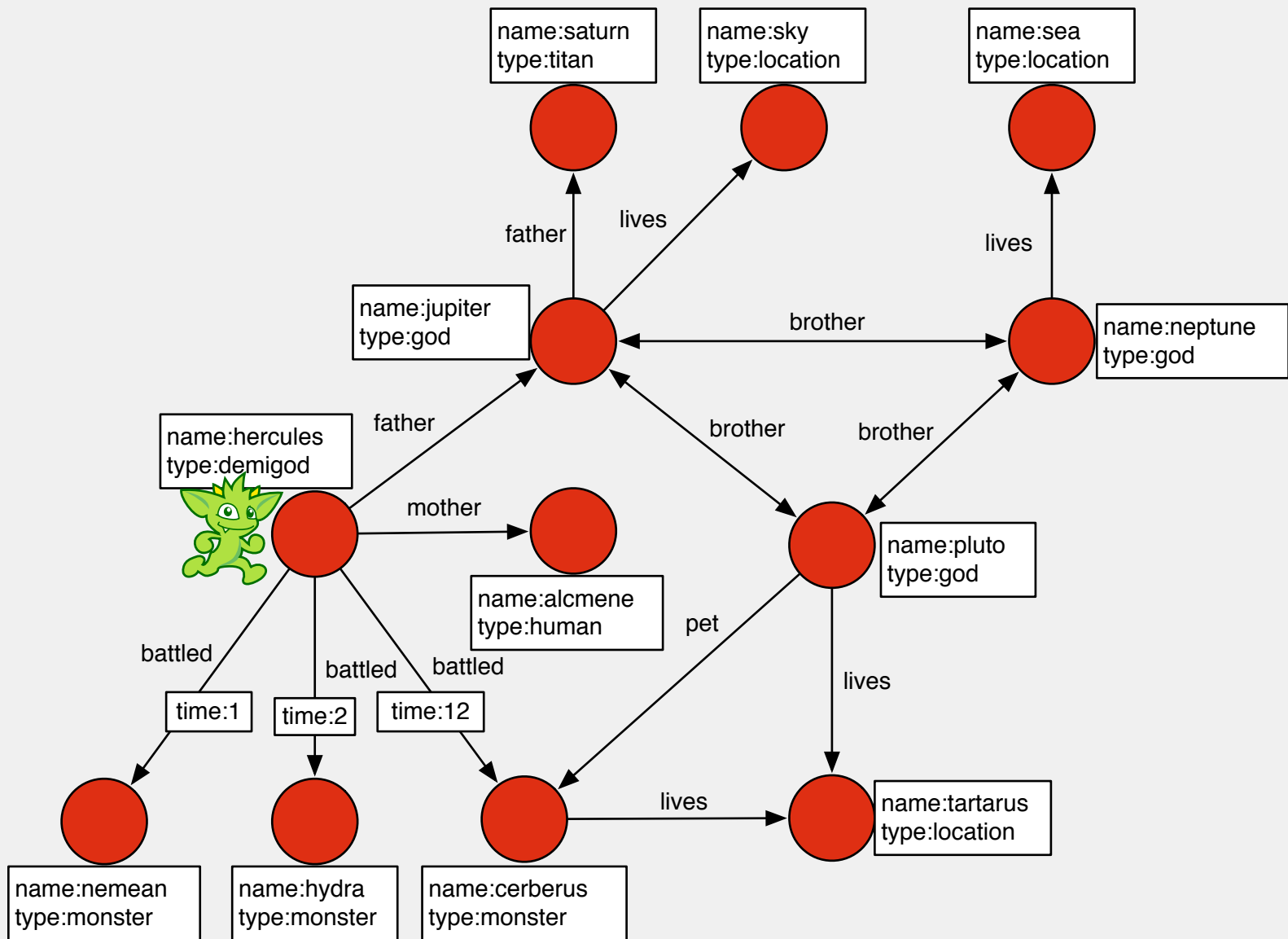
LOCAL MACHINE MODE

```
gremlin> g.createKeyIndex('name',Vertex.class)
==>null
gremlin> g.stopTransaction(SUCCESS)
==>null
```

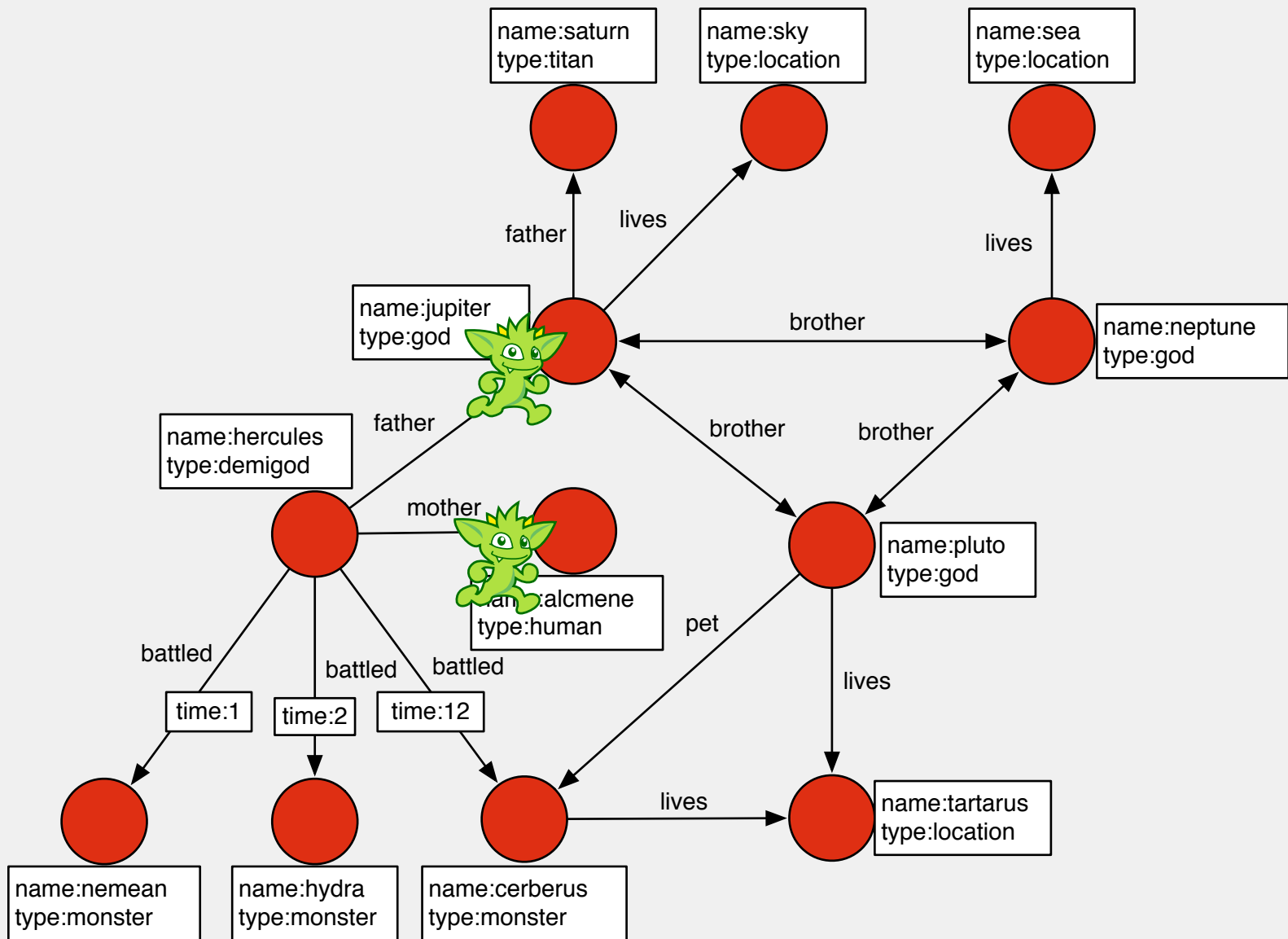


```
gremlin> g.loadGraphML('data/graph-of-the-gods.xml')
==>null
```

* The Graph of the Gods is a toy dataset distributed with Titan



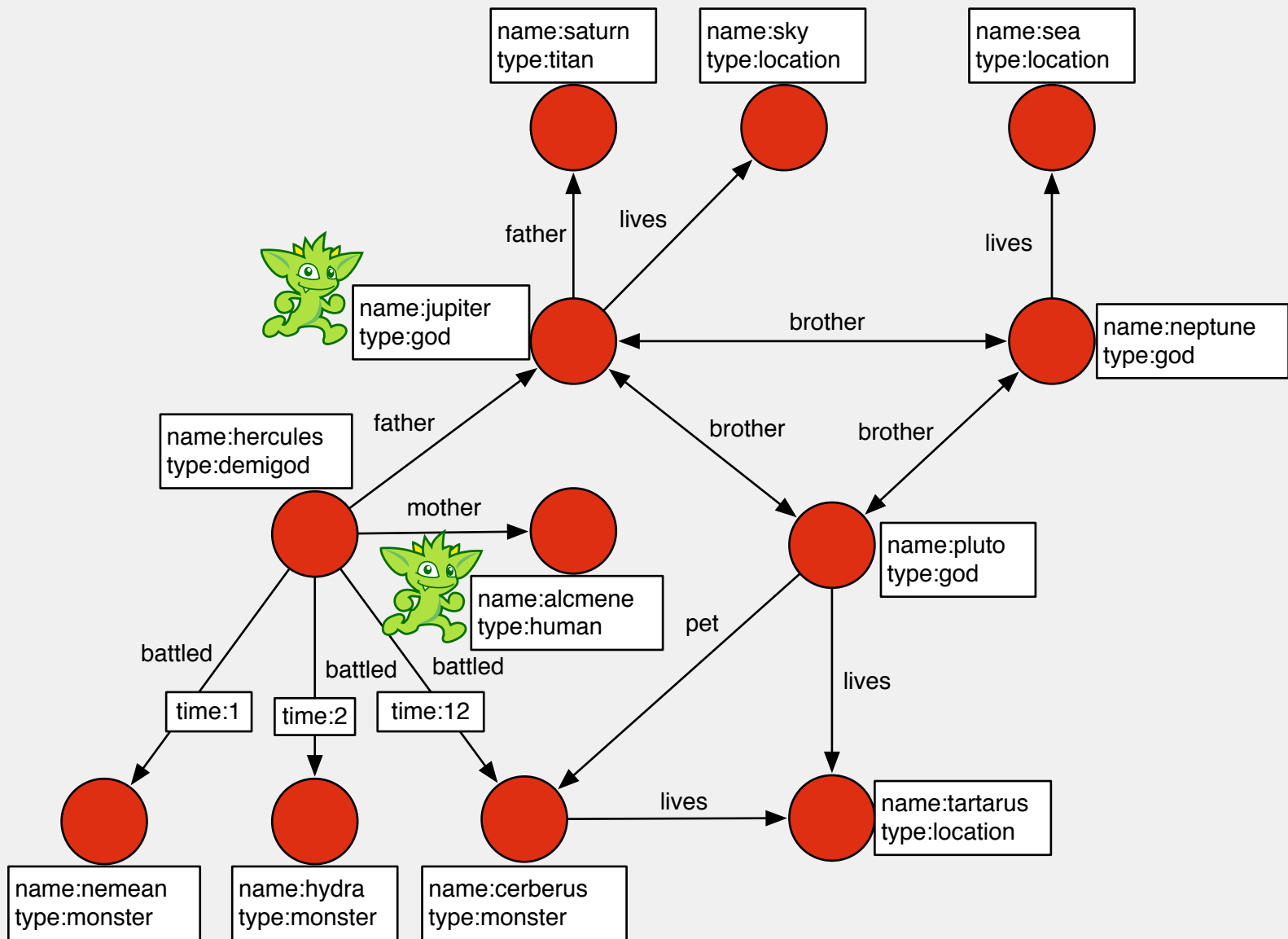
```
gremlin> hercules = g.V('name','hercules').next()
==>v[24]
```



```

gremlin> hercules.out('mother','father')
==>v[44]
==>v[16]

```

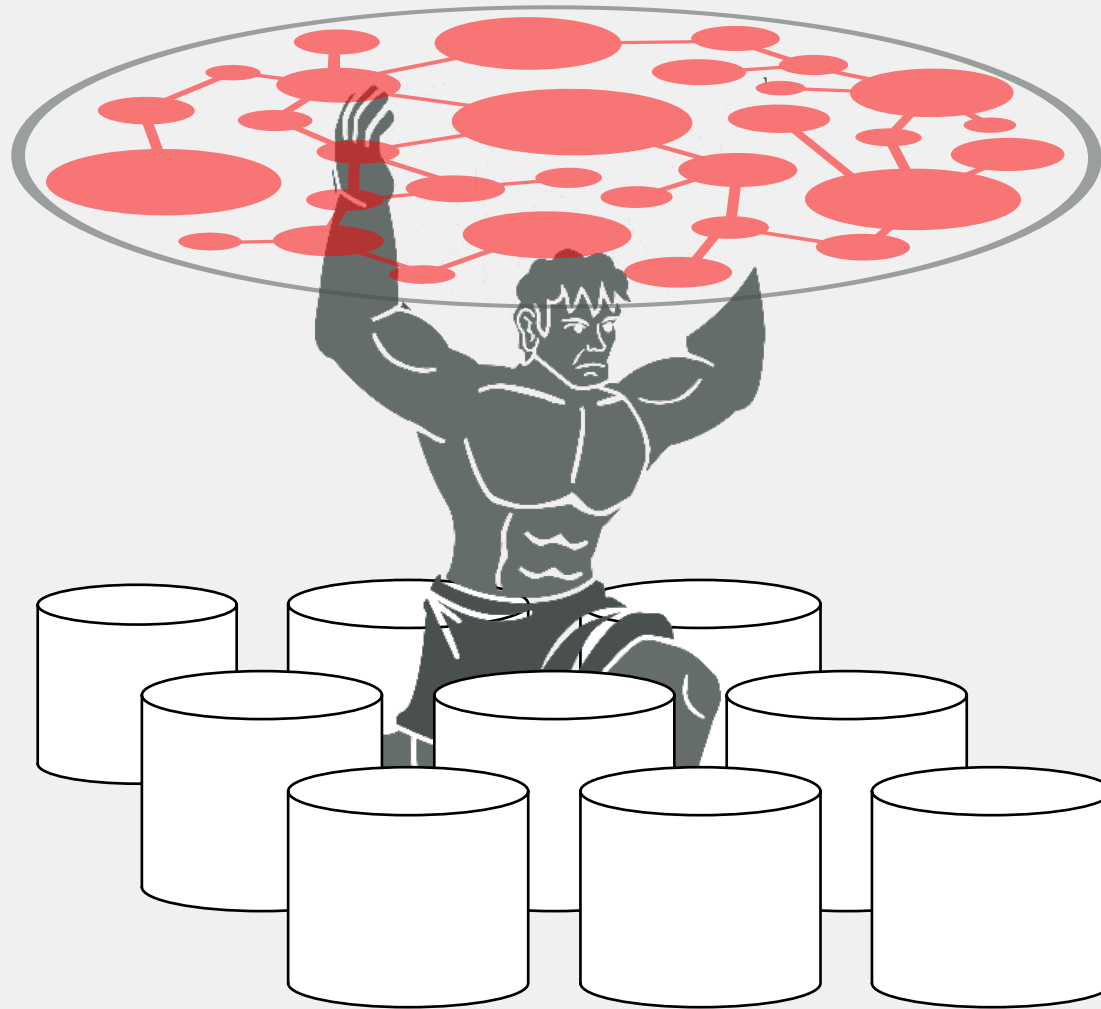


```
gremlin> hercules.out('mother','father').name
==>alcmena
==>jupiter
```

THAT WAS TITAN LOCAL.

NEXT IS TITAN DISTRIBUTED.

BACKEND AGNOSTIC



Cassandra

-OR-

A P A C H E
HBASE

TITAN DISTRIBUTED VIA CASSANDRA



```
titan$ bin/gremlin.sh
```

```
      \,,,/  
      (o o)  
-----o00o-(_)o00o-----  
gremlin> conf = new BaseConfiguration();  
==>org.apache.commons.configuration.BaseConfiguration@763861e6  
gremlin> conf.setProperty("storage.backend", "cassandra");  
gremlin> conf.setProperty("storage.hostname", "77.77.77.77");  
gremlin> g = TitanFactory.open(conf);  
==>titangraph[cassandra:77.77.77.77]  
gremlin>
```

* There are numerous graph configurations: <https://github.com/thinkaurelius/titan/wiki/Graph-Configuration>

INHERITED FEATURES



- Continuously available with no single point of failure.
- No write bottlenecks to the graph as there is no master/slave architecture.
- Built-in replication ensures data is available during machine failure.
- Caching layer ensures that continuously accessed data is available in memory.
- Elastic scalability allows for the introduction and removal of machines.

TITAN DISTRIBUTED VIA HBASE



```
titan$ bin/gremlin.sh
```

```
      \,,,/
      (o o)
-----o00o-(_)o00o-----
gremlin> conf = new BaseConfiguration();
==>org.apache.commons.configuration.BaseConfiguration@763861e6
gremlin> conf.setProperty("storage.backend", "hbase");
gremlin> conf.setProperty("storage.hostname", "77.77.77.77");
gremlin> g = TitanFactory.open(conf);
==>titangraph[hbase:77.77.77.77]
gremlin>
```

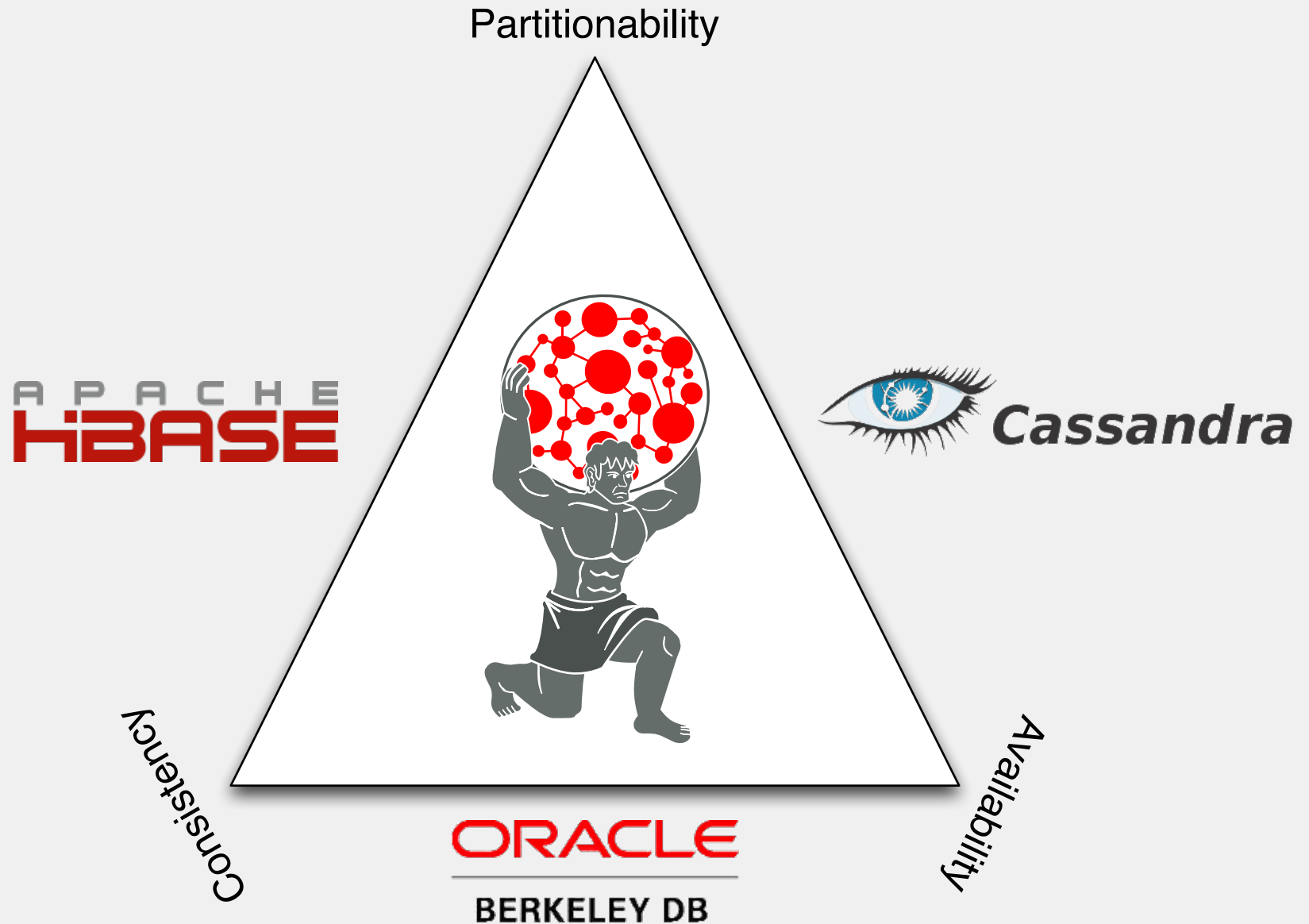
* There are numerous graph configurations: <https://github.com/thinkaurelius/titan/wiki/Graph-Configuration>

INHERITED FEATURES

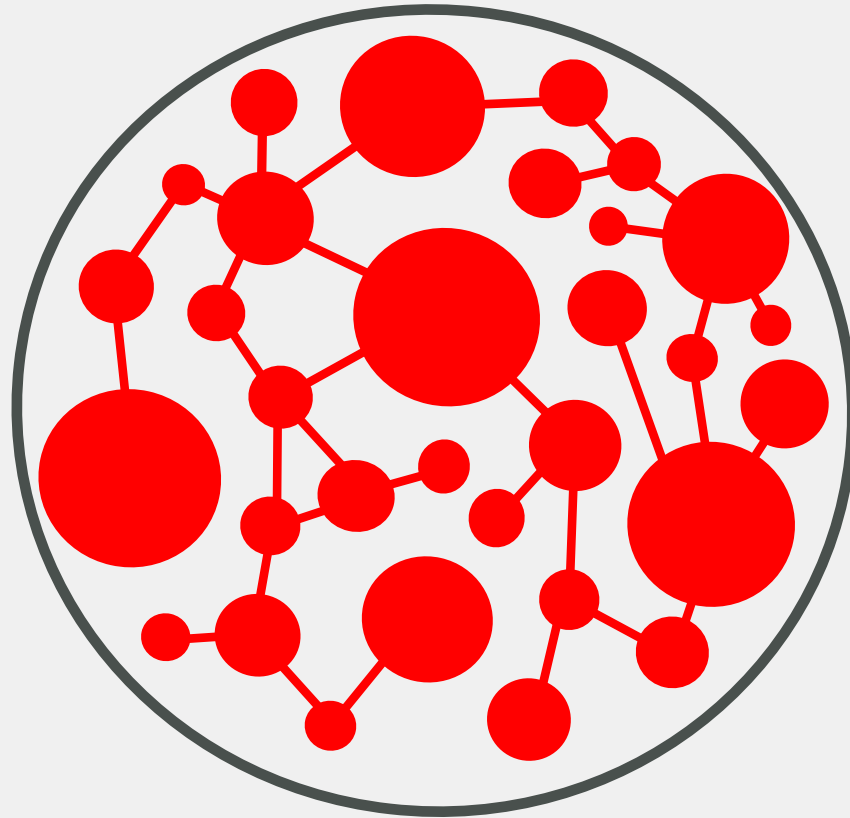


- Strictly consistent reads and writes.
- Linear scalability with the addition of machines.
- Base classes for backing Hadoop MapReduce jobs with HBase tables.
- HDFS-based data replication.
- Generally good integration with the tools in the Hadoop ecosystem.

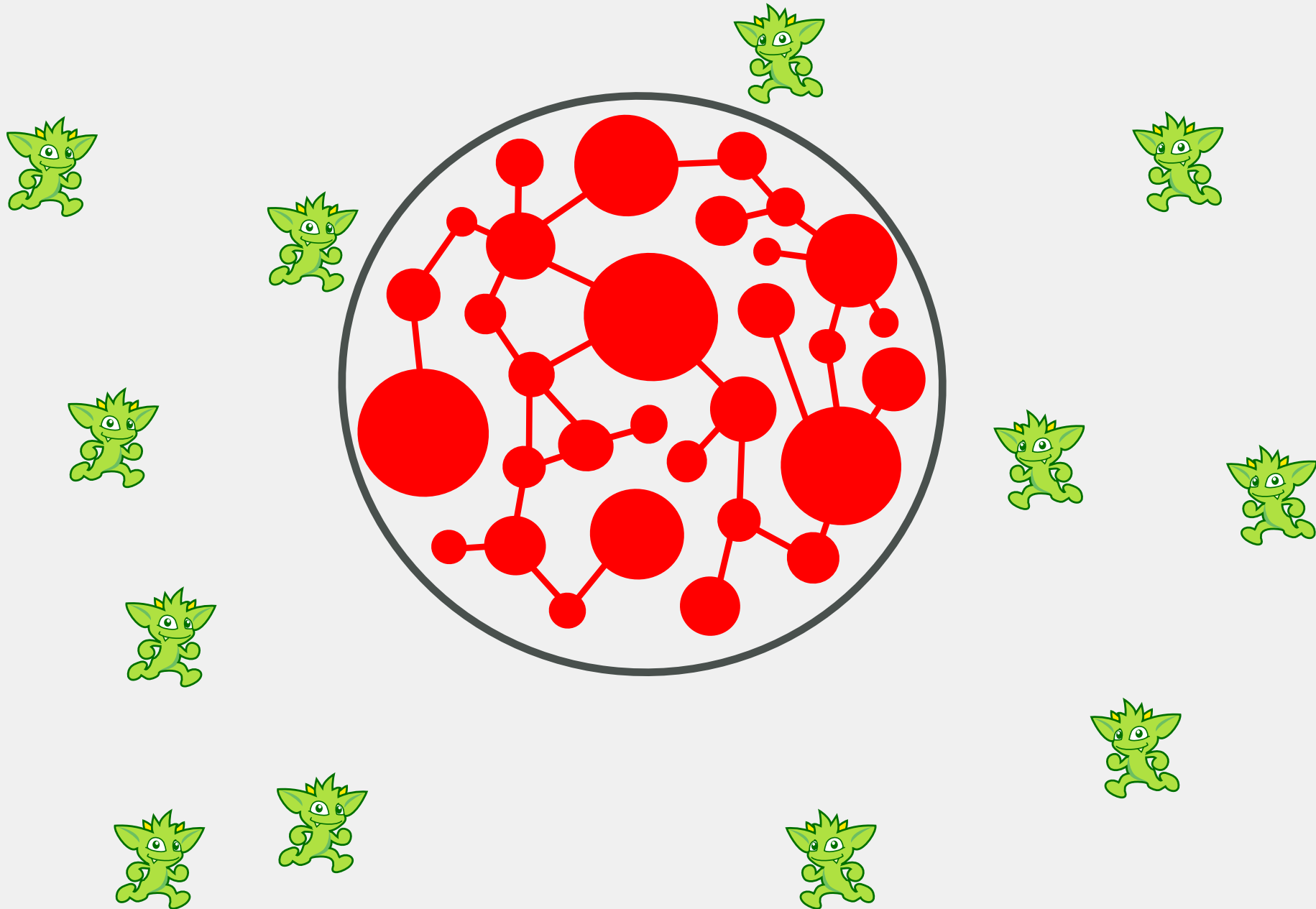
TITAN AND THE CAP THEOREM



Titan is all about ...

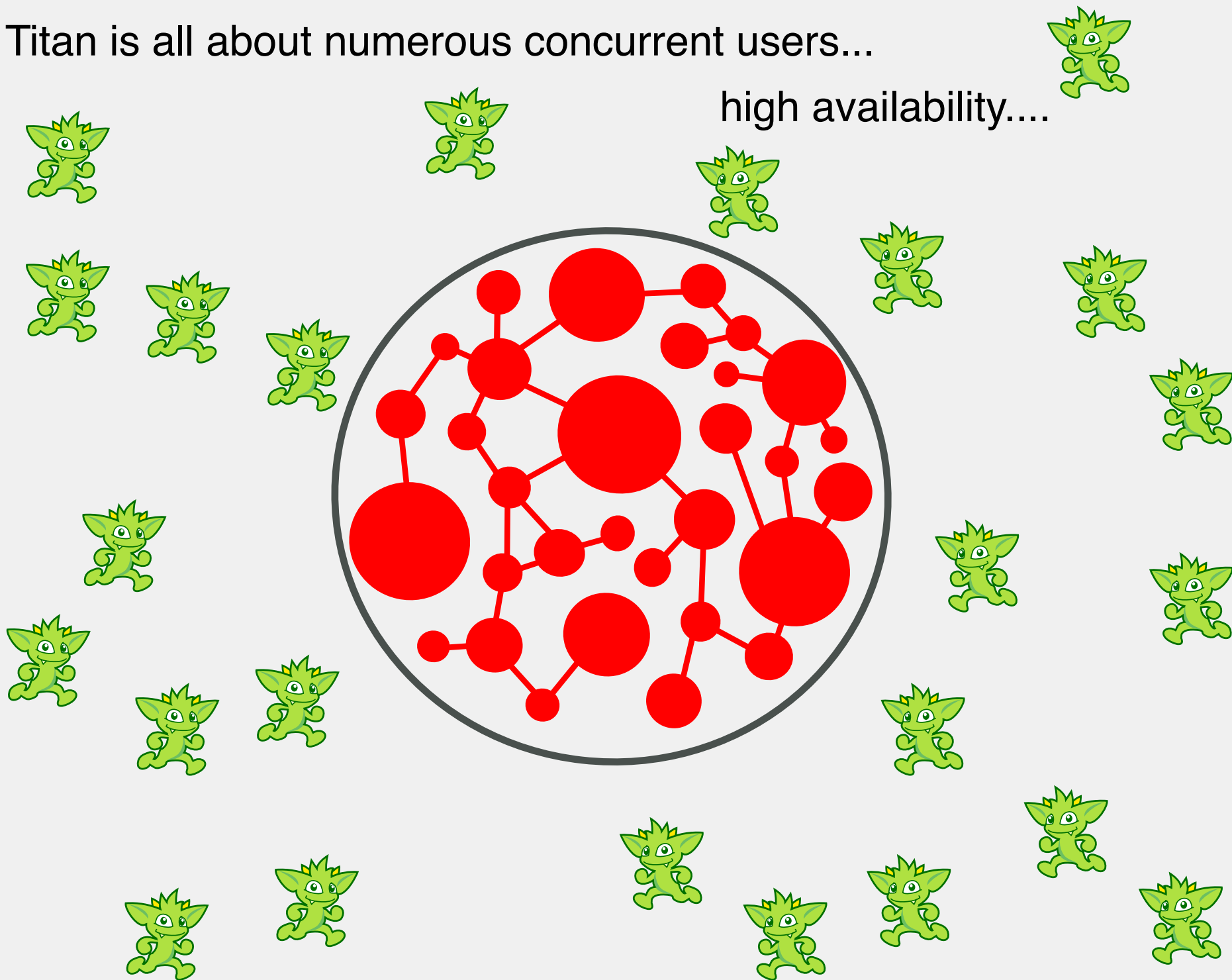


Titan is all about numerous concurrent users...



Titan is all about numerous concurrent users...

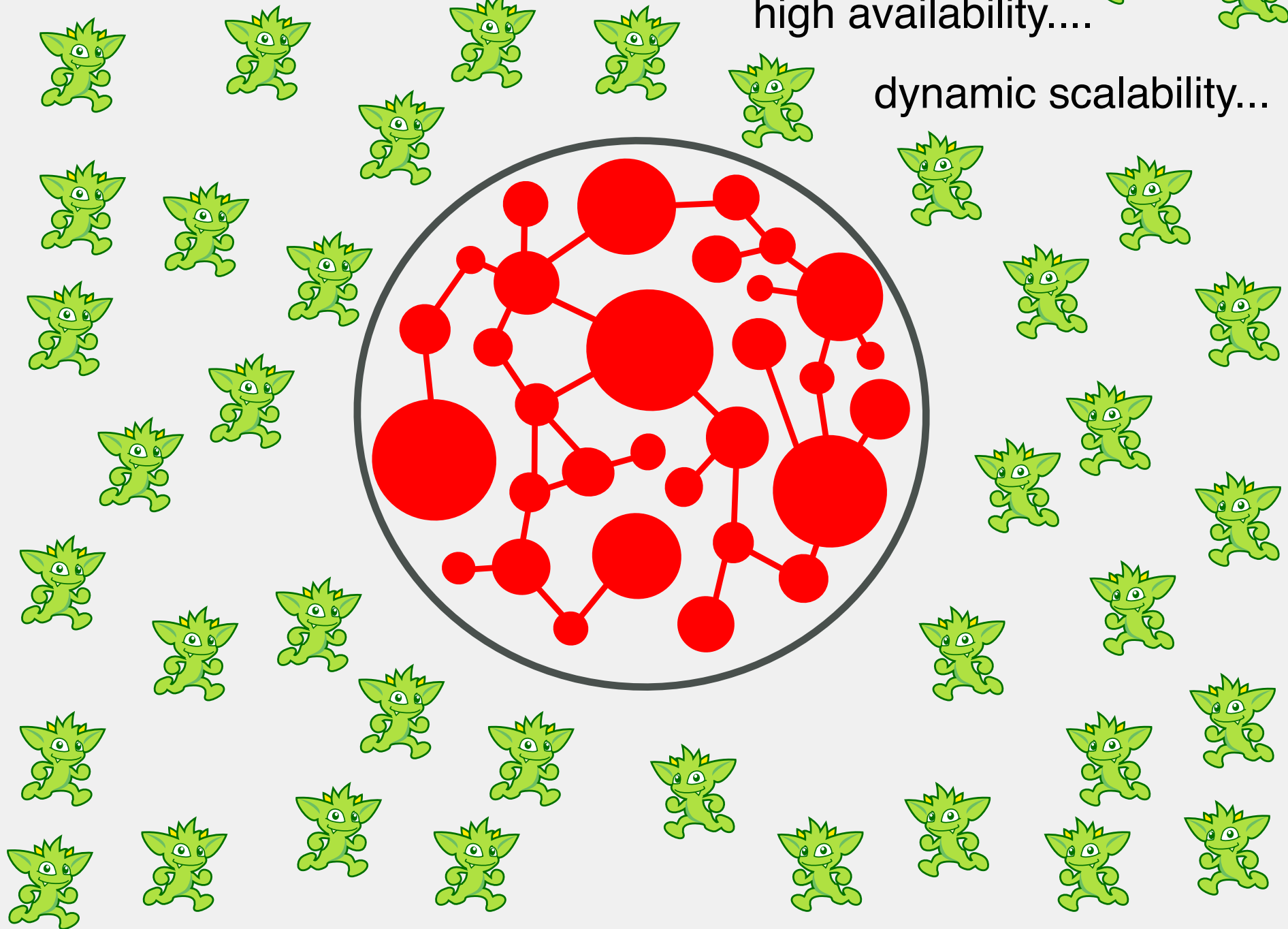
high availability....



Titan is all about numerous concurrent users...

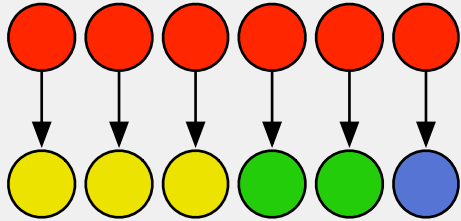
high availability....

dynamic scalability...

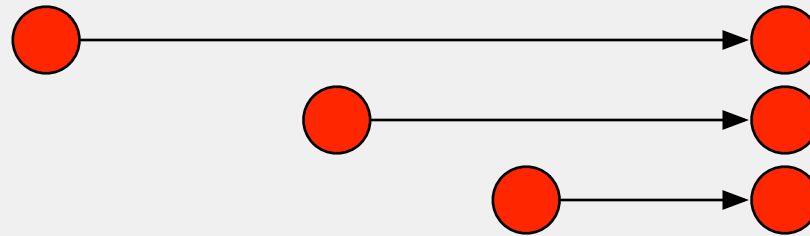


THE HOW OF TITAN

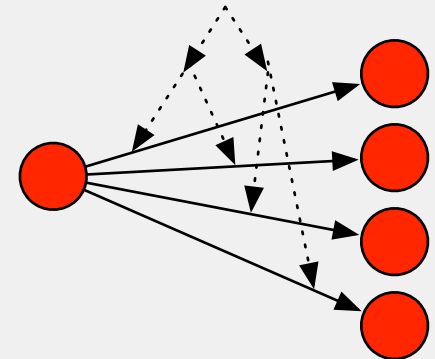
DATA MANAGEMENT



EDGE COMPRESSION

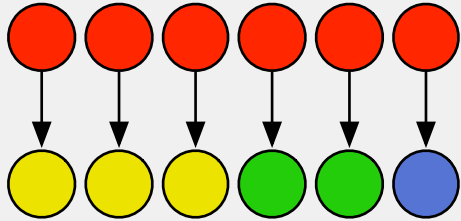


VERTEX-CENTRIC INDICES



THE HOW OF TITAN

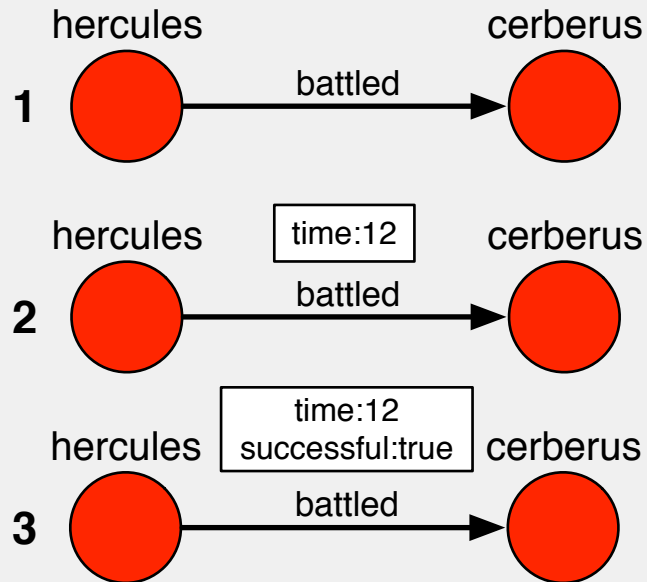
DATA MANAGEMENT



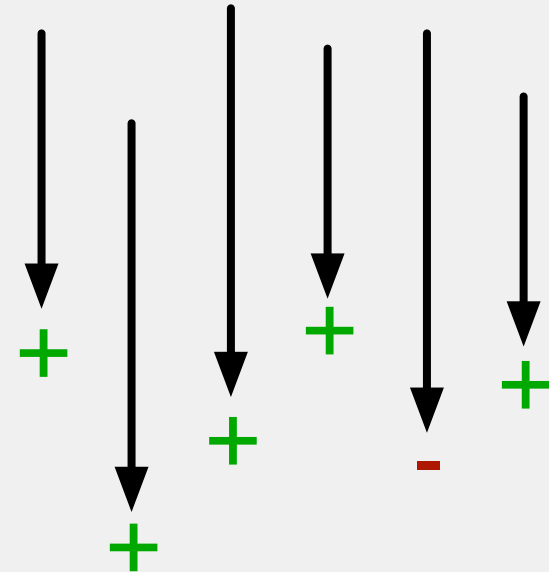
DATA MANAGEMENT

MAIN DESIGN PRINCIPLES

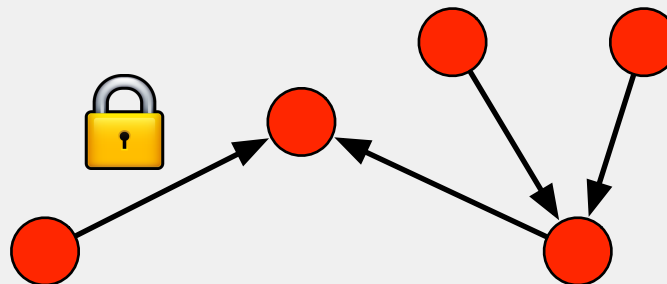
Immutable, Atomic Edges



Optimistic Concurrency Control



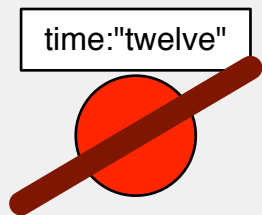
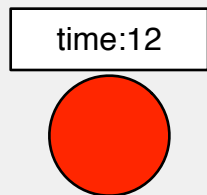
Fined-Grained Locking Control



DATA MANAGEMENT TYPE DEFINITION

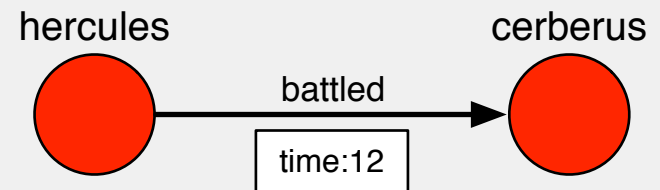
Datatype Constraints

```
TitanKey timeKey =  
    g.makeType().name("time")  
        .dataType(Integer.class)
```



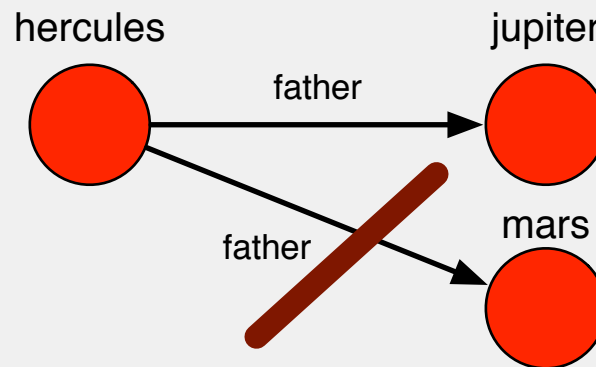
Edge Label Signatures

```
TitanLabel battled =  
    g.makeType().name("battled")  
        .signature(timeKey)
```



Functional Declarations

```
TitanLabel father =  
    g.makeType().name("father")  
        .functional()
```



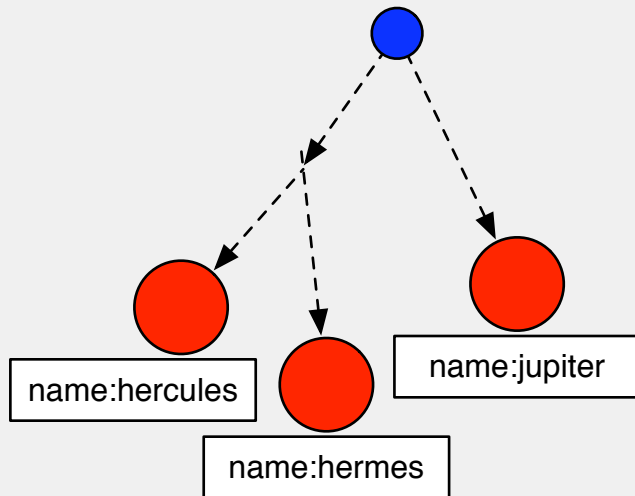
Data management configurations allow Titan to optimize how information is stored/retrieved from disk.

DATA MANAGEMENT

TYPE DEFINITION

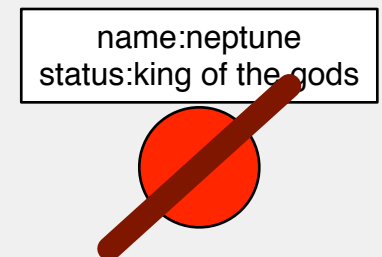
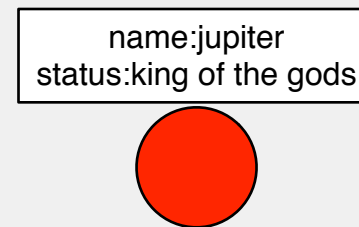
Endogenous Indices

```
g.createKeyIndex("name", Vertex.class)
```



Unique Property Key/Value Pairs

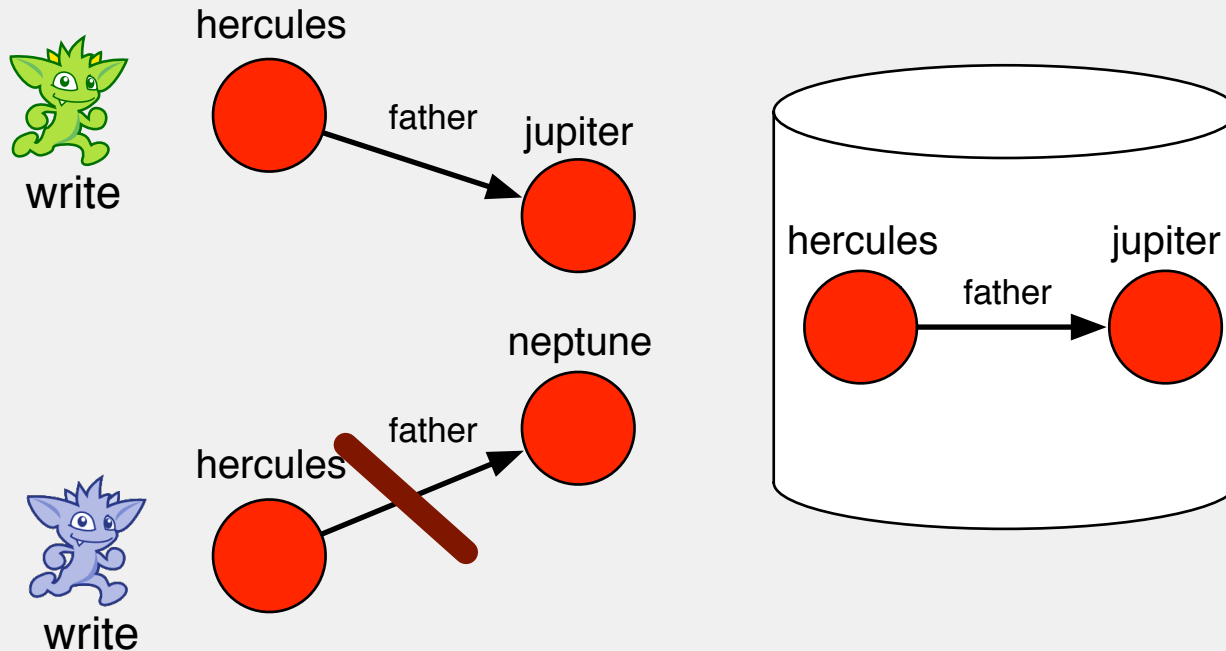
```
TitanKey status =  
    g.makeType().name("status")  
        .unique()
```



Data management configurations allow Titan to optimize how information is stored/retrieved from disk.

DATA MANAGEMENT LOCKING SYSTEM

Ensures consistency over non-consistent storage backends.

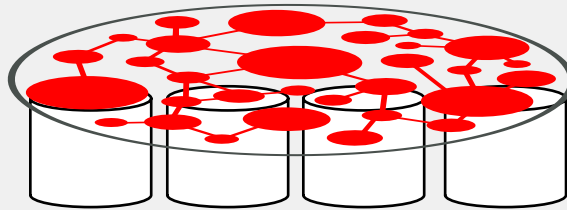


1. Acquire lock at the end of the transaction.
 - locking mechanism depends on storage layer consistency guarantees.
2. Verify original read.
3. Fail transaction if any precondition is violated.

DATA MANAGEMENT

ID MANAGEMENT

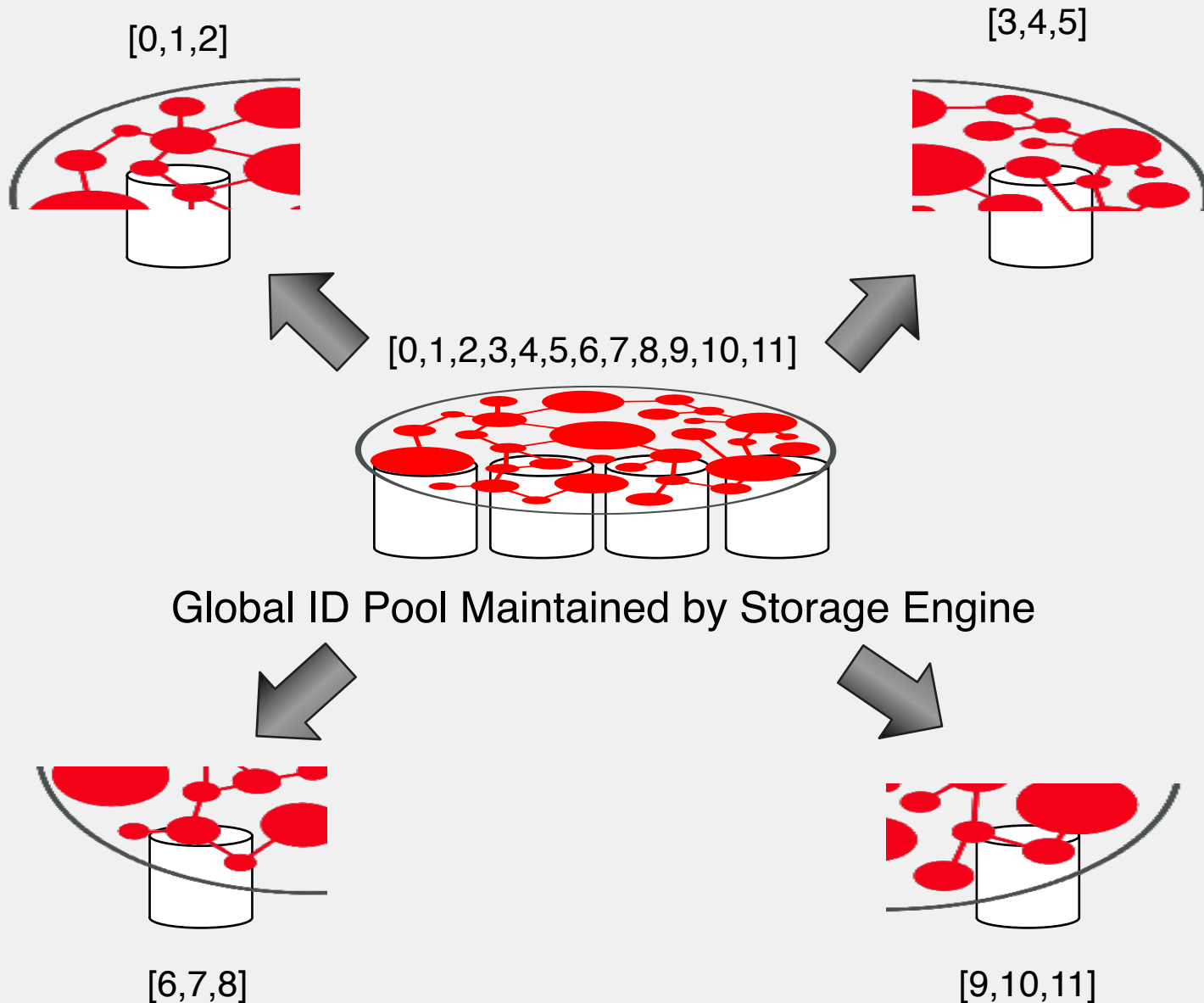
[0,1,2,3,4,5,6,7,8,9,10,11]



Global ID Pool Maintained by Storage Engine

DATA MANAGEMENT

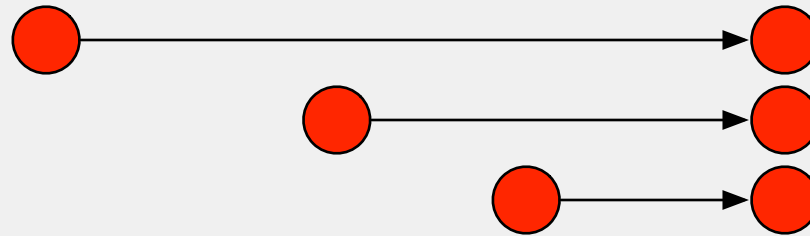
ID MANAGEMENT



Pool Subsets Assigned to Individual Instances

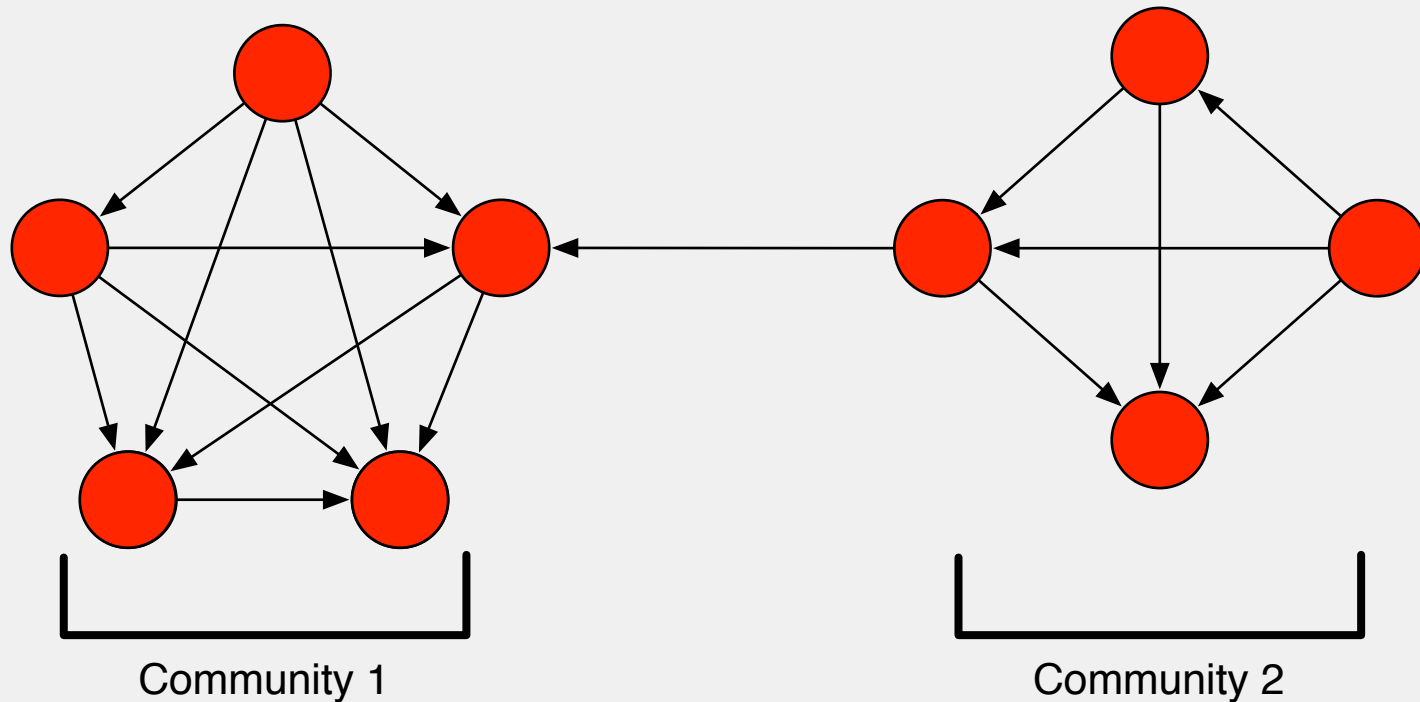
THE HOW OF TITAN

EDGE COMPRESSION



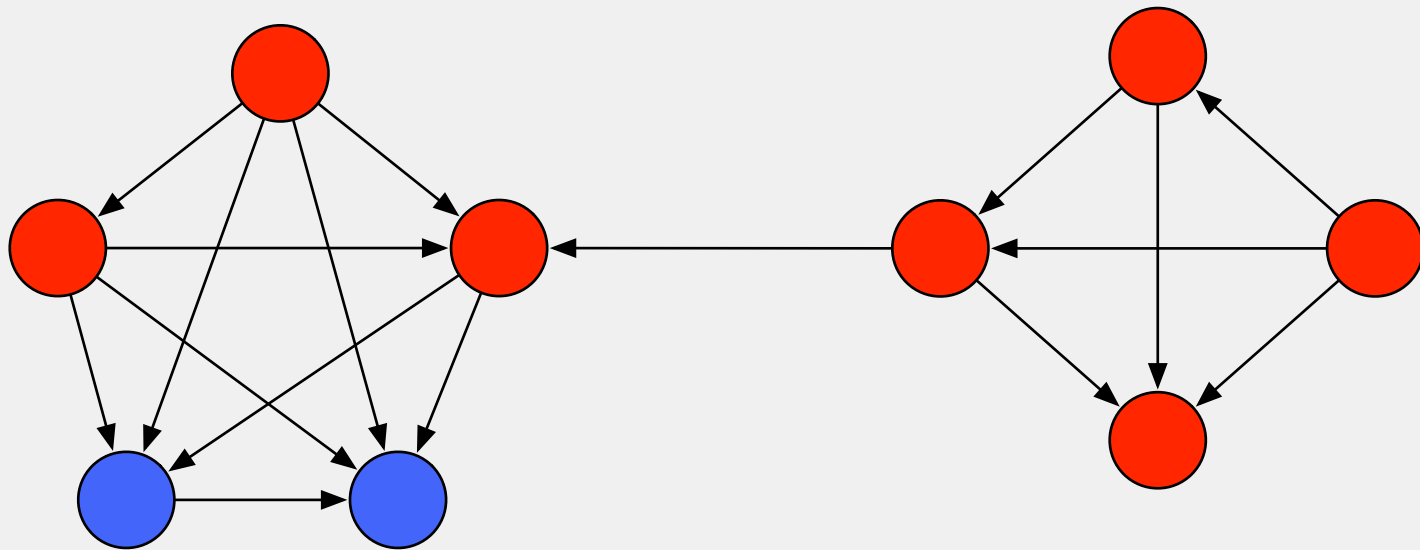
EDGE COMPRESSION

Natural graphs have a small world, community/cluster property.

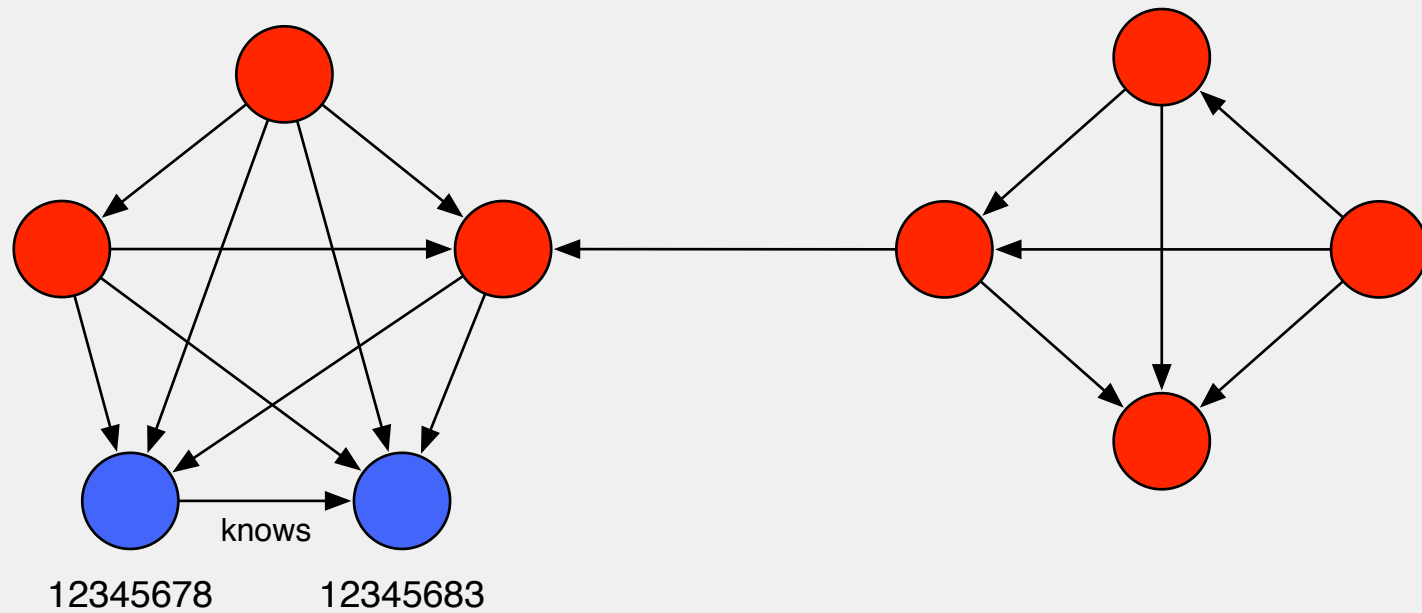


High intra-connectivity within a community and low inter-connectivity between communities.

EDGE COMPRESSION



EDGE COMPRESSION



EDGE COMPRESSION



EDGE COMPRESSION



12345678	9	12345683	24 bytes
----------	---	----------	----------

EDGE COMPRESSION



12345678	9	12345683	24 bytes
----------	---	----------	----------

12345678	9	+5
----------	---	----

EDGE COMPRESSION



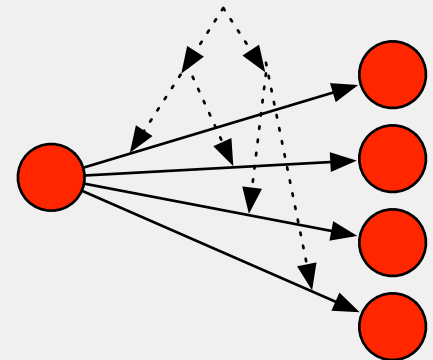
12345678	9	12345683	24 bytes
----------	---	----------	----------

12345678	9	+5
----------	---	----

12345678	9	+5	7 bytes
----------	---	----	---------

THE HOW OF TITAN

VERTEX-CENTRIC INDICES



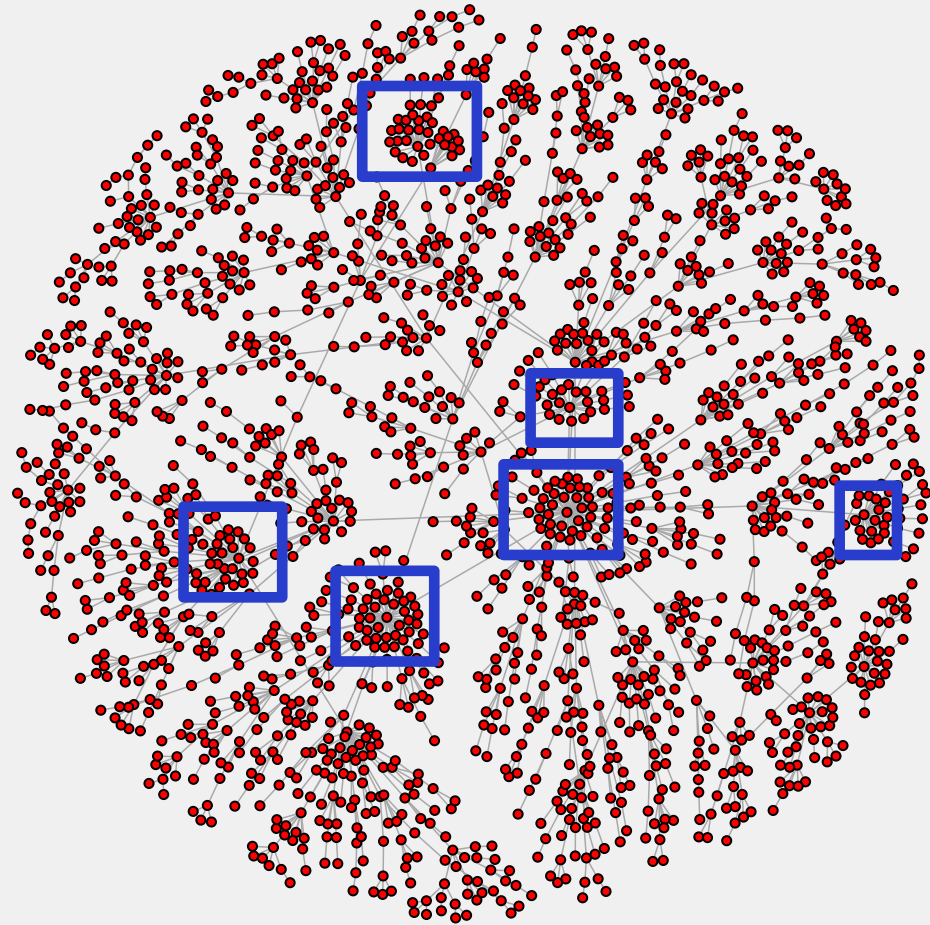
VERTEX-CENTRIC INDICES

THE SUPER NODE PROBLEM

- Natural, real-world graphs contain vertices of high degree.

- Even if rare, their degree ensures that they exist on many paths.

- Traversing a high degree vertex means touching numerous incident edges and potentially touching most of the graph in only a few steps.



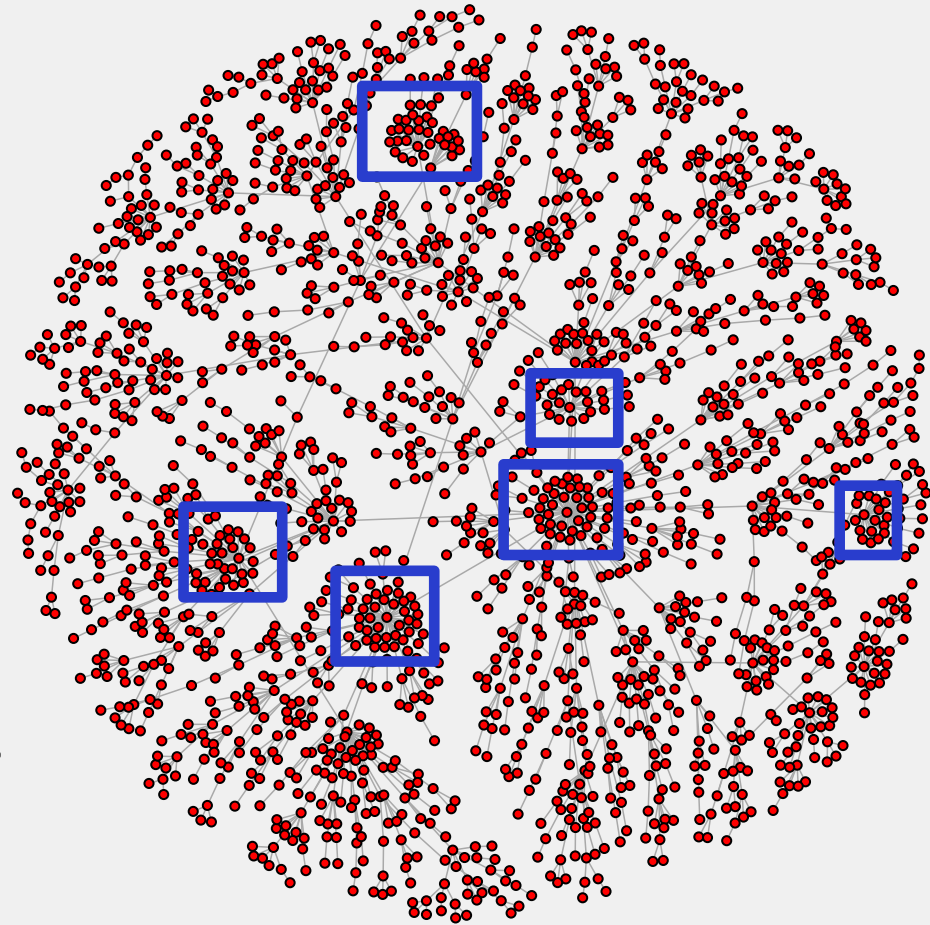
VERTEX-CENTRIC INDICES

A SUPER NODE SOLUTION

- A "super node" only exists from the vantage point of classic "textbook style" graphs.

- In the world of property graphs, intelligent disk-level filtering can interpret a "super node" as a more manageable low-degree vertex.

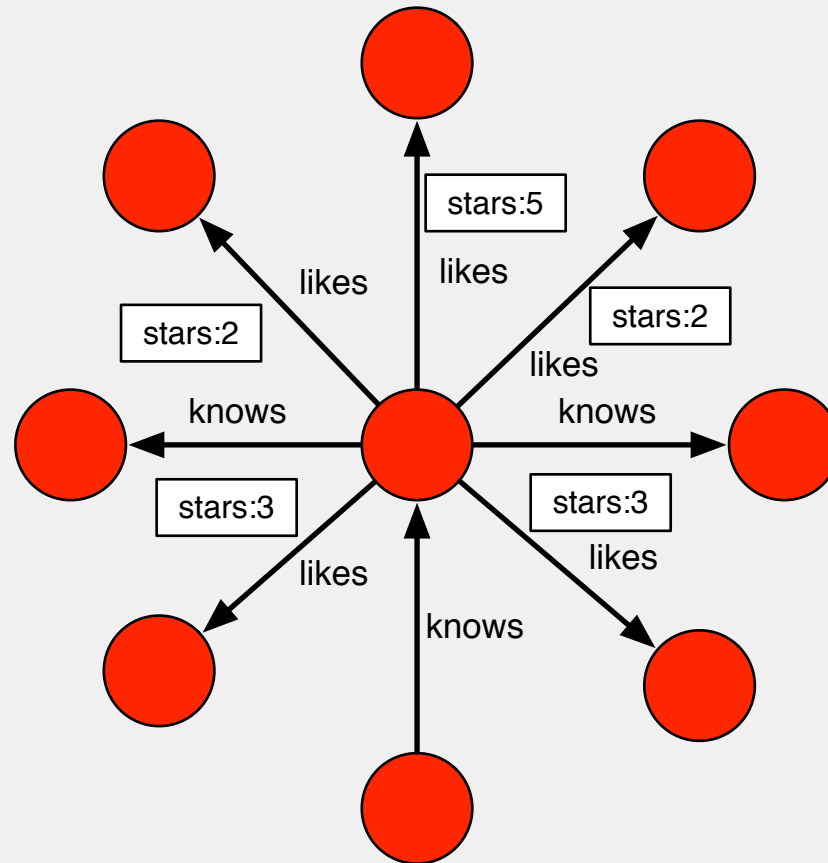
- Vertex-centric querying utilizes B-Trees and sort orders for speedy lookup of incident edges with particular qualities.



VERTEX-CENTRIC INDICES

PUSHDOWN PREDICATES

```
vertex.query()
```

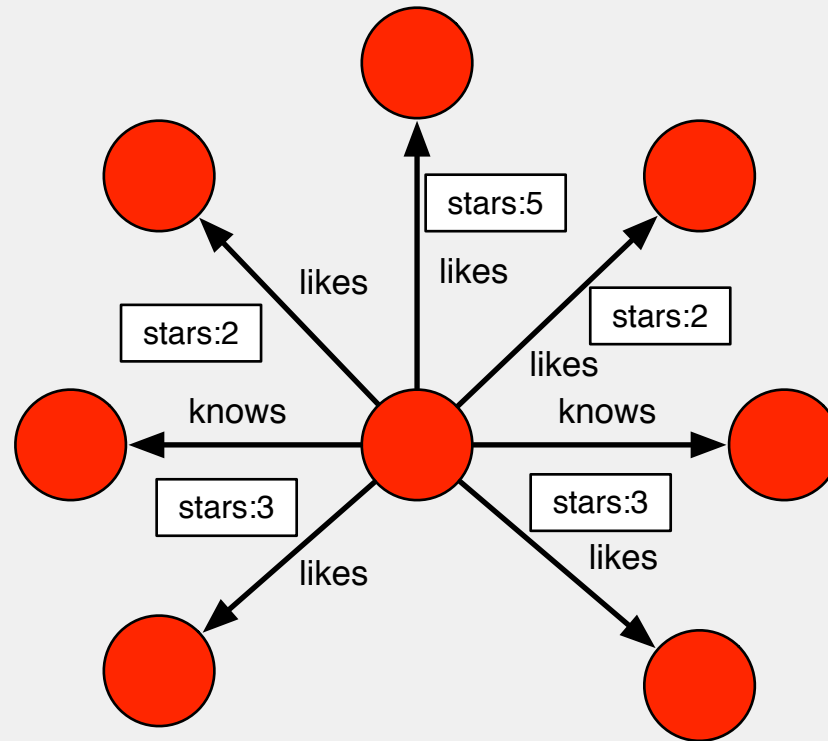


8 edges

VERTEX-CENTRIC INDICES

PUSHDOWN PREDICATES

```
vertex.query().direction(OUT)
```

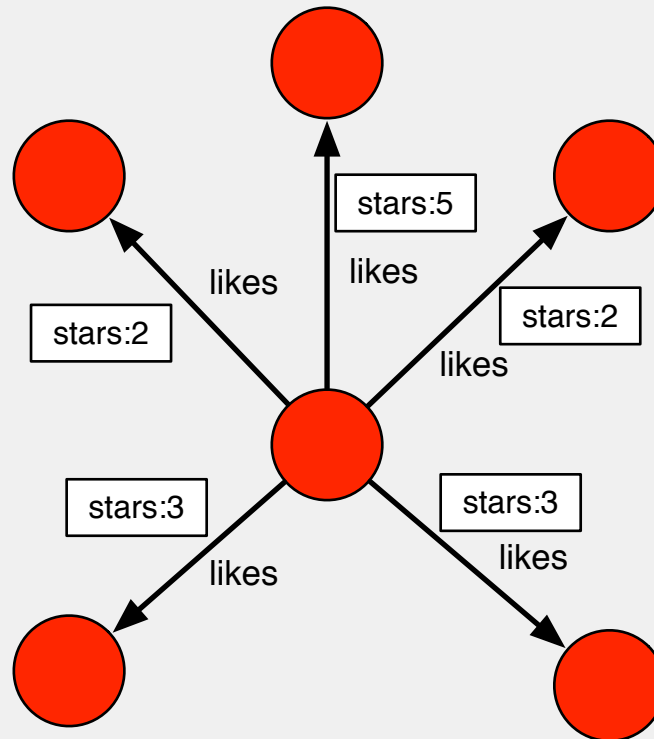


7 edges

VERTEX-CENTRIC INDICES

PUSHDOWN PREDICATES

```
vertex.query().direction(OUT)  
  .labels("likes")
```

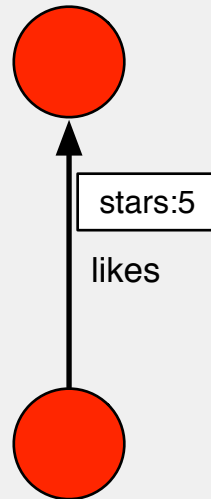


5 edges

VERTEX-CENTRIC INDICES

PUSHDOWN PREDICATES

```
vertex.query().direction(OUT)  
  .labels("likes").has("stars",5)
```



1 edge

VERTEX-CENTRIC INDICES

PUSHDOWN PREDICATES

PREDICATES

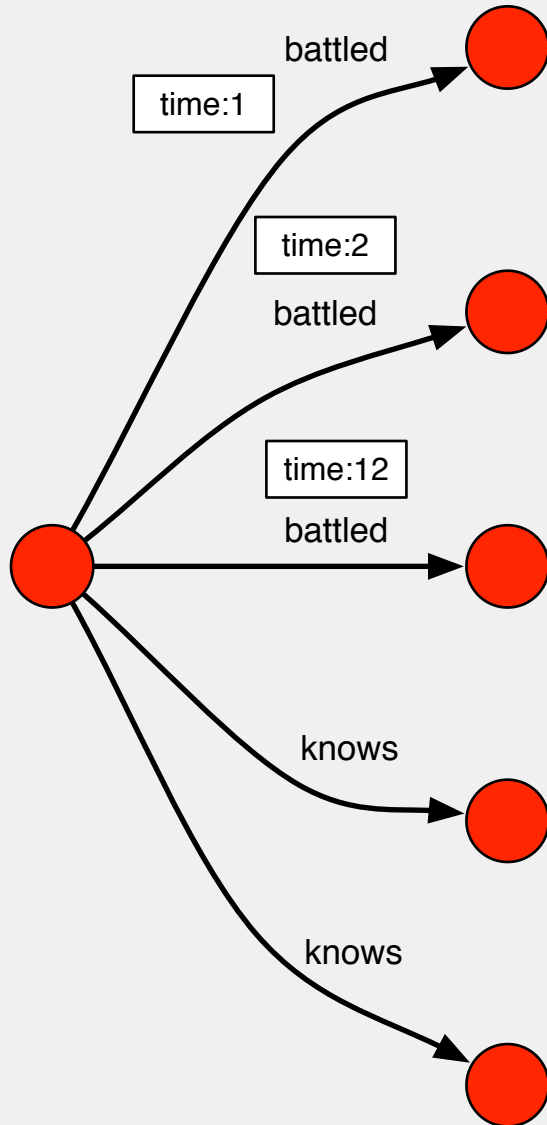
```
Query Query.direction(Direction)
Query Query.labels(String... labels)
Query Query.has(String, Object, Compare)
Query Query.has(String, Object)
Query Query.range(String, Object, Object)
```

GETTERS

```
Iterable<Vertex> Query.vertices()
Iterable<Edge> Query.edges()
```

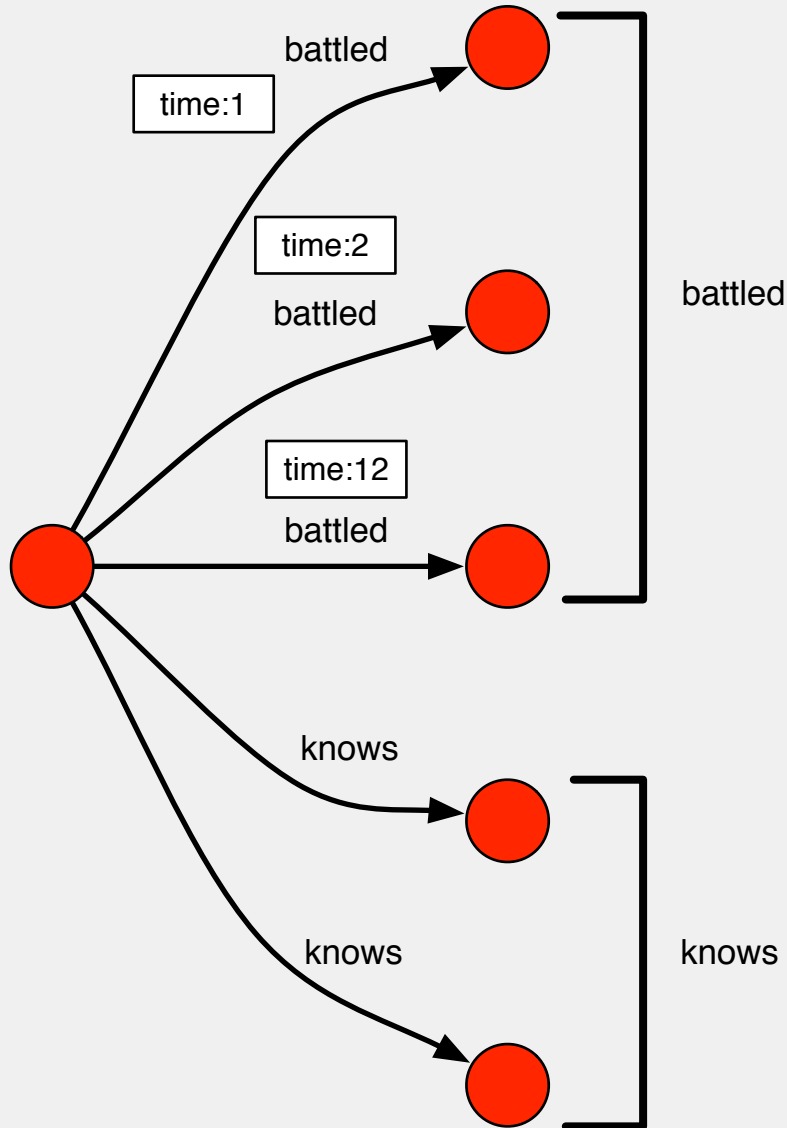
VERTEX-CENTRIC INDICES

DISK-LEVEL SORTING/INDEXING



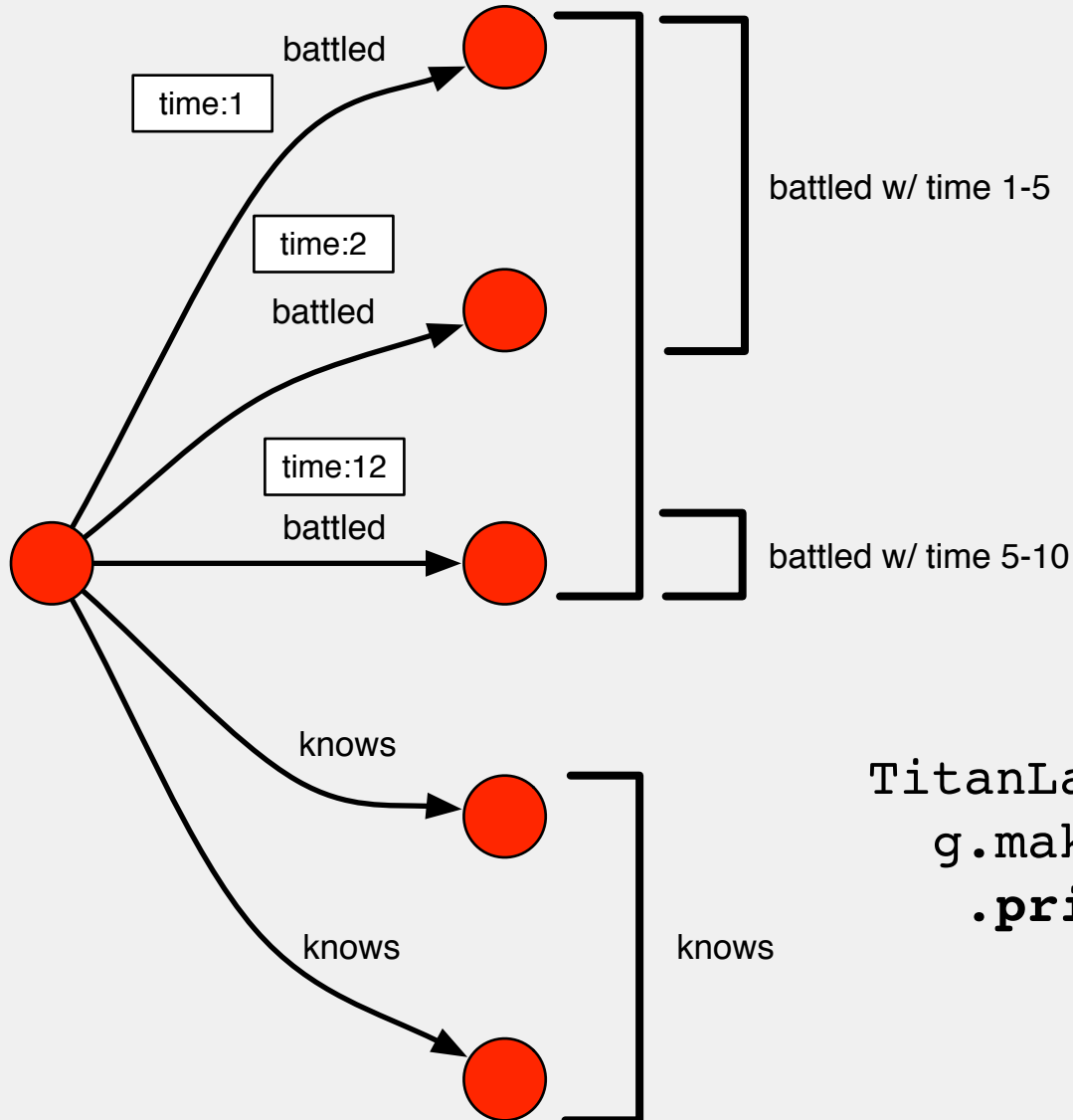
VERTEX-CENTRIC INDICES

DISK-LEVEL SORTING/INDEXING



VERTEX-CENTRIC INDICES

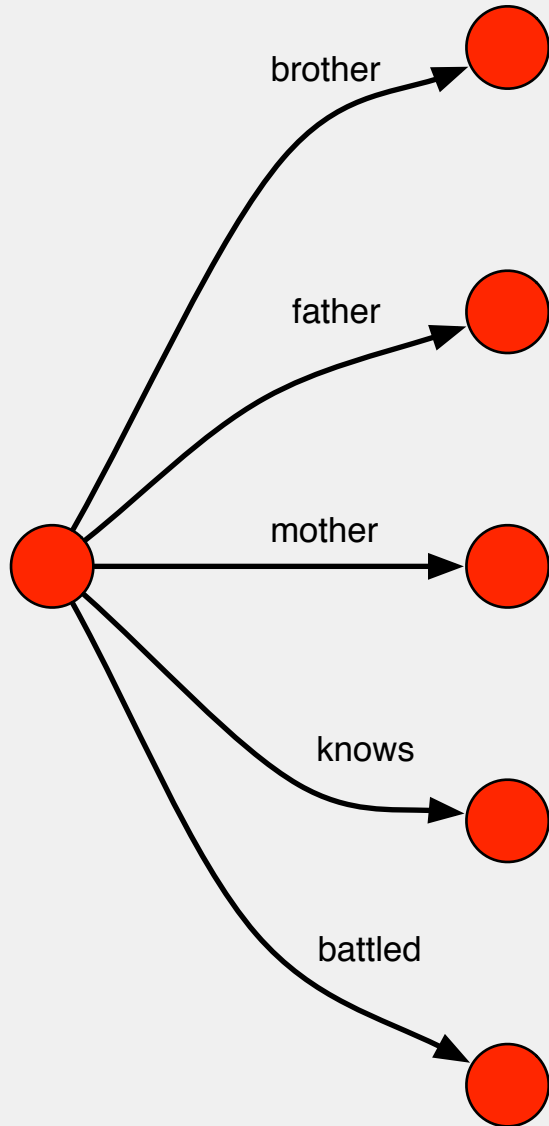
DISK-LEVEL SORTING/INDEXING



```
TitanLabel battled =  
    g.makeType().name("battled")  
    .primaryKey(time)
```

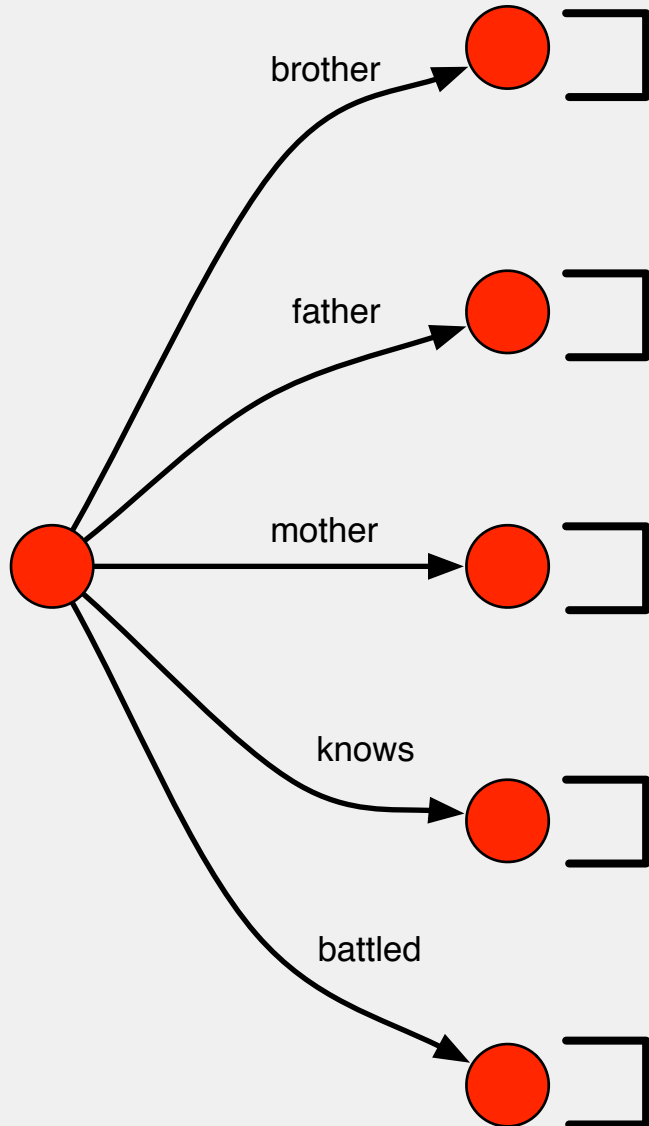
VERTEX-CENTRIC INDICES

DISK-LEVEL SORTING/INDEXING



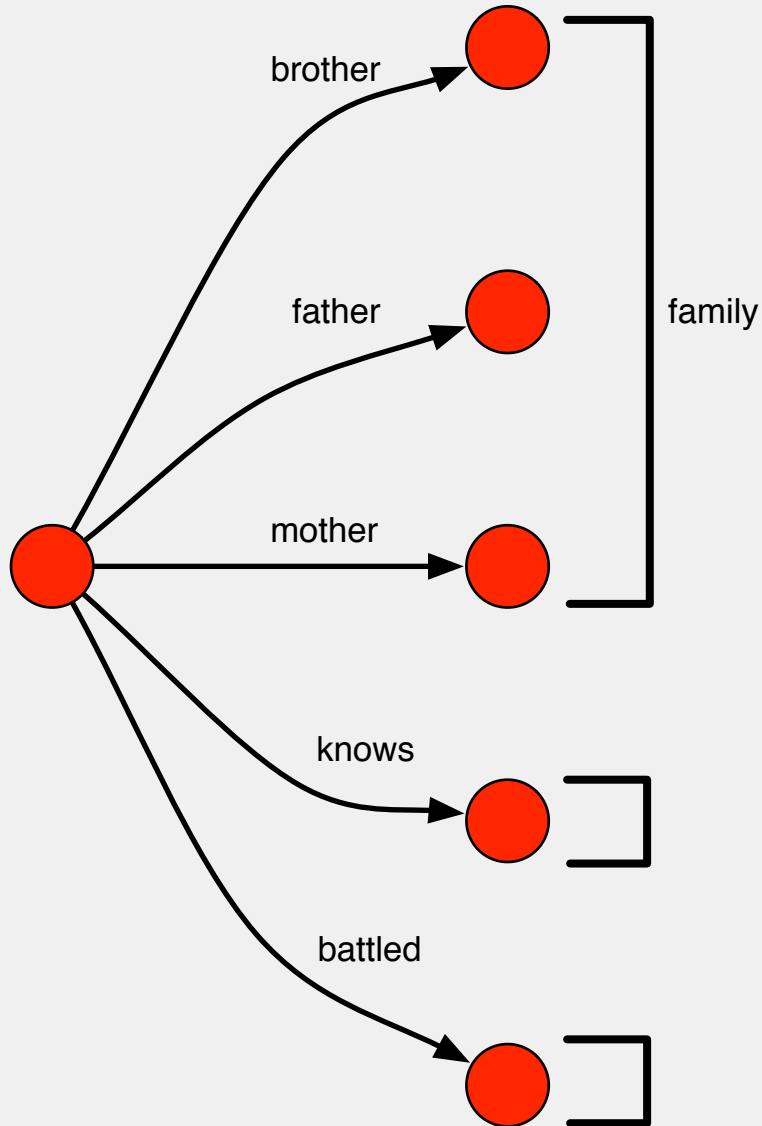
VERTEX-CENTRIC INDICES

DISK-LEVEL SORTING/INDEXING



VERTEX-CENTRIC INDICES

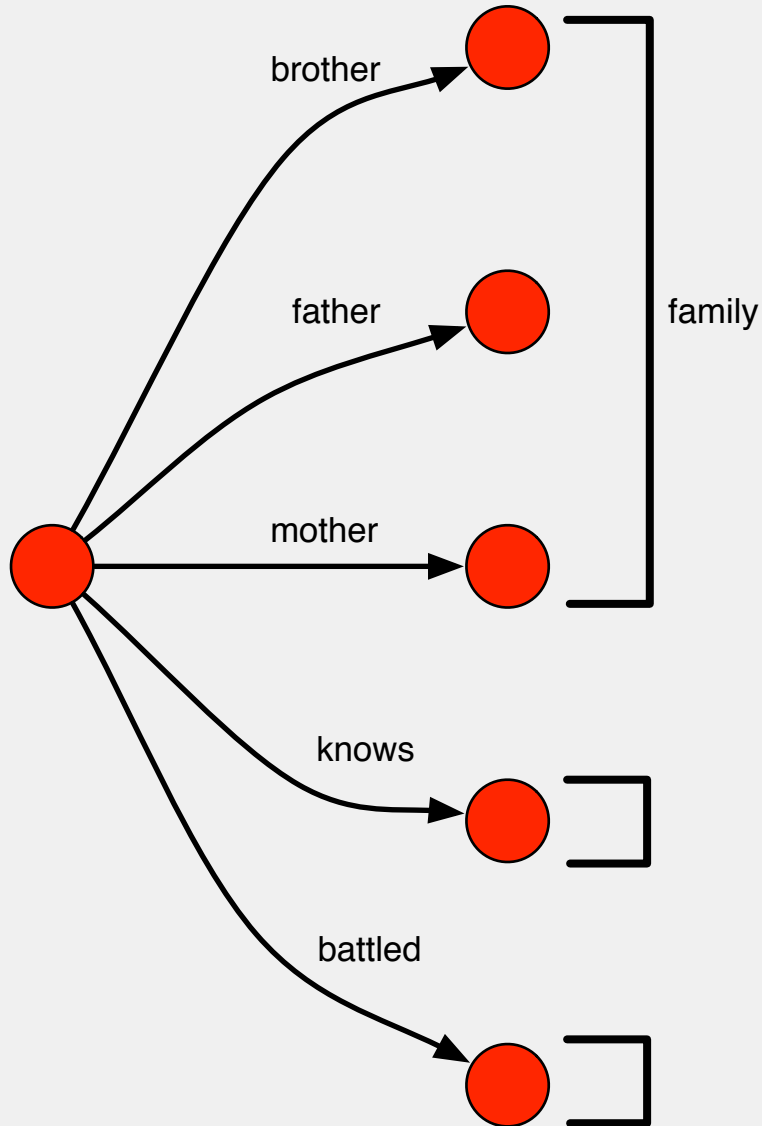
DISK-LEVEL SORTING/INDEXING



```
TypeGroup family =  
    TypeGroup.of(2, "family");  
TitanLabel father =  
    g.makeType().name("father")  
        .group(family).makeEdgeLabel();  
TitanLabel mother =  
    g.makeType().name("mother")  
        .group(family).makeEdgeLabel();  
TitanLabel brother =  
    g.makeType().name("brother")  
        .group(family).makeEdgeLabel();
```

VERTEX-CENTRIC INDICES

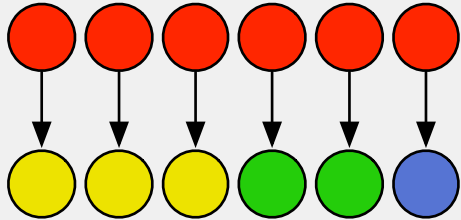
DISK-LEVEL SORTING/INDEXING



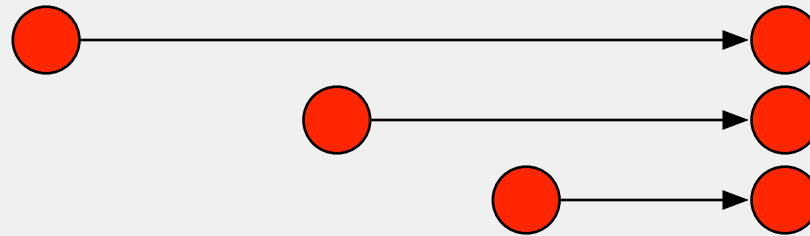
```
vertex.query().group("family")...
```

THAT IS HOW TITAN WORKS

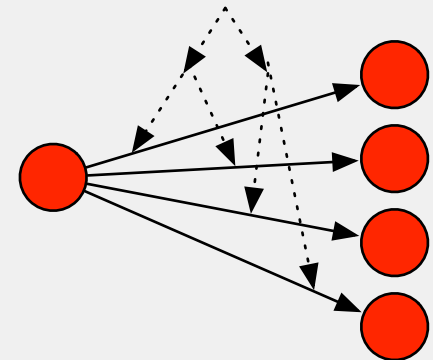
DATA MANAGEMENT



EDGE COMPRESSION



VERTEX-CENTRIC INDICES

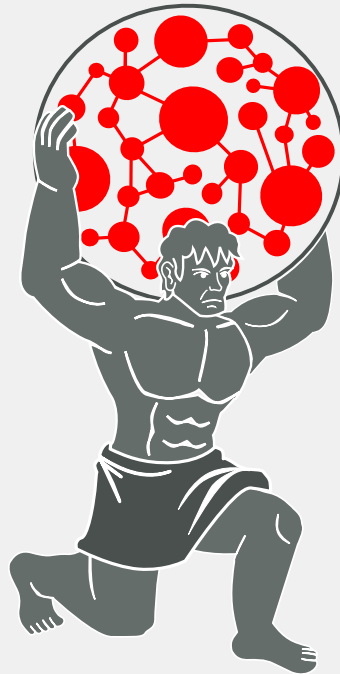


WHAT IF YOU WANTED TO CREATE
TWITTER FROM SCRATCH?



SIMULATING TWITTER

3 BILLION EDGES
100 MILLION VERTICES
10000 CONCURRENT USERS
50 MACHINES
1 GRAPH DATABASE



COMING JULY 2012

PART 3: THE FUTURE OF AURELIUS



MARKO A. RODRIGUEZ



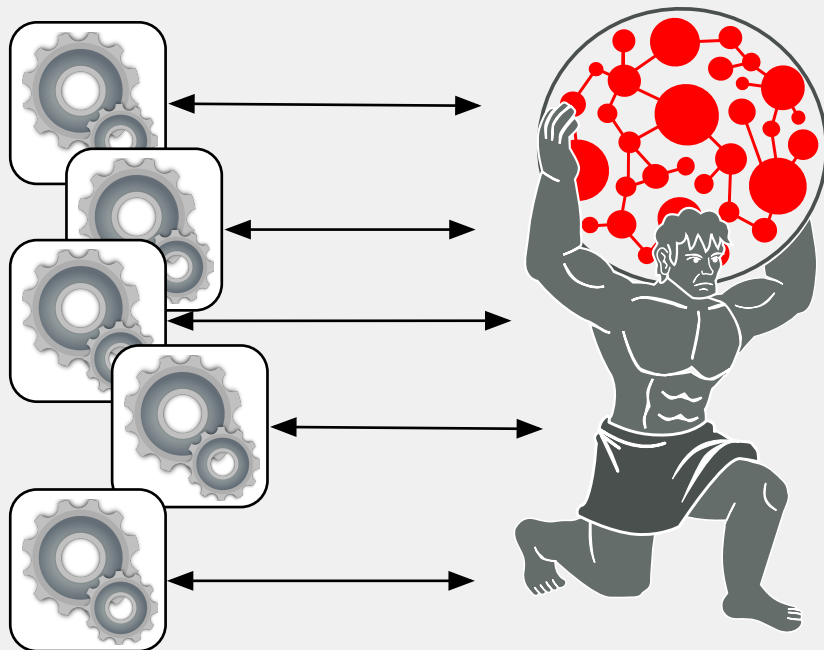
MATTHIAS BROECHELER



AURELIUS' GRAPH COMPUTING STORY

● Titan as the highly scalable, distributed graph database solution.

OLTP

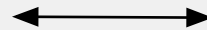
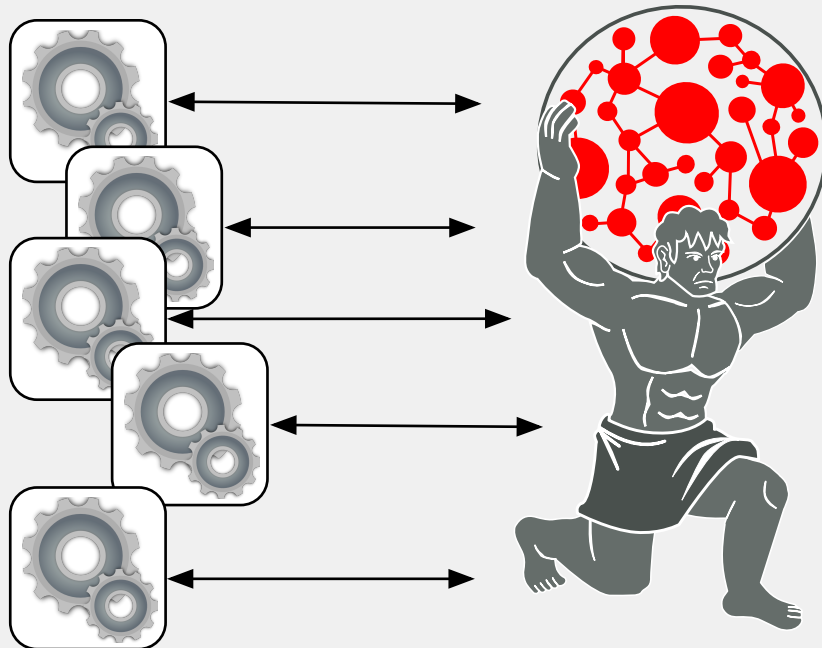




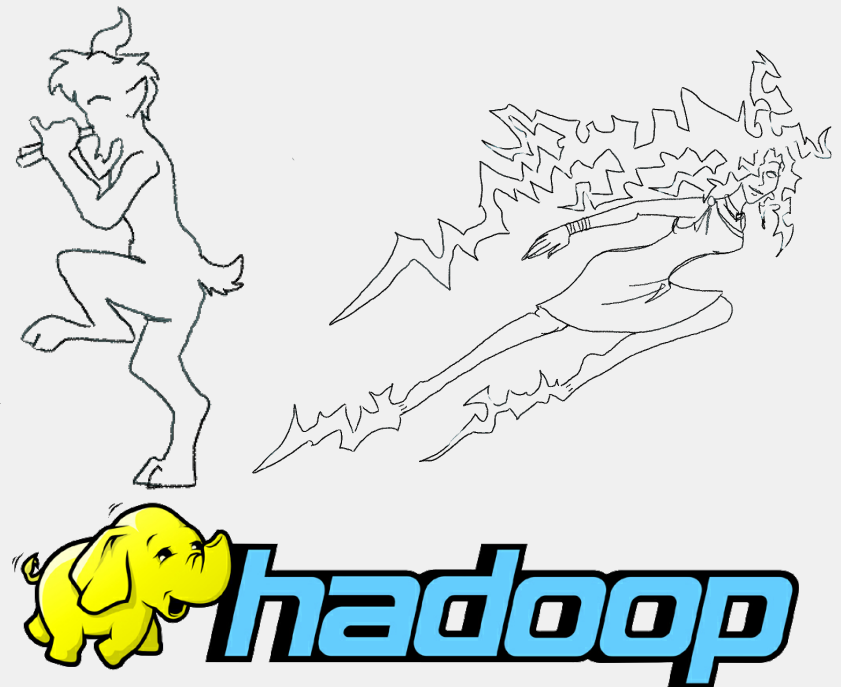
AURELIUS' GRAPH COMPUTING STORY

- Titan as the highly scalable, distributed graph database solution.
- Titan as the source (and potential sink) for other graph processing solutions.

OLTP



OLAP

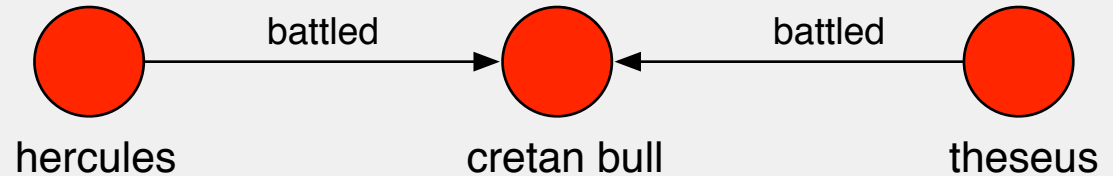


FAUNUS

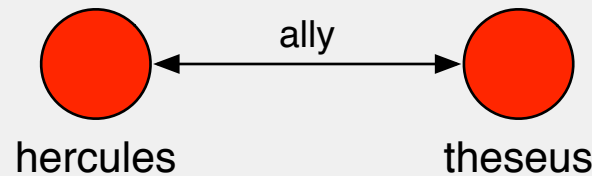
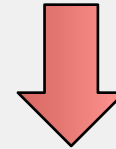


GOD OF HERDS

FAUNUS PATH ALGEBRA FOR HADOOP



$$\mathbf{A} \cdot \mathbf{A}^T \circ n(\mathbf{I})$$

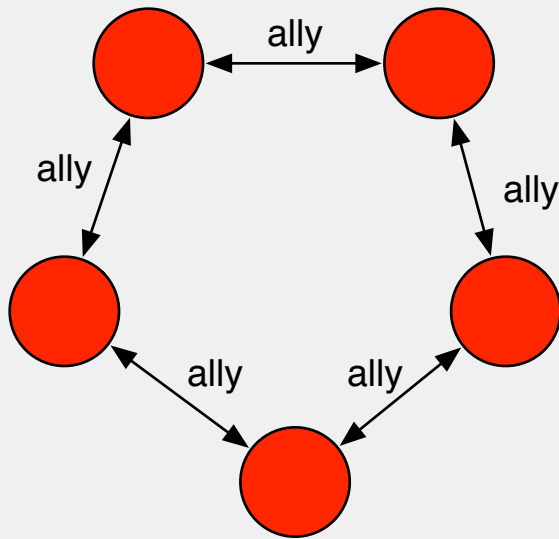


Derived graphs are **single-relational** and are typically **much smaller** than their multi-relational source. Therefore, derived graphs can be subjected to "textbook-style" graph algorithms in both a **meaningful** and **efficient** manner.

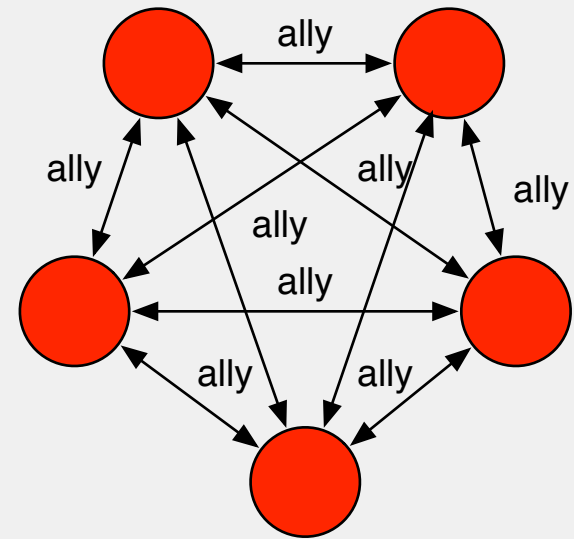
WHO IS THE MOST CENTRAL ALLY?

FAUNUS PATH ALGEBRA FOR HADOOP

$$\mathbf{B} = \mathbf{A} \cdot \mathbf{A}^\top \circ n(\mathbf{I})$$



$$\mathbf{B} \cdot \mathbf{B} \circ n(\mathbf{I})$$



"My allies' allies are my allies."

$$(\mathbf{A} \cdot \mathbf{A}^\top)^2 \circ n(\mathbf{I})$$

FAUNUS

PATH ALGEBRA FOR HADOOP



- Used for global graph operations.
- Implements the multi-relational path algebra as a collection of Map/Reduce operations
- Reduce a massive property graph into a smaller semantically-rich single-relational graph.

Project codename: TinkerPoop

- 
- Support for "HadoopGraph" and HDFS file formats



FULGORA



GODDESS OF LIGHTNING

FULGORA

AN EFFICIENT IN-MEMORY GRAPH ENGINE

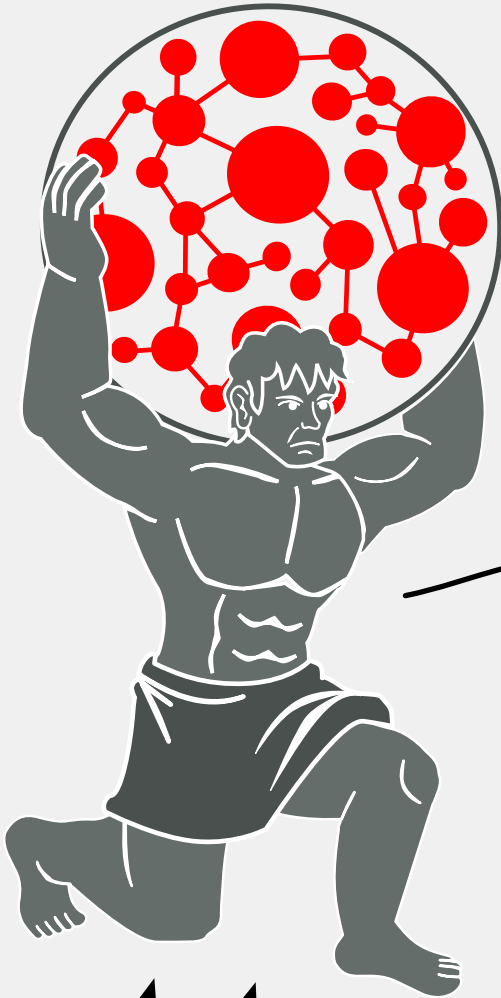


- Non-transactional, in-memory graph engine.
It is not a "database."
- Process ~90 billion edges in 68-Gigs of RAM
assuming a small world topology.
- Perform complex graph algorithms in-memory.
global graph analysis
multi-relational graph analysis



THE AURELIUS OLAP FLOW

**Stores a massive-scale
property graph**



**Generates a large-scale
single-relational graph**



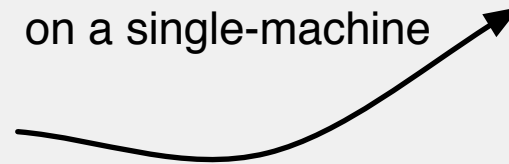
Map/Reduce



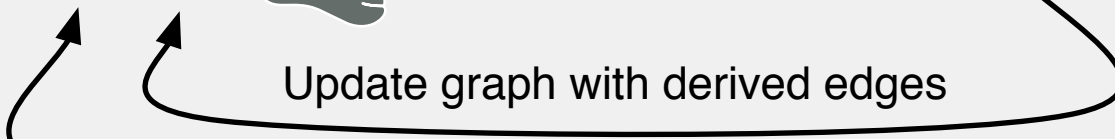
**Analyzes compressed, large-scale
single or multi-relational
graphs in memory**



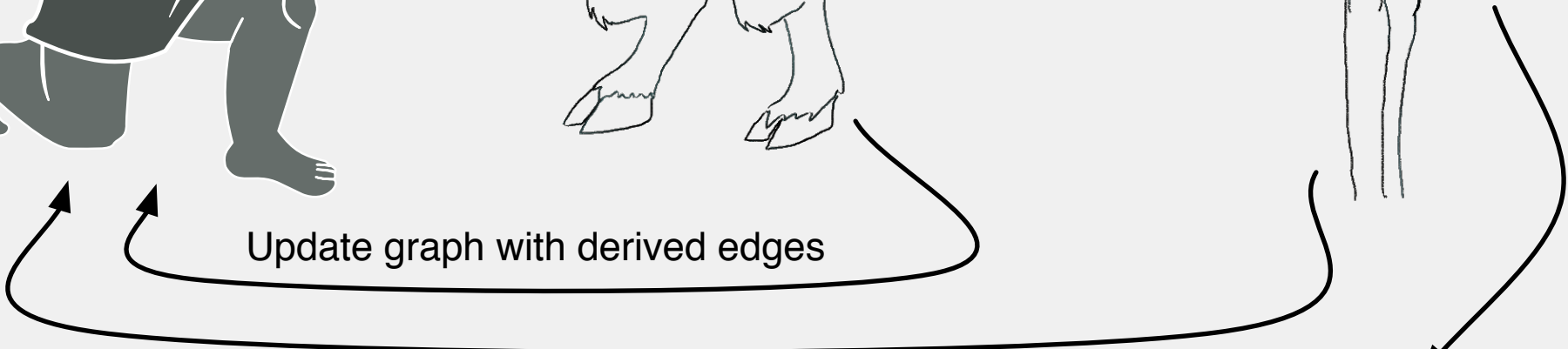
Load into RAM
on a single-machine



Update graph with derived edges



Update element properties with algorithm results



to a stats package

THE AURELIUS OLAP FLOW

**Stores a massive-scale
property graph**



**Generates a large-scale
single-relational graph**



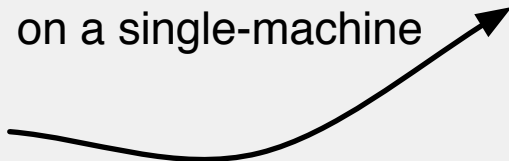
Map/Reduce



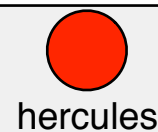
**Analyzes compressed, large-scale
single or multi-relational
graphs in memory**



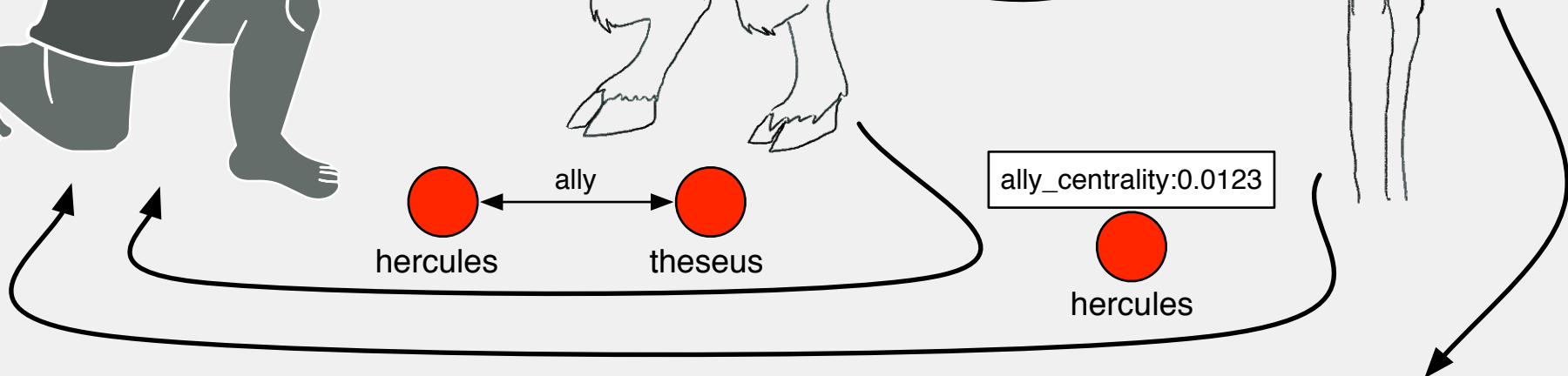
Load into RAM
on a single-machine



ally Centrality:0.0123

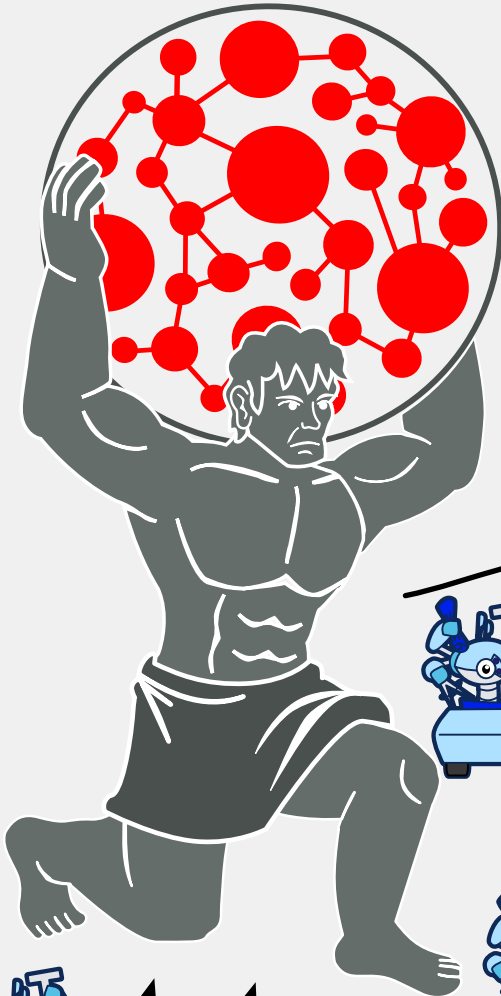


to a stats package



THE AURELIUS OLAP FLOW

**Stores a massive-scale
property graph**



**Generates a large-scale
single-relational graph**



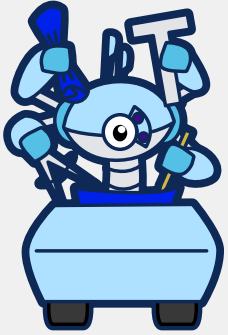
**Analyzes compressed, large-scale
single or multi-relational
graphs in memory**



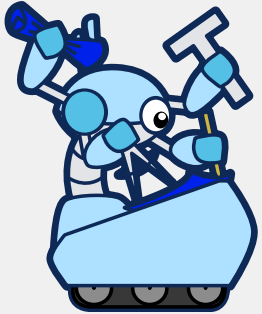
to a stats package



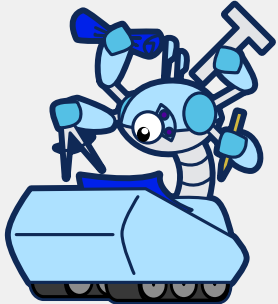
AURELIUS' USE OF BLUEPRINTS



Aurelius products use the Blueprints API so any graph product can communicate with any other graph product.

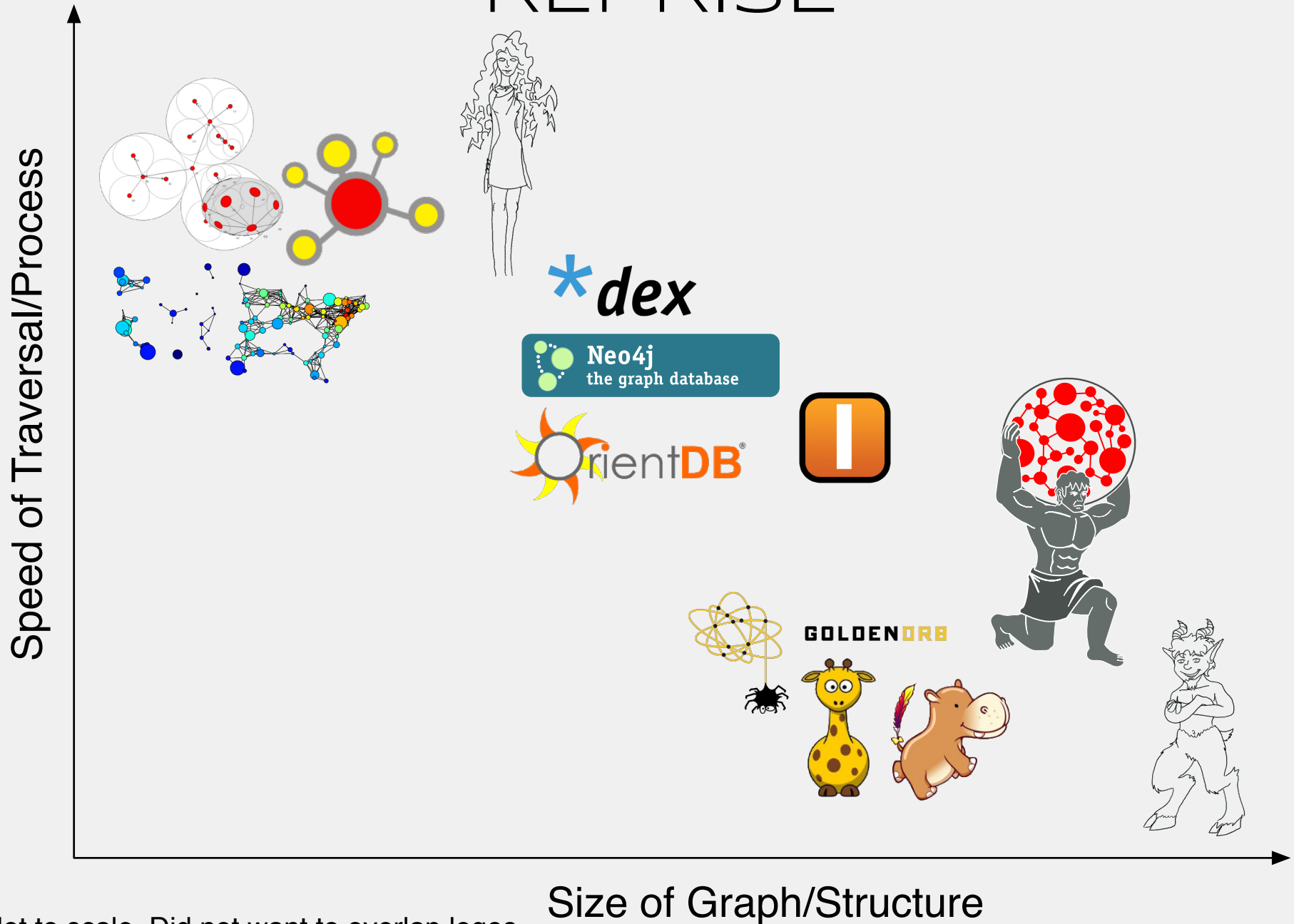


The code for graph databases, frameworks, algorithms, and batch-processing are written in terms of the Blueprints API.



Aurelius encourages developers to use Blueprints/ TinkerPop in order to grow a rich ecosystem of interoperable graph technologies.

THE GRAPH LANDSCAPE REPRISE



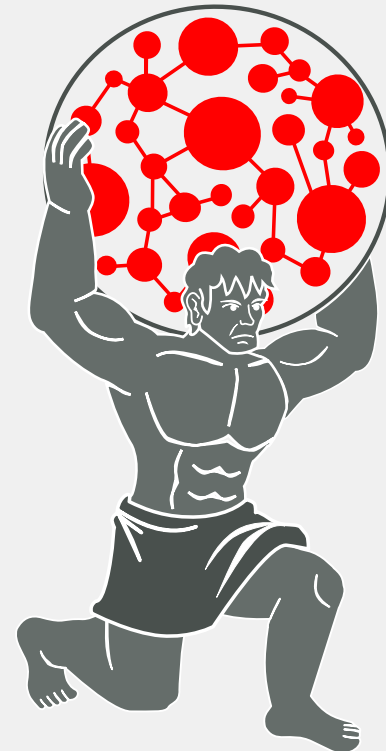
NEXT STEPS

Make use of and/or contribute to the free, open source Titan product.

Learn about applying graph theory and network science.



<http://thinkaurelius.com>



<http://thinkaurelius.github.com/titan/>



THANK YOU

CREDITS

PRESENTERS

MARKO A. RODRIGUEZ
MATTHIAS BROCHELER

FINANCIAL SUPPORT
PEARSON EDUCATION
AURELIUS

LOCATION PROVISIONS
JIVE SOFTWARE

MANY THANKS TO
DAN LAROCQUE
TINKERPOP COMMUNITY
STEPHEN MALLETT
BOBBY NORTON
KETRINA YIM